



Padrões, arquiteturas e implementações de design de nuvem

AWS Recomendações



AWS Recomendações: Padrões, arquiteturas e implementações de design de nuvem

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

As marcas comerciais e imagens de marcas da Amazon não podem ser usadas no contexto de nenhum produto ou serviço que não seja da Amazon, nem de qualquer maneira que possa gerar confusão entre os clientes ou que deprecie ou desprestigie a Amazon. Todas as outras marcas comerciais que não pertencem à Amazon pertencem a seus respectivos proprietários, que podem ou não ser afiliados, patrocinados pela Amazon ou ter conexão com ela.

Table of Contents

Introdução	1
Resultados de negócios desejados	2
Anti-corruption padrão de camada	3
Intenção	3
Motivação	3
Aplicabilidade	3
Problemas e considerações	4
Implementação	5
High-level arquitetura	5
Implementação usando AWS serviços	6
Código de exemplo	7
GitHub repositório	8
Conteúdo relacionado	9
Padrões de roteamento de API	10
Roteamento de nome de host	10
Caso de uso típico	10
Prós	11
Contras	11
Roteamento de caminhos	12
Caso de uso típico	12
Proxy reverso de serviço HTTP	12
API Gateway	14
CloudFront	16
Roteamento de cabeçalho HTTP	17
Prós	18
Contras	18
Padrão de disjuntor	19
Intenção	19
Motivação	19
Aplicabilidade	20
Problemas e considerações	20
Implementação	21
High-level arquitetura	21
Implementação usando AWS serviços	22

Código de exemplo	23
GitHub repositório	24
Referências do blog	25
Conteúdo relacionado	25
Padrão de fornecimento de eventos	26
Intenção	26
Motivação	26
Aplicabilidade	26
Problemas e considerações	27
Implementação	29
High-level arquitetura	29
Implementação usando serviços da AWS	31
Referências do blog	33
Padrão de arquitetura hexagonal	34
Intenção	34
Motivação	34
Aplicabilidade	34
Problemas e considerações	35
Implementação	35
High-level arquitetura	36
Implementação usando Serviços da AWS	37
Código de exemplo	38
Conteúdo relacionado	42
Vídeos	42
Publish-subscribe padrão	43
Intenção	43
Motivação	43
Aplicabilidade	43
Problemas e considerações	44
Implementação	45
High-level arquitetura	45
Implementação usando serviços da AWS	46
Workshop	48
Referências do blog	48
Conteúdo relacionado	48
Tente novamente com o padrão de recuo	49

Intenção	49
Motivação	49
Aplicabilidade	49
Problemas e considerações	49
Implementação	50
High-level arquitetura	50
Implementação usando AWS serviços	51
Código de exemplo	52
GitHub repositório	53
Conteúdo relacionado	53
Padrões saga	54
Coreografia saga	55
Orquestração da saga	55
Coreografia saga	56
Intenção	56
Motivação	57
Aplicabilidade	57
Problemas e considerações	58
Implementação	59
Conteúdo relacionado	62
Orquestração da saga	62
Intenção	62
Motivação	62
Aplicabilidade	63
Problemas e considerações	63
Implementação	64
Referências do blog	69
Conteúdo relacionado	69
Vídeos	69
Scatter-gather padrão	70
Intenção	70
Motivação	70
Aplicabilidade	70
Problemas e considerações	71
Implementação	72
High-level arquitetura	72

Implementação usando Serviços da AWS	75
Workshop	78
Referências do blog	78
Conteúdo relacionado	78
padrão strangler fig	79
Intenção	79
Motivação	79
Aplicabilidade	80
Problemas e considerações	80
Implementação	82
High-level arquitetura	82
Implementação usando AWS serviços	87
Workshop	91
Referências do blog	91
Conteúdo relacionado	91
Padrão de caixa de saída transacional	92
Intenção	92
Motivação	92
Aplicabilidade	92
Problemas e considerações	93
Implementação	93
High-level arquitetura	93
Implementação usando AWS serviços	94
Código de exemplo	99
Uso de uma tabela de outbox	99
Uso da captura de dados de alteração (CDC)	100
GitHub repositório	102
Recursos	103
Histórico do documento	104
.....	cvi

Padrões, arquiteturas e implementações de design de nuvem

Anitha Deenadayalan, Amazon Web Services (AWS)

Maio de 2024 ([histórico do documento](#))

Este guia fornece orientação para implementar padrões de design de modernização comumente usados usando serviços AWS. Um número cada vez maior de aplicativos modernos é projetado usando arquiteturas de microsserviços para obter escalabilidade, melhorar a velocidade de lançamento, reduzir o escopo do impacto das mudanças e reduzir a regressão. Isso leva a uma maior produtividade do desenvolvedor e maior agilidade, melhor inovação e maior foco nas necessidades comerciais. As arquiteturas de microsserviços também apoiam o uso da melhor tecnologia para o serviço e o banco de dados e promovem o código poliglota e a persistência poliglota.

Tradicionalmente, os aplicativos monolíticos são executados em um único processo, usam um armazenamento de dados e são executados em servidores que escalam verticalmente. Em comparação, os aplicativos de microsserviços modernos são refinados, têm domínios de falha independentes, são executados como serviços em toda a rede e podem usar mais de um armazenamento de dados, dependendo do caso de uso. Os serviços são escalados horizontalmente e uma única transação pode abranger vários bancos de dados. As equipes de desenvolvimento devem concentrar-se na comunicação de rede, na persistência poliglota, na escalabilidade horizontal, na consistência eventual e no tratamento de transações nos armazenamentos de dados ao desenvolver aplicativos usando arquiteturas de microsserviços. Portanto, os padrões de modernização são essenciais para resolver problemas comuns no desenvolvimento de aplicativos modernos e ajudam a acelerar a entrega de software.

Este guia fornece uma referência técnica para arquitetos de nuvem, líderes técnicos, proprietários de aplicativos e negócios e desenvolvedores que desejam escolher a arquitetura de nuvem certa para padrões de design com base nas melhores práticas bem arquitetadas. Cada padrão discutido neste guia aborda um ou mais cenários conhecidos em arquiteturas de microsserviços. O guia discute os problemas e considerações associados a cada padrão, fornece uma implementação arquitetônica de alto nível e descreve a implementação do padrão na AWS. Exemplos de código aberto do GitHub e links de workshops são fornecidos quando disponíveis.

O guia aborda os seguintes padrões:

- [Camada anticorrupção](#)
- [Padrões de roteamento da API:](#)
 - [Roteamento de nome de host](#)
 - [Roteamento de caminho](#)
 - [Roteamento de cabeçalho HTTP](#)
- [Disjuntor](#)
- [Fornecimento de eventos](#)
- [Arquitetura hexagonal](#)
- [Publicar/assinar](#)
- [Tente novamente com o recuo](#)
- [Padrões saga:](#)
 - [Coreografia saga](#)
 - [Orquestração saga](#)
- [Scatter-gather](#)
- [Strangler fig](#)
- [Caixa de saída transacional](#)

Resultados de negócios desejados

Usando os padrões discutidos neste guia para modernizar os seus aplicativos, você pode:

- Projete e implemente arquiteturas confiáveis, seguras e operacionalmente eficientes, otimizadas para custo e desempenho.
- Reduza o tempo de ciclo para casos de uso que exigem esses padrões, para que você possa se concentrar nos desafios específicos da organização.
- Acelere o desenvolvimento padronizando implementações de padrões usando os serviços da AWS.
- Ajude seus desenvolvedores a criar aplicativos modernos sem herdar dívidas técnicas.

Anti-corruption padrão de camada

Intenção

O padrão de camada anticorrupção (ACL) atua como uma camada de mediação que converte a semântica do modelo de domínio de um sistema para outro. Ele converte o modelo do contexto limitado upstream (monólito) em um modelo adequado ao contexto limitado downstream (microserviço) antes de consumir o contrato de comunicação estabelecido pela equipe ascendente. Esse padrão pode ser aplicável quando o contexto limitado downstream contém um subdomínio principal ou o modelo upstream é um sistema legado não modificável. Também reduz o risco de transformação e a interrupção dos negócios, evitando alterações nos chamadores quando suas chamadas precisam ser redirecionadas de forma transparente para o sistema de destino.

Motivação

Durante o processo de migração, quando uma aplicação monolítica é migrada para microserviços, pode haver mudanças na semântica do modelo de domínio do serviço recém-migrado. Quando os recursos do monólito são necessários para chamar esses microserviços, as chamadas devem ser roteadas para o serviço migrado sem exigir nenhuma alteração nos serviços de chamada. O padrão da ACL permite que o monólito chame os microserviços de forma transparente, atuando como um adaptador ou uma camada de fachada que converte as chamadas para a semântica mais recente.

Aplicabilidade

Considere usar esse padrão quando:

- Sua aplicação monolítica existente precisa se comunicar com uma função que foi migrada para um microserviço, e o modelo e a semântica do domínio de serviço migrado diferem do recurso original.
- Dois sistemas têm semânticas diferentes e precisam trocar dados, mas não é prático modificar um sistema para ser compatível com o outro sistema.
- Você quer usar uma abordagem rápida e simplificada para adaptar um sistema a outro com o mínimo impacto.
- Sua aplicação está se comunicando com um sistema externo.

Problemas e considerações

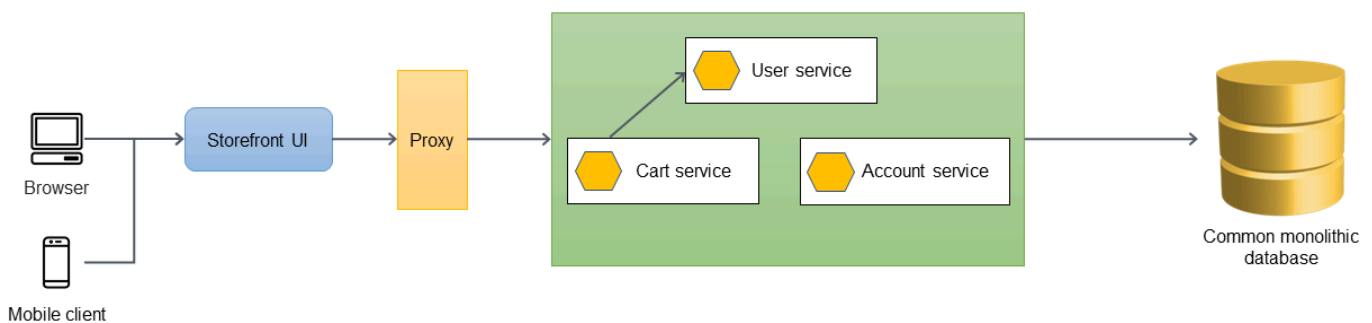
- Dependências da equipe: quando diferentes serviços em um sistema pertencem a equipes diferentes, a nova semântica do modelo de domínio nos serviços migrados pode levar a mudanças nos sistemas de chamada. No entanto, as equipes podem não conseguir fazer essas mudanças de forma coordenada, pois podem ter outras prioridades. A ACL separa os receptores e converte as chamadas para corresponder à semântica dos novos serviços, evitando assim a necessidade de os chamadores fazerem alterações no sistema atual.
- Sobrecarga operacional: o padrão da ACL exige esforço adicional para operar e manter. Esse trabalho inclui a integração da ACL com ferramentas de monitoramento e alerta, o processo de liberação e os processos de integração contínua e entrega contínua (CI/CD).
- Ponto único de falha: qualquer falha na ACL pode tornar o serviço de destino inacessível, causando problemas na aplicação. Para mitigar esse problema, você deve incorporar recursos de repetição e disjuntores. Consulte a nova [tentativa com os padrões de recuo](#) e [disjuntor](#) para entender mais sobre essas opções. A configuração de alertas e registros apropriados melhorará o tempo médio de resolução (MTTR).
- Dívida técnica: como parte de sua estratégia de migração ou modernização, considere se a ACL será uma solução transitória ou provisória ou uma solução de longo prazo. Se for uma solução provisória, você deve registrar a ACL como uma dívida técnica e desativá-la depois que todos os chamadores dependentes tiverem sido migrados.
- Latência: a camada adicional pode introduzir latência devido à conversão de solicitações de uma interface para outra. Recomendamos que você defina e teste a tolerância de performance em aplicações sensíveis ao tempo de resposta antes de implantar a ACL em ambientes de produção.
- Gargalo de escalabilidade: em aplicações de alta carga em que os serviços podem ser escalados até o pico de carga, a ACL pode se tornar um gargalo e causar problemas de escalabilidade. Se o serviço de destino for escalado sob demanda, você deverá projetar a ACL para escalar adequadamente.
- Service-specific ou implementação compartilhada: você pode criar a ACL como um objeto compartilhado para converter e redirecionar chamadas para vários serviços ou classes específicas de serviços. Leve em consideração a latência, a escalabilidade e a tolerância a falhas ao determinar o tipo de implementação da ACL.

Implementação

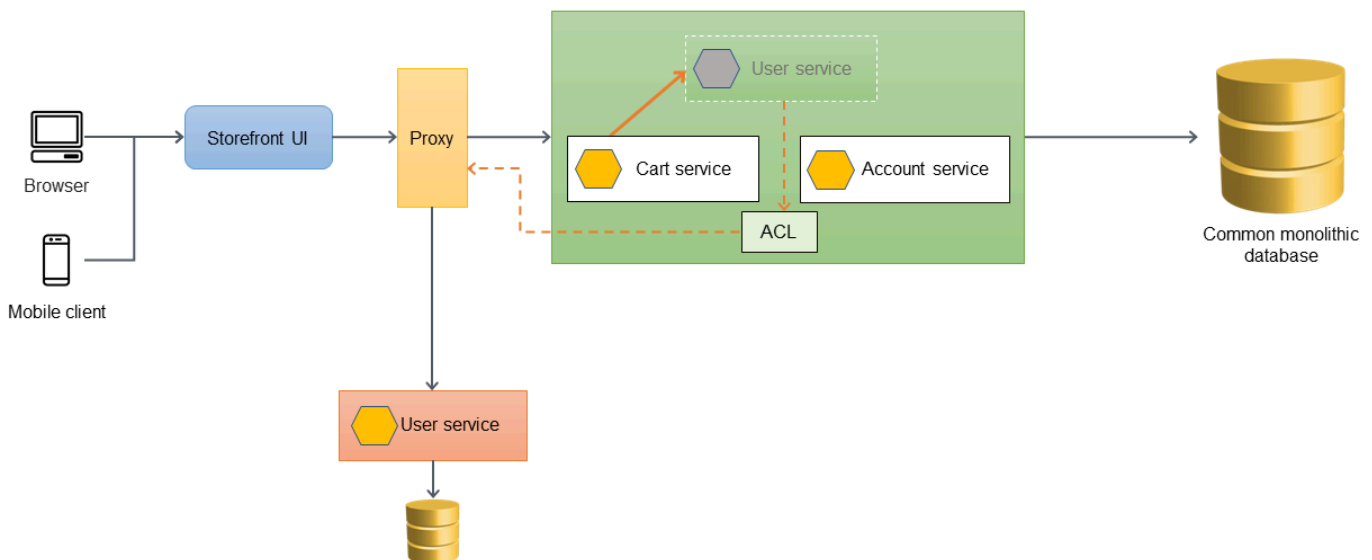
Você pode implementar a ACL dentro da sua aplicação monolítica como uma classe específica para o serviço que está sendo migrado ou como um serviço independente. A ACL deve ser desativada após a migração de todos os serviços dependentes para a arquitetura de microsserviços.

High-level arquitetura

No exemplo de arquitetura a seguir, uma aplicação monolítica tem três serviços: serviço de usuário, serviço de carrinho e serviço de conta. O serviço de carrinho depende do serviço do usuário, e a aplicação usa um banco de dados relacional monolítico.

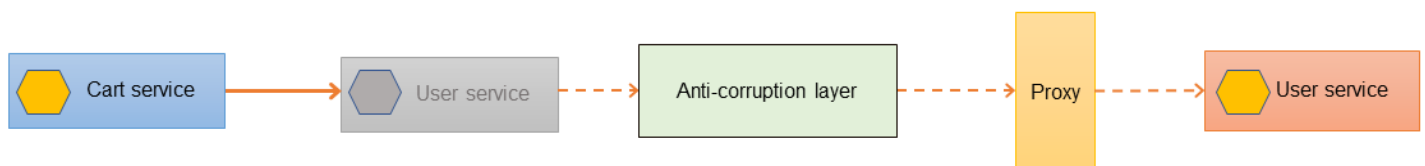


Na arquitetura a seguir, o serviço do usuário foi migrado para um novo microsserviço. O serviço de carrinho chama o serviço de usuário, mas a implementação não está mais disponível no monólito. Também é provável que a interface do serviço recém-migrado não corresponda à interface anterior, quando estava dentro da aplicação monolítica.



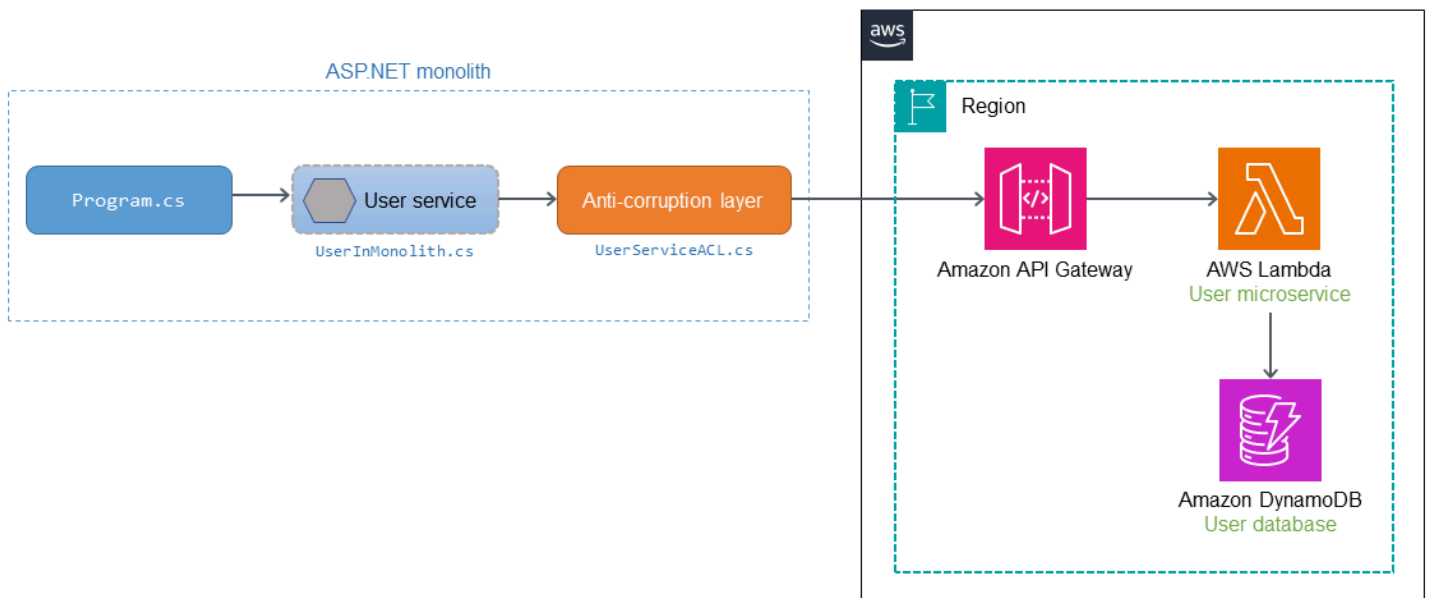
Se o serviço de carrinho precisar ligar diretamente para o serviço de usuário recém-migrado, isso exigirá alterações no serviço de carrinho e um teste completo da aplicação monolítica. Isso pode aumentar o risco de transformação e a interrupção dos negócios. O objetivo deve ser minimizar as alterações na funcionalidade existente da aplicação monolítica.

Nesse caso, recomendamos que você introduza uma ACL entre o serviço de usuário antigo e o serviço de usuário recém-migrado. A ACL funciona como um adaptador ou uma fachada que converte as chamadas na interface mais recente. A ACL pode ser implementada dentro da aplicação monolítica como uma classe (por exemplo, `UserServiceFacade` ou `UserServiceAdapter`) específica do serviço que foi migrado. A camada anticorrupção deverá ser desativada depois que todos os serviços dependentes tiverem sido migrados para a arquitetura de microsserviços.



Implementação usando AWS serviços

O diagrama a seguir mostra como você pode implementar esse exemplo de ACL usando serviços da AWS.



O microsserviço do usuário é migrado do aplicativo ASP.NET monolítico e implantado como uma função na AWS. [AWS Lambda](#) As chamadas para a função do Lambda são roteadas pelo [Amazon](#)

[API Gateway](#). A ACL é implantada no monólito para converter a chamada e se adaptar à semântica do microserviço do usuário.

Quando `Program.cs` chama o serviço do usuário (`UserInMonolith.cs`) dentro do monólito, a chamada é roteada para a ACL (`UserServiceACL.cs`). A ACL converte a chamada para a nova semântica e interface e chama o microserviço por meio do endpoint do API Gateway. O chamador (`Program.cs`) não está ciente da conversão e do roteamento que ocorrem no serviço ao usuário e na ACL. Como o chamador não está ciente das mudanças no código, há menos interrupções nos negócios e menor risco de transformação.

Código de exemplo

O trecho de código a seguir fornece as alterações no serviço original e a implementação de `UserServiceACL.cs`. Quando uma solicitação é recebida, o serviço de usuário original chama a ACL. A ACL converte o objeto de origem para corresponder à interface do serviço recém-migrado, chama o serviço e retorna a resposta ao chamador.

```
public class UserInMonolith: IUserInMonolith
{
    private readonly IACL _userServiceACL;
    public UserInMonolith(IACL userServiceACL) => (_userServiceACL) = (userServiceACL);
    public async Task<HttpStatusCode> UpdateAddress(UserDetails userDetails)
    {
        //Wrap the original object in the derived class
        var destUserDetails = new UserDetailsWrapped("user", userDetails);
        //Logic for updating address has been moved to a microservice
        return await _userServiceACL.CallMicroservice(destUserDetails);
    }
}

public class UserServiceACL: IACL
{
    static HttpClient _client = new HttpClient();
    private static string _apiGatewayDev = string.Empty;

    public UserServiceACL()
    {
        IConfiguration config = new
        ConfigurationBuilder().AddJsonFile(AppContext.BaseDirectory + "../../../
config.json").Build();
        _apiGatewayDev = config["APIGatewayURL:Dev"];
```

```
        _client.DefaultRequestHeaders.Accept.Add(new
MediaTypeWithQualityHeaderValue("application/json"));
    }
    public async Task<HttpStatusCode> CallMicroservice(ISourceObject details)
    {
        _apiGatewayDev += "/" + details.ServiceName;
        Console.WriteLine(_apiGatewayDev);

        var userDetails = details as UserDetails;
        var userMicroserviceModel = new UserMicroserviceModel();
        userMicroserviceModel.UserId = userDetails.UserId;
        userMicroserviceModel.Address = userDetails.AddressLine1 + ", " +
userDetails.AddressLine2;
        userMicroserviceModel.City = userDetails.City;
        userMicroserviceModel.State = userDetails.State;
        userMicroserviceModel.Country = userDetails.Country;

        if (Int32.TryParse(userDetails.ZipCode, out int zipCode))
        {
            userMicroserviceModel.ZipCode = zipCode;
            Console.WriteLine("Updated zip code");
        }
        else
        {
            Console.WriteLine("String could not be parsed.");
            return HttpStatusCode.BadRequest;
        }

        var jsonString =
JsonSerializer.Serialize<UserMicroserviceModel>(userMicroserviceModel);
        var payload = JsonSerializer.Serialize(userMicroserviceModel);
        var content = new StringContent(payload, Encoding.UTF8, "application/json");

        var response = await _client.PostAsync(_apiGatewayDev, content);
        return response.StatusCode;
    }
}
```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/anti-corruption-layer-pattern>.

Conteúdo relacionado

- [Padrão strangler fig](#)
- [Padrão de disjuntor](#)
- [Tente novamente com o padrão de recuo](#)

Padrões de roteamento de API

Em ambientes de desenvolvimento ágil, equipes autônomas (por ex., esquadrões e tribos) possuem um ou mais serviços que incluem muitos microsserviços. As equipes expõem esses serviços como APIs para permitir que seus consumidores interajam com seu grupo de serviços e ações.

Há três métodos principais para expor as APIs HTTP aos consumidores upstream usando nomes de host e caminhos:

Método	Descrição	Exemplo
Roteamento de nomes de host	Exponha cada serviço como um nome de host.	<code>billing.api.example.com</code>
Roteamento de caminhos	Exponha cada serviço como um caminho.	<code>api.example.com/billing</code>
Header-based roteamento	Exponha cada serviço como um cabeçalho HTTP.	<code>x-example-action: something</code>

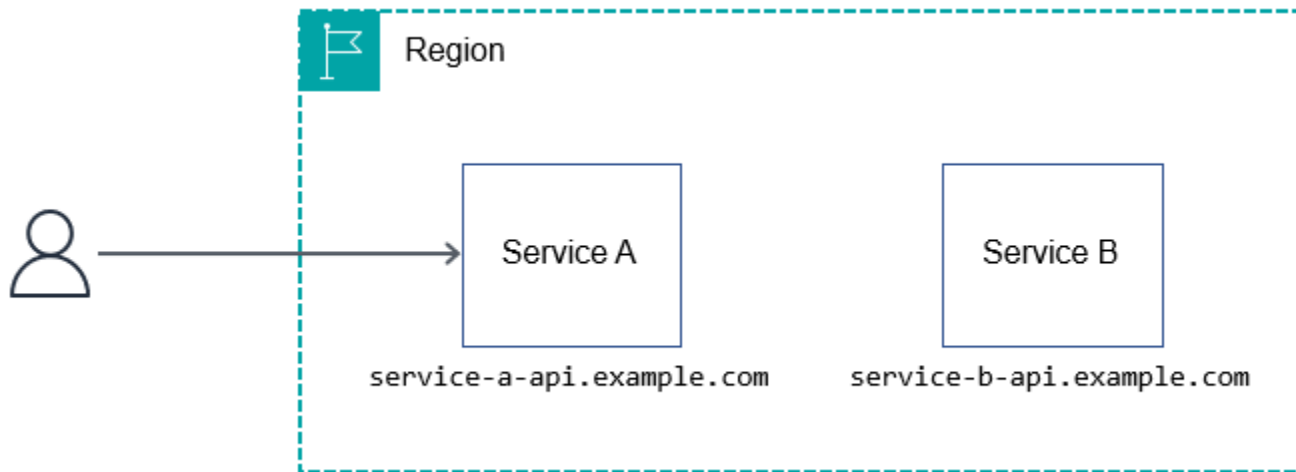
Esta seção descreve os casos de uso típicos desses três métodos de roteamento e suas vantagens e desvantagens para ajudá-lo a decidir qual método se adapta melhor aos seus requisitos e estrutura organizacional.

Padrão de roteamento por nome de host

O roteamento por nome de host é um mecanismo para isolar serviços de API, dando a cada API seu próprio nome de host; por exemplo, `service-a.api.example.com` ou `service-a.example.com`.

Caso de uso típico

O roteamento usando nomes de host reduz a quantidade de atrito nas versões, pois nada é compartilhado entre as equipes de serviço. As equipes são responsáveis por gerenciar tudo, desde entradas de DNS até operações de serviço em produção.



Prós

O roteamento de nome de host é, de longe, o método mais simples e escalável para roteamento de API HTTP. Você pode usar qualquer AWS serviço relevante para criar uma arquitetura que siga esse método – você pode criar uma arquitetura com [Amazon API Gateway](#), [AWS AppSync](#), [Application Load Balancers](#) e [Amazon Elastic Compute Cloud \(Amazon EC2\)](#) ou qualquer outro serviço. HTTP-compliant

As equipes podem usar o roteamento de nome de host para possuir totalmente seu subdomínio. Também facilita o isolamento, o teste e a orquestração de implantações específicas Regiões da AWS ou de versões; por exemplo, `ou. region.service-a.api.example.com` ou `dev.region.service-a.api.example.com`

Contras

Quando você usa o roteamento de nome de host, seus consumidores precisam se lembrar de nomes de host diferentes para interagir com cada API que você expõe. Você pode mitigar esse problema fornecendo um SDK de cliente. No entanto, os SDKs do cliente vêm com seu próprio conjunto de desafios. Por exemplo, eles precisam oferecer suporte a atualizações contínuas, vários idiomas, versionamento, comunicação de alterações significativas causadas por problemas de segurança ou correções de bugs, documentação e assim por diante.

Ao usar o roteamento de nome de host, você também precisa registrar o subdomínio ou domínio toda vez que criar um novo serviço.

Padrão de roteamento de caminhos

O roteamento por caminhos é o mecanismo de agrupar várias ou todas as APIs sob o mesmo nome de host e usar um URI de solicitação para isolar serviços; por exemplo, `api.example.com/service-a` ou `api.example.com/service-b`.

Caso de uso típico

A maioria das equipes opta por esse método porque deseja uma arquitetura simples — um desenvolvedor precisa se lembrar de apenas uma URL, como `api.example.com`, para interagir com a API HTTP. A documentação da API geralmente é mais fácil de digerir porque geralmente é mantida em conjunto em vez de ser dividida em diferentes portais ou PDFs.

Path-based o roteamento é considerado um mecanismo simples para compartilhar uma API HTTP. No entanto, envolve sobrecarga operacional, como configuração, autorização, integrações e latência adicional devido a vários saltos. Também requer processos maduros de gerenciamento de mudanças para garantir que uma configuração incorreta não interrompa todos os serviços.

AWS Ativado, há várias maneiras de compartilhar uma API e rotear de forma eficaz para o serviço correto. As seções a seguir abordam três abordagens: proxy reverso do serviço HTTP, API Gateway e Amazon CloudFront. Nenhuma das abordagens sugeridas para unificar os serviços de API depende dos serviços posteriores em execução no AWS. Os serviços podem ser executados em qualquer lugar sem problemas ou com qualquer tecnologia, desde que estejam HTTP-compatible.

Proxy reverso de serviço HTTP

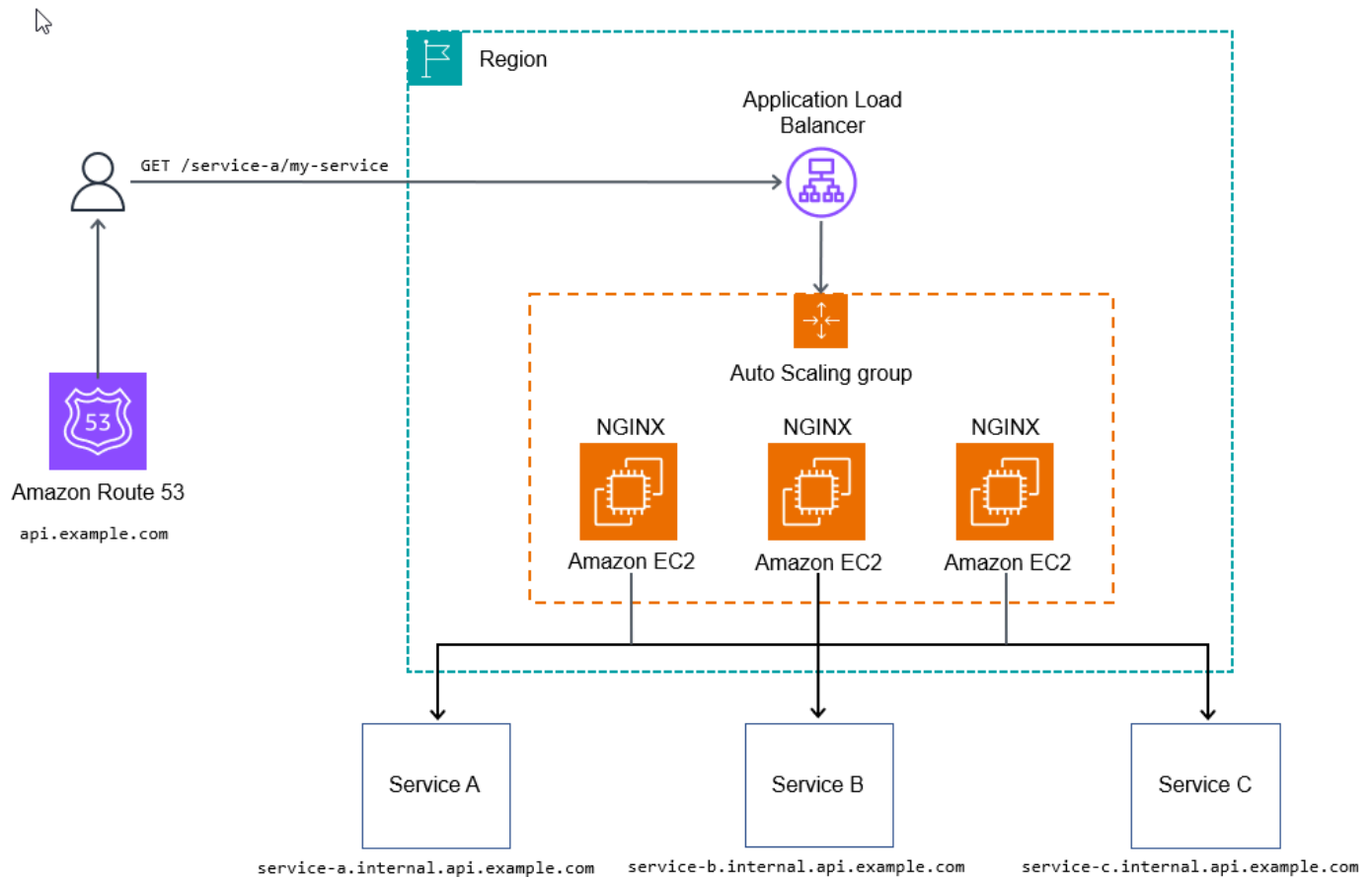
Você pode usar um servidor HTTP, como o [NGINX](#), para criar configurações de roteamento dinâmico. Em uma arquitetura [Kubernetes](#), você também pode criar uma regra de entrada para corresponder a um caminho para um serviço. (Este guia não aborda a entrada do Kubernetes; consulte a [documentação do Kubernetes](#) para obter mais informações.)

A configuração a seguir para o NGINX mapeia dinamicamente uma solicitação HTTP de `api.example.com/my-service/` para `my-service.internal.api.example.com`.

```
server {  
    listen 80;  
  
    location (^/[\w-]+)/(.*) {  
        proxy_pass $scheme://$1.internal.api.example.com/$2;  
    }  
}
```

```
}  
}
```

O diagrama a seguir ilustra o método de proxy reverso de serviço HTTP.



Essa abordagem pode ser suficiente para alguns casos de uso que não usam configurações adicionais para iniciar o processamento de solicitações, permitindo que a API posterior colete métricas e registros.

Para se preparar para a prontidão de produção operacional, você deve poder adicionar observabilidade para todos os níveis de sua pilha, adicionar configurações adicionais ou adicionar scripts para personalizar seu ponto de entrada de API para permitir atributos mais avançados, como limitação de taxa ou tokens de uso.

Prós

O objetivo final do método de proxy reverso de serviço HTTP é criar uma abordagem escalável e gerenciável para unificar APIs em um único domínio, de forma que pareça coerente para qualquer

consumidor de API. Essa abordagem também permite que suas equipes de serviço implantem e gerenciem suas próprias APIs, com o mínimo de sobrecarga após a implantação. AWS serviços gerenciados para rastreamento, como [AWS X-Ray](#) ou [AWS WAF](#), ainda são aplicáveis aqui.

Contras

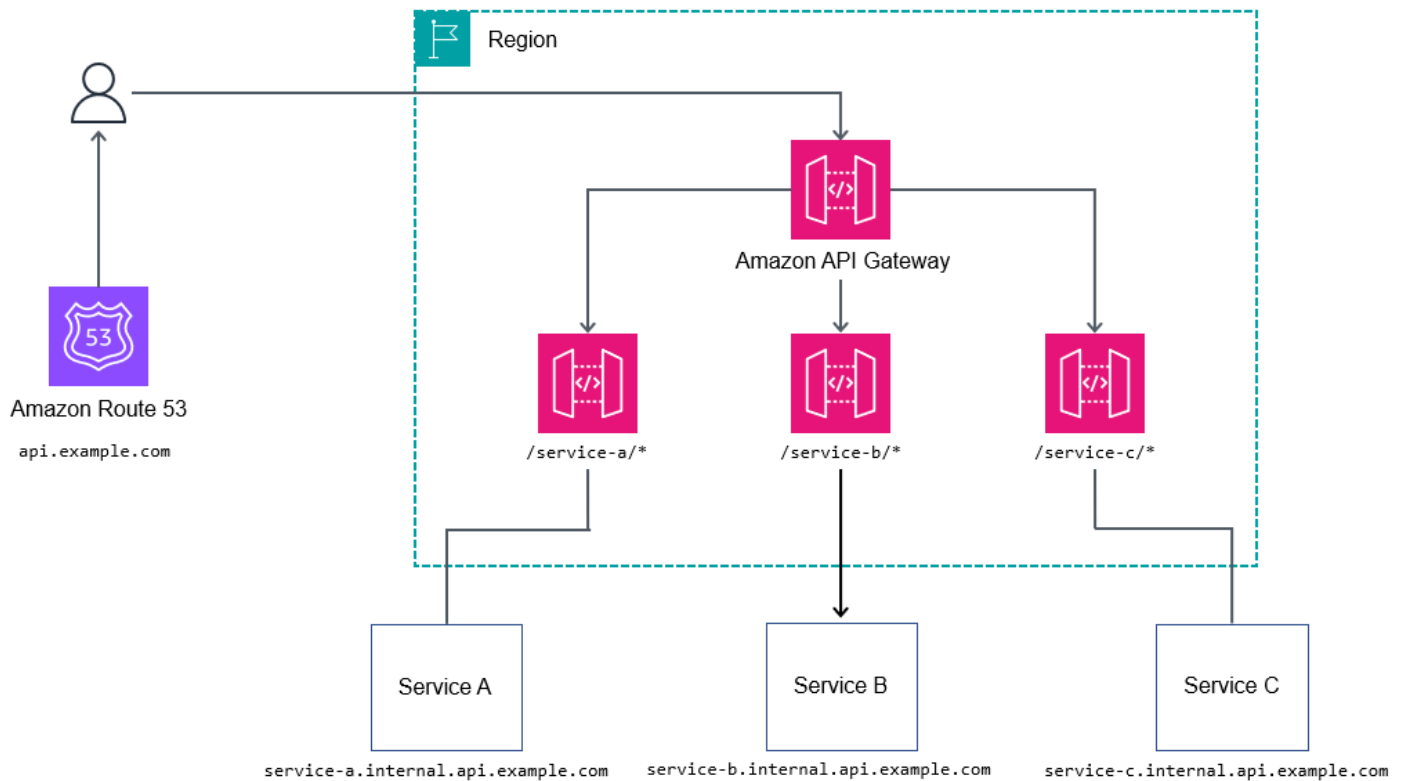
A principal desvantagem dessa abordagem são os testes e o gerenciamento extensivos dos componentes de infraestrutura necessários, embora isso possa não ser um problema se você tiver equipes de engenharia de confiabilidade de sites (SRE) instaladas.

Há um ponto de inflexão de custos com esse método. Em volumes baixos a médios, ele é mais caro do que alguns dos outros métodos discutidos neste guia. Em grandes volumes, é muito econômico (cerca de 100 mil transações por segundo ou mais).

API Gateway

O serviço [Amazon API Gateway](#) (APIs REST e APIs HTTP) pode rotear o tráfego de forma semelhante ao método de proxy reverso do serviço HTTP. Usar um gateway de API no modo proxy HTTP fornece uma maneira simples de agrupar muitos serviços em um ponto de entrada para o subdomínio de nível superior `api.example.com` e, em seguida, solicitações de proxy para o serviço aninhado; por exemplo, `billing.internal.api.example.com`.

Provavelmente não é uma boa ideia ser muito granular mapeando todos os caminhos em todos os serviços no gateway de API raiz ou principal. Em vez disso, opte por caminhos curinga, como o `/billing/*`, para encaminhar solicitações para o serviço de cobrança. Ao não mapear todos os caminhos no gateway de API raiz ou principal, você ganha mais flexibilidade em relação às suas APIs, porque não precisa atualizar o gateway de API raiz a cada alteração da API.



Prós

Para controlar fluxos de trabalho mais complexos, como alterar os atributos da solicitação, as APIs REST expõem a Linguagem de Modelo Apache Velocity (VTL) para permitir que você modifique a solicitação e a resposta. As APIs REST podem fornecer benefícios adicionais, como estes:

- [Autenticação N/Z com AWS Identity and Access Management \(IAM\), Amazon Cognito ou autorizadores AWS Lambda](#)
- [AWS X-Ray para rastreamento](#)
- [Integração com AWS WAF](#)
- [Limitação de taxa básica](#)
- Tokens de uso para agrupar consumidores em diferentes níveis (consulte [Acelerar solicitações de API para obter melhor throughput](#) na documentação do API Gateway)

Contras

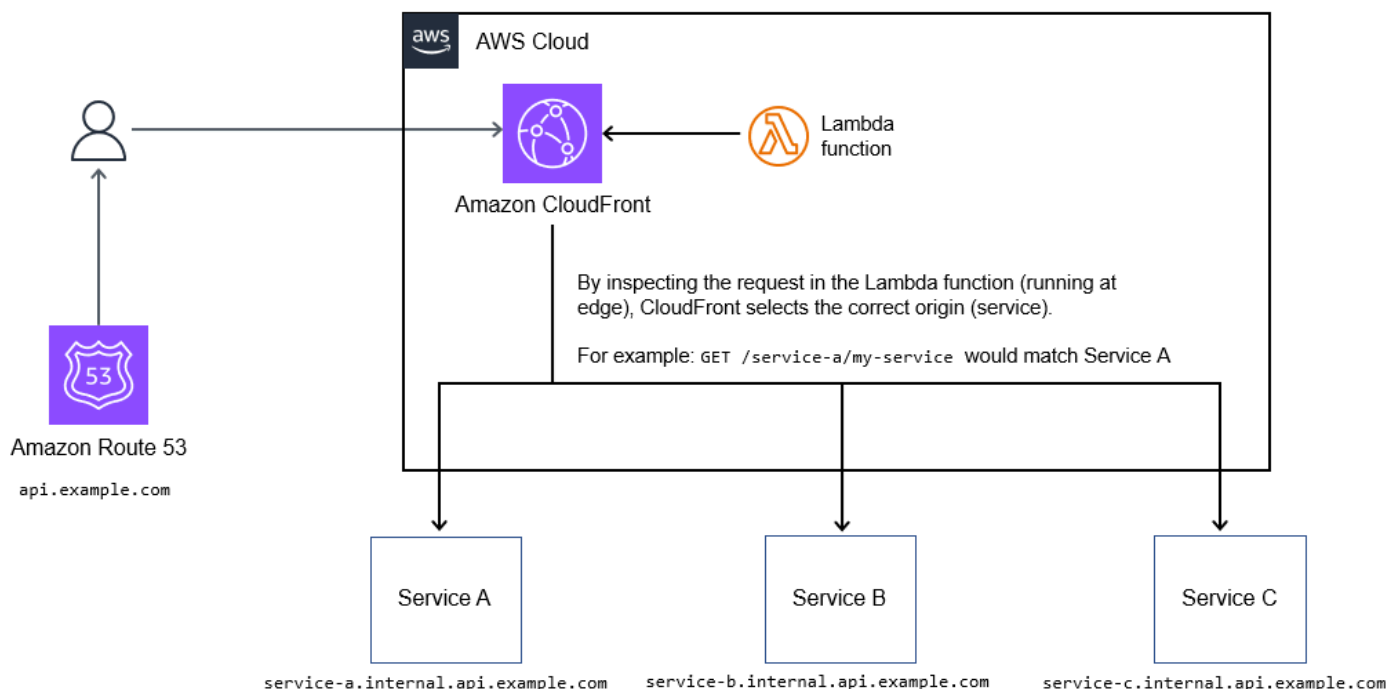
Em grandes volumes, o custo pode ser um problema para alguns usuários.

CloudFront

Você pode usar o [recurso de seleção dinâmica de origem](#) na [Amazon CloudFront](#) para selecionar condicionalmente uma origem (um serviço) para encaminhar a solicitação. Você pode usar esse atributo para rotear vários serviços por meio de um único nome de host, como `api.example.com`.

Caso de uso típico

A lógica de roteamento vive como código dentro da função Lambda @Edge, portanto, ela oferece suporte a mecanismos de roteamento altamente personalizáveis, A/B como testes, lançamentos canários, sinalização de recursos e reescrita de caminhos. Isso é ilustrado no diagrama a seguir.



Prós

Se você precisa armazenar em cache as respostas da API, esse método é uma boa maneira de unificar uma coleção de serviços em um único endpoint. É um método econômico para unificar coleções de APIs.

Além disso, CloudFront oferece suporte à [criptografia em nível de campo](#), bem como à integração com limitação básica AWS WAF de taxa e ACLs básicas.

Contras

Esse método suporta no máximo 250 origens (serviços) que podem ser unificadas. Esse limite é suficiente para a maioria das implantações, mas pode causar problemas com um grande número de APIs à medida que você aumenta seu portfólio de serviços.

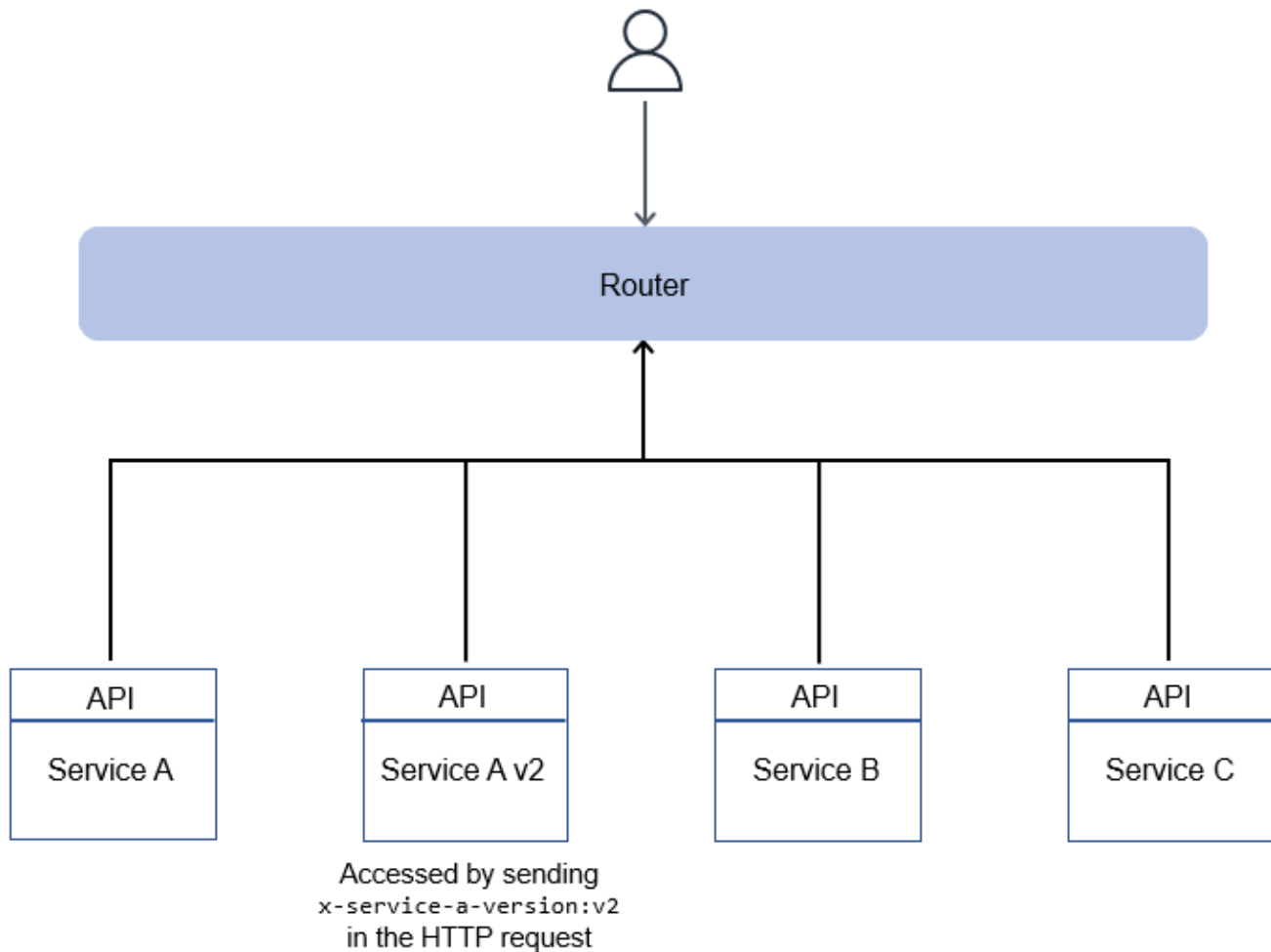
Atualmente, a atualização das funções do Lambda @Edge leva alguns minutos. CloudFront também leva até 30 minutos para concluir a propagação das alterações em todos os pontos de presença. Em última análise, isso bloqueia novas atualizações até que elas sejam concluídas.

Padrão de roteamento de cabeçalho HTTP

Header-based o roteamento permite que você direcione o serviço correto para cada solicitação especificando um cabeçalho HTTP na solicitação HTTP. Por exemplo, enviar o cabeçalho `x-service-a-action: get-thing` permitiria `get thing` a partir de Service A. O caminho da solicitação ainda é importante, pois oferece orientação sobre em qual atributo você está tentando trabalhar.

Além de usar o roteamento de cabeçalho HTTP para ações, você pode usá-lo como um mecanismo para roteamento de versões, habilitando sinalizadores de recursos, A/B testes ou necessidades semelhantes. Na realidade, você provavelmente usará o roteamento de cabeçalho com um dos outros métodos de roteamento para criar APIs robustas.

A arquitetura do roteamento de cabeçalho HTTP normalmente tem uma fina camada de roteamento na frente dos microsserviços que encaminha para o serviço correto e retorna uma resposta, conforme ilustrado no diagrama a seguir. Essa camada de roteamento pode cobrir todos os serviços ou apenas alguns serviços para habilitar uma operação, como roteamento com base na versão.



Prós

As alterações de configuração exigem o mínimo esforço e podem ser automatizadas facilmente. Esse método também é flexível e oferece suporte a formas criativas de expor somente operações específicas que você desejaria de um serviço.

Contras

Assim como no método de roteamento de nome de host, o roteamento de cabeçalho HTTP pressupõe que você tenha controle total sobre o cliente e possa manipular cabeçalhos HTTP personalizados. Proxies, redes de entrega de conteúdo (CDNs) e balanceadores de carga podem limitar o tamanho do cabeçalho. Embora seja improvável que isso seja uma preocupação, pode ser um problema dependendo de quantos cabeçalhos e cookies você adiciona.

Padrão de disjuntor

Intenção

O padrão do disjuntor pode impedir que um serviço de chamada repita uma chamada para outro serviço (receptor) quando a chamada já gerou repetidos tempos limite ou falhas. O padrão também é usado para detectar quando o serviço do receptor está funcional novamente.

Motivação

Quando vários microserviços colaboram para lidar com solicitações, um ou mais serviços podem ficar indisponíveis ou apresentar alta latência. Quando aplicações complexas usam microserviços, uma interrupção em um microserviço pode levar à falha da aplicação. Os microserviços se comunicam por meio de chamadas de procedimentos remotos, e erros transitórios podem ocorrer na conectividade da rede, causando falhas. (Os erros transitórios podem ser tratados usando o padrão de [repetição com recuo](#).) Durante a execução síncrona, a cascata de tempos limite ou falhas pode causar uma experiência ruim para o usuário.

No entanto, em algumas situações, as falhas podem levar mais tempo para serem resolvidas, por exemplo, quando o serviço do receptor está inativo ou uma contenção no banco de dados resulta em tempos limite. Nesses casos, se o serviço de chamada repetir as chamadas repetidamente, essas novas tentativas poderão resultar em contenção na rede e no consumo do grupo de threads do banco de dados. Além disso, se vários usuários tentarem a aplicação novamente e de forma repetida, isso piorará o problema e poderá causar degradação da performance de toda a aplicação.

O padrão do disjuntor foi popularizado por Michael Nygard em seu livro *Release It* (Nygard 2018). Esse padrão de design pode impedir que um serviço de chamada repita uma chamada de serviço que já tenha gerado repetidos tempos limite ou falhas. Ele também pode detectar quando o serviço do receptor está funcional novamente.

Os objetos do disjuntor funcionam como disjuntores elétricos que interrompem automaticamente a corrente quando há uma anormalidade no circuito. Os disjuntores elétricos desligam ou interrompem o fluxo da corrente quando há uma falha. Da mesma forma, o objeto do disjuntor está situado entre o chamador e o serviço do receptor e dispara se o receptor não estiver disponível.

As [falácias da computação distribuída](#) são um conjunto de afirmações feitas por Peter Deutsch e outros da Sun Microsystems. Eles dizem que programadores que são novos em aplicações

distribuídas invariavelmente fazem suposições falsas. A confiabilidade da rede, as expectativas de latência zero e as limitações de largura de banda resultam em aplicações de software escritas com o mínimo de tratamento de erros de rede.

Durante uma interrupção na rede, as aplicações podem esperar por uma resposta indefinidamente e consumir continuamente os recursos da aplicação. Deixar de repetir as operações quando a rede estiver disponível também pode levar à degradação da aplicação. Se as chamadas de API para um banco de dados ou um serviço externo expirarem devido a problemas de rede, chamadas repetidas sem disjuntor podem afetar o custo e a performance.

Aplicabilidade

Use esse padrão quando:

- O serviço de chamadas faz uma chamada que provavelmente falhará.
- A alta latência exibida pelo serviço do receptor (por exemplo, quando as conexões do banco de dados são lentas) gera tempos limite no serviço do chamador.
- O serviço do chamador faz uma chamada síncrona, mas o serviço do receptor não está disponível ou apresenta alta latência.

Problemas e considerações

- Implementação independente de serviço: para evitar o excesso de código, recomendamos que você implemente o objeto disjuntor de forma independente de microsserviços. API-driven
- Fechamento do circuito pelo receptor: quando o receptor se recupera de um problema ou falha de performance, ele pode atualizar o status do circuito para CLOSED. Esta é uma extensão do padrão do disjuntor e poderá ser implementada se seu objetivo de tempo de recuperação (RTO) exigir.
- Chamadas multithreaded: o valor do tempo limite de expiração é definido como o período de tempo em que o circuito permanece desligado antes que as chamadas sejam roteadas novamente para verificar a disponibilidade do serviço. Quando o serviço do receptor é chamado em vários threads, a primeira chamada que falhou define o valor do tempo limite de expiração. Sua implementação deve garantir que as chamadas subsequentes não alterem o tempo limite de expiração indefinidamente.
- Forçar a abertura ou o fechamento do circuito: os administradores do sistema devem ter a capacidade de abrir ou fechar um circuito. Isso pode ser feito atualizando o valor do tempo limite de expiração na tabela do banco de dados.

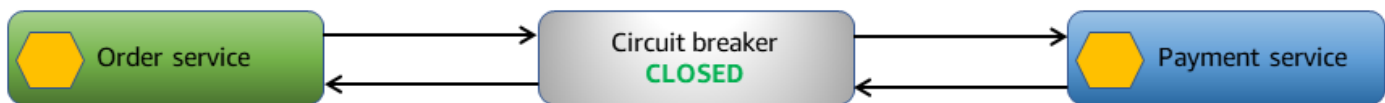
- Observabilidade: a aplicação deve ter o registro em log configurado para identificar as chamadas que falham quando o disjuntor está aberto.

Implementação

High-level arquitetura

No exemplo a seguir, o chamador é o serviço de pedidos e o receptor é o serviço de pagamento.

Quando não há falhas, o serviço de pedidos roteia todas as chamadas para o serviço de pagamento pelo disjuntor, conforme mostra o diagrama a seguir.



Se o serviço de pagamento atingir o tempo limite, o disjuntor poderá detectar o tempo limite e rastrear a falha.



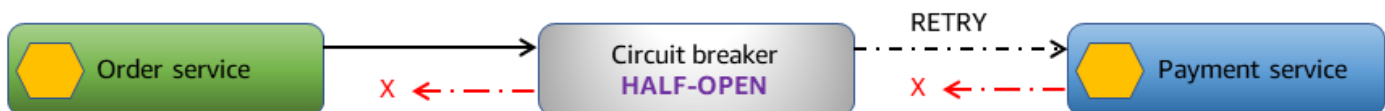
Circuit breaker with payment service failure

Se os tempos limite excederem um limite especificado, a aplicação abrirá o circuito. Quando o circuito está aberto, o objeto do disjuntor não roteia as chamadas para o serviço de pagamento. Ele retorna uma falha imediata quando o serviço de pedidos chama o serviço de pagamento.



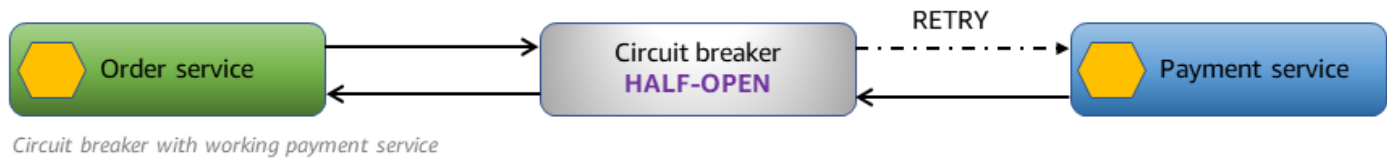
Circuit breaker stops routing to payment service

O objeto do disjuntor tenta periodicamente verificar se as chamadas para o serviço de pagamento foram bem-sucedidas.



Circuit breaker periodically retries payment service

Quando a chamada para o serviço de pagamento é bem-sucedida, o circuito é fechado e todas as chamadas adicionais são roteadas para o serviço de pagamento novamente.



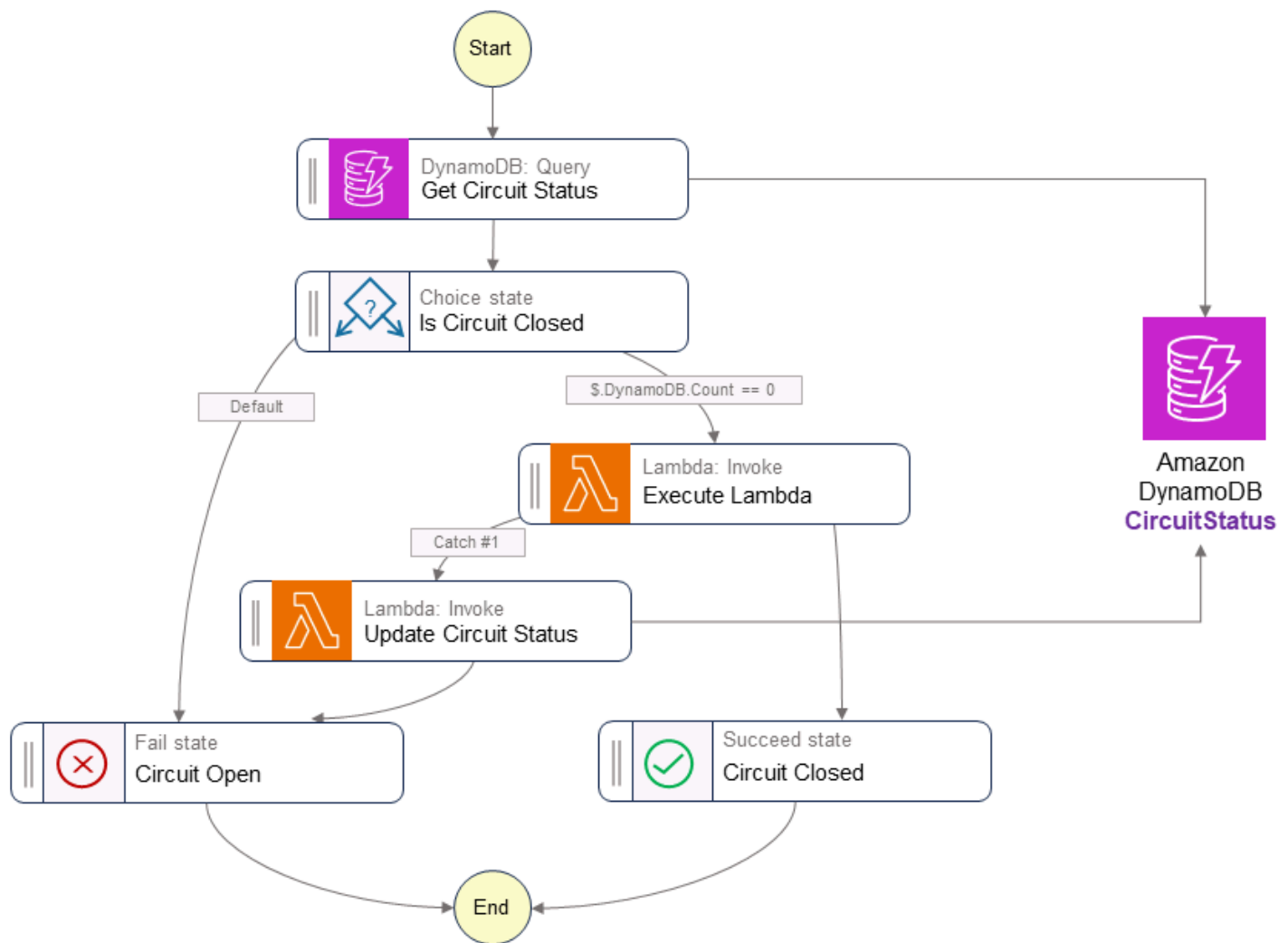
Implementação usando AWS serviços

A solução de amostra usa fluxos de trabalho expressos no [AWS Step Functions](#) para implementar o padrão do disjuntor. A máquina de estado do Step Functions permite que você configure os recursos de repetição e o fluxo de controle baseado em decisões necessários para a implementação do padrão.

A solução também usa uma tabela do [Amazon DynamoDB](#) como armazenamento de dados para rastrear o status do circuito. Isso pode ser substituído por um armazenamento de dados na memória, como o [Amazon ElastiCache \(Redis OSS\)](#), para melhor desempenho.

Quando um serviço deseja chamar outro serviço, ele inicia o fluxo de trabalho com o nome do serviço do receptor. O fluxo de trabalho obtém o status do disjuntor na tabela `CircuitStatus` do DynamoDB, que armazena os serviços atualmente degradados. Se `CircuitStatus` contiver um registro não expirado para o receptor, o circuito estará aberto. O fluxo de trabalho do Step Functions retorna uma falha imediata e tem como saída um estado `FAIL`.

Se a tabela `CircuitStatus` não contiver um registro para o receptor ou contiver um registro expirado, o serviço estará operacional. A etapa `ExecuteLambda` na definição da máquina de estado chama a função do Lambda que é enviada por meio de um valor de parâmetro. Se a chamada for bem-sucedida, o fluxo de trabalho do Step Functions será encerrado com um estado `SUCCESS`.



Se a chamada de serviço falhar ou ocorrer um tempo limite, a aplicação tentará novamente com um recuo exponencial por um número definido de vezes. Se a chamada de serviço falhar após as novas tentativas, o fluxo de trabalho vai inserir um registro na tabela `CircuitStatus` do serviço com um `ExpiryTimeStamp`, e o fluxo de trabalho sairá com um estado `FAIL`. As chamadas subsequentes para o mesmo serviço retornarão uma falha imediata, desde que o disjuntor esteja aberto. A etapa `Get Circuit Status` na definição da máquina de estado verifica a disponibilidade do serviço com base no valor `ExpiryTimeStamp`. Os itens expirados são excluídos da tabela `CircuitStatus` usando o recurso de tempo de vida (TTL) do DynamoDB.

Código de exemplo

O código a seguir usa a função `GetCircuitStatus` do Lambda para verificar o status do disjuntor.

```
var serviceDetails = _dbContext.QueryAsync<CircuitBreaker>(serviceName,
    QueryOperator.GreaterThan,
        new List<object>
            {currentTimeStamp}).GetRemainingAsync();

if (serviceDetails.Result.Count > 0)
{
    functionData.CircuitStatus = serviceDetails.Result[0].CircuitStatus;
}
else
{
    functionData.CircuitStatus = "";
}
```

O código a seguir mostra as instruções do Amazon States Language no fluxo de trabalho do Step Functions.

```
"Is Circuit Closed": {
    "Type": "Choice",
    "Choices": [
        {
            "Variable": "$.CircuitStatus",
            "StringEquals": "OPEN",
            "Next": "Circuit Open"
        },
        {
            "Variable": "$.CircuitStatus",
            "StringEquals": "",
            "Next": "Execute Lambda"
        }
    ]
},
"Circuit Open": {
    "Type": "Fail"
}
```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/circuit-breaker-netcore-blog>.

Referências do blog

- [Usando o padrão de disjuntor com o Amazon AWS Step Functions DynamoDB](#)

Conteúdo relacionado

- [Padrão strangler fig](#)
- [Tente novamente com o padrão de recuo](#)

Padrão de fornecimento de eventos

Intenção

Em arquiteturas orientadas a eventos, o padrão de fornecimento de eventos armazena os eventos que resultam em uma mudança de estado em um armazenamento de dados. Isso ajuda a capturar e manter um histórico completo das mudanças de estado e promove auditabilidade, rastreabilidade e a capacidade de analisar estados anteriores.

Motivação

Vários microsserviços podem colaborar para lidar com solicitações e se comunicam por meio de eventos. Esses eventos podem resultar em uma mudança no estado (dados). Armazenar objetos de eventos na ordem em que eles ocorrem fornece informações valiosas sobre o estado atual da entidade de dados e informações adicionais sobre como ela chegou a esse estado.

Aplicabilidade

Use o padrão de fornecimento de eventos quando:

- É necessário um histórico imutável dos eventos que ocorrem em um aplicativo para o rastreamento.
- Projeções de dados poliglota são exigidas de uma única fonte de verdade (SSOT).
- Point-in a reconstrução temporal do estado do aplicativo é necessária.
- Long-term o armazenamento do estado do aplicativo não é necessário, mas talvez você queira reconstruí-lo conforme necessário.
- As workloads têm volumes diferentes de leitura e gravação. Por exemplo, você tem workloads com muita gravação que não exigem processamento em tempo real.
- A captura de dados de alteração (CDC) é necessária para analisar o desempenho do aplicativo e outras métricas.
- Os dados de auditoria são necessários para todos os eventos que acontecem em um sistema para fins de emissão de relatórios e conformidade.
- Você deseja derivar cenários hipotéticos alterando (inserindo, atualizando ou excluindo) eventos durante o processo de repetição para determinar o possível estado final.

Problemas e considerações

- **Controle otimista de concorrência:** esse padrão armazena todos os eventos que causam uma mudança de estado no sistema. Vários usuários ou serviços podem tentar atualizar os mesmos dados ao mesmo tempo, causando colisões de eventos. Essas colisões acontecem quando eventos conflitantes são criados e aplicados ao mesmo tempo, o que resulta em um estado final de dados que não corresponde à realidade. Para resolver esse problema, você pode implementar estratégias para detectar e resolver colisões de eventos. Por exemplo, você pode implementar um esquema otimista de controle de concorrência incluindo versionamento ou adicionando registros de data e hora a eventos para rastrear a ordem das atualizações.
- **Complexidade:** a implementação do fornecimento de eventos exige uma mudança de mentalidade das operações tradicionais de CRUD para o pensamento orientado por eventos. O processo de repetição, usado para restaurar o sistema ao estado original, pode ser complexo para garantir a idempotência dos dados. Repositório de eventos, backups e instantâneos também podem aumentar a complexidade.
- **Consistência eventual:** as projeções de dados derivadas dos eventos acabam sendo consistentes devido à latência na atualização de dados usando o padrão de segmentação de responsabilidade de consulta de comando (CQRS) ou visões materializadas. Quando os consumidores processam dados de um repositório de eventos e os publicadores enviam novos dados, a projeção de dados ou o objeto do aplicativo podem não representar o estado atual.
- **Consulta:** a recuperação de dados atuais ou agregados dos logs de eventos pode ser mais complexa e lenta em comparação com bancos de dados tradicionais, especialmente para consultas complexas e tarefas de geração de relatórios. Para mitigar esse problema, o fornecimento de eventos geralmente é implementado com o padrão CQRS.
- **Tamanho e custo do repositório de eventos:** o repositório de eventos pode experimentar um crescimento exponencial em tamanho à medida que os eventos persistem, especialmente em sistemas com alta taxa de throughput de eventos ou períodos de retenção prolongados. Consequentemente, você deve arquivar periodicamente os dados do evento em um armazenamento econômico para evitar que o repositório de eventos fique muito grande.
- **Escalabilidade do repositório de eventos:** o repositório de eventos deve lidar de forma eficiente com grandes volumes de operações de gravação e leitura. Escalar um repositório de eventos pode ser um desafio, por isso é importante ter um repositório de dados que forneça fragmentos e partições.
- **Eficiência e otimização:** escolha ou projete um repositório de eventos que gerencie as operações de gravação e leitura com eficiência. O repositório de eventos deve ser otimizado para o

volume de eventos e os padrões de consulta esperados para o aplicativo. A implementação de mecanismos de indexação e consulta pode acelerar a recuperação de eventos ao reconstruir o estado do aplicativo. Você também pode considerar o uso de bancos de dados ou bibliotecas especializadas em repositórios de eventos que ofereçam atributos de otimização de consultas.

- **Snapshots:** você deve fazer backup dos logs de eventos em intervalos regulares com ativação baseada no tempo. A repetição dos eventos no último backup bem-sucedido conhecido dos dados deve levar à recuperação pontual do estado do aplicativo. O objetivo de ponto de recuperação (RPO) é o tempo máximo aceitável desde o último ponto de recuperação de dados. O RPO determina o que é considerado uma perda aceitável de dados entre o último ponto de recuperação e a interrupção do serviço. A frequência dos instantâneos diários do repositório de dados e eventos deve ser baseada no RPO do aplicativo.
- **Sensibilidade temporal:** os eventos são armazenados na ordem em que ocorrem. Portanto, a confiabilidade da rede é um fator importante a ser considerado ao implementar esse padrão. Problemas de latência podem levar a um estado incorreto do sistema. Use filas de primeiro a entrar, primeiro a sair (FIFO) com entrega no máximo uma vez para levar os eventos até o repositório do evento.
- **Desempenho de repetição de eventos:** pode ser demorado repetir um número substancial de eventos para reconstruir o estado atual do aplicativo. São necessários esforços de otimização para melhorar o desempenho, principalmente ao reproduzir eventos de dados arquivados.
- **Atualizações externas do sistema:** os aplicativos que usam o padrão de fornecimento de eventos podem atualizar os armazenamentos de dados em sistemas externos e podem capturar essas atualizações como objetos de eventos. Durante as repetições de eventos, isso pode se tornar um problema se o sistema externo não esperar uma atualização. Nesses casos, você pode usar sinalizadores de atributos para controlar as atualizações externas do sistema.
- **Consultas do sistema externo:** quando as chamadas externas do sistema são confidenciais à data e hora da chamada, os dados recebidos podem ser armazenados em armazenamentos de dados internos para uso durante as repetições.
- **Versionamento de eventos:** à medida que o aplicativo evolui, a estrutura dos eventos (esquema) pode mudar. É necessário implementar uma estratégia de versionamento para eventos para garantir a compatibilidade com versões anteriores e futuras. Isso pode envolver a inclusão de um campo de versão na carga útil do evento e o tratamento adequado de diferentes versões do evento durante a repetição.

Implementação

High-level arquitetura

Comandos e eventos

Em aplicativos de microsserviços distribuídos e orientados por eventos, os comandos representam as instruções ou solicitações enviadas a um serviço, normalmente com a intenção de iniciar uma mudança em seu estado. O serviço processa esses comandos e avalia a validade e a aplicabilidade do comando em seu estado atual. Se o comando for executado com êxito, o serviço responderá emitindo um evento que indica a ação tomada e as informações de estado relevantes. Por exemplo, no diagrama a seguir, o serviço de reserva responde ao comando Reservar corrida emitindo o evento Corrida reservada.



Repositórios de eventos

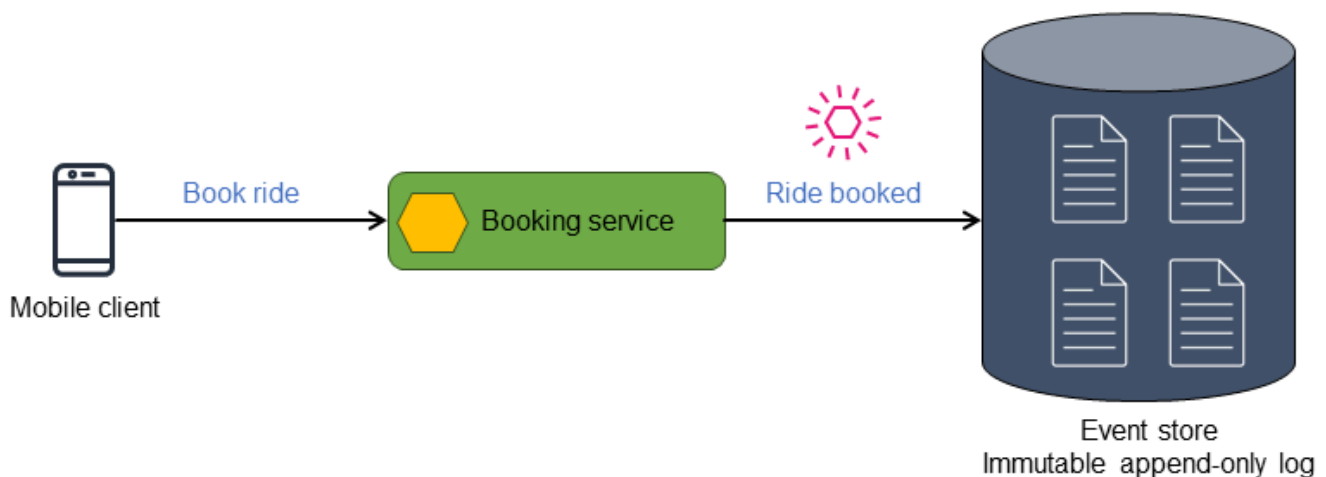
Os eventos são registrados em um armazenamento de dados ou repositório imutável, somente para anexos e ordenado cronologicamente, conhecido como repositório de eventos. Cada mudança de estado é tratada como um objeto de evento individual. Um objeto de entidade ou um repositório de dados com um estado inicial conhecido, seu estado atual e qualquer visualização pontual pode ser reconstruído reproduzindo os eventos na ordem de sua ocorrência.

O repositório de eventos atua como um registro histórico de todas as ações e mudanças de estado e serve como uma valiosa fonte única de verdade. Você pode usar o repositório de eventos para derivar o estado final e atualizado do sistema passando os eventos por um processador de repetição, que aplica esses eventos para produzir uma representação precisa do estado mais recente do sistema. Você também pode usar o repositório de eventos para gerar a perspectiva pontual do estado reproduzindo os eventos por meio de um processador de repetição. No padrão de

fornecimento de eventos, o estado atual pode não ser totalmente representado pelo objeto de evento mais recente. Você pode derivar o estado atual de três maneiras:

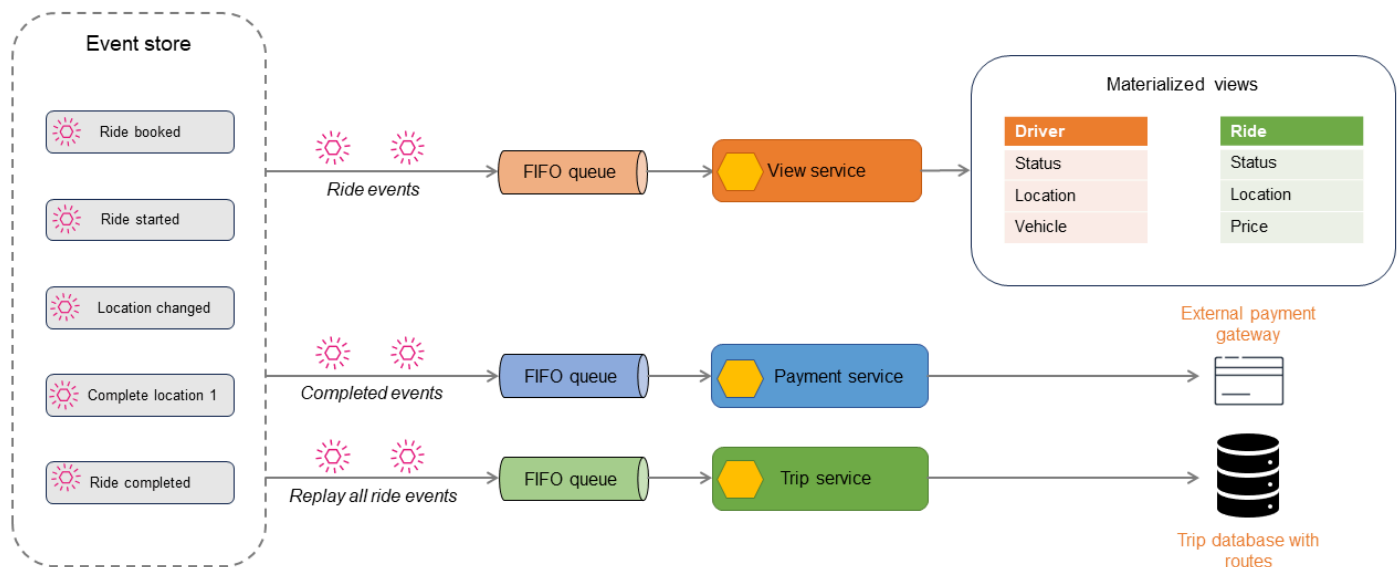
- Ao agregar eventos relacionados. Os objetos de eventos relacionados são combinados para gerar o estado atual para consulta. Essa abordagem geralmente é usada em conjunto com o padrão CQRS, em que os eventos são combinados e gravados no repositório de dados somente para leitura.
- Usando visões materializadas. Você pode empregar o fornecimento de eventos com o padrão de visão materializada para calcular ou resumir os dados do evento e obter o estado atual dos dados relacionados.
- Reproduzindo eventos. Objetos de eventos podem ser reproduzidos para realizar ações para gerar o estado atual.

O diagrama a seguir mostra o evento Ride booked sendo armazenado em um repositório de eventos.



O repositório de eventos publica os eventos que armazena, e os eventos podem ser filtrados e roteados para o processador apropriado para ações subsequentes. Por exemplo, os eventos podem ser roteados para um processador de visualização que resume o estado e mostra uma visão materializada. Os eventos são transformados no formato de dados do repositório de dados de destino. Essa arquitetura pode ser estendida para derivar diferentes tipos de repositórios de dados, o que leva à persistência poliglota dos dados.

O diagrama a seguir descreve os eventos em um aplicativo de reserva de corrida. Todos os eventos que ocorrem no aplicativo são armazenados no repositório de eventos. Os eventos armazenados são então filtrados e encaminhados para diferentes consumidores.



Os eventos de corrida podem ser usados para gerar repositórios de dados somente para leitura usando o CQRS ou o padrão de visão materializada. Você pode obter o estado atual da corrida, do motorista ou da reserva consultando os repositórios de leitura. Alguns eventos, como *Location changed* ou *Ride completed*, são publicados para outro consumidor para processamento de pagamentos. Quando a corrida é concluída, todos os eventos da corrida são repetidos para criar um histórico da corrida para fins de auditoria ou geração de relatórios.

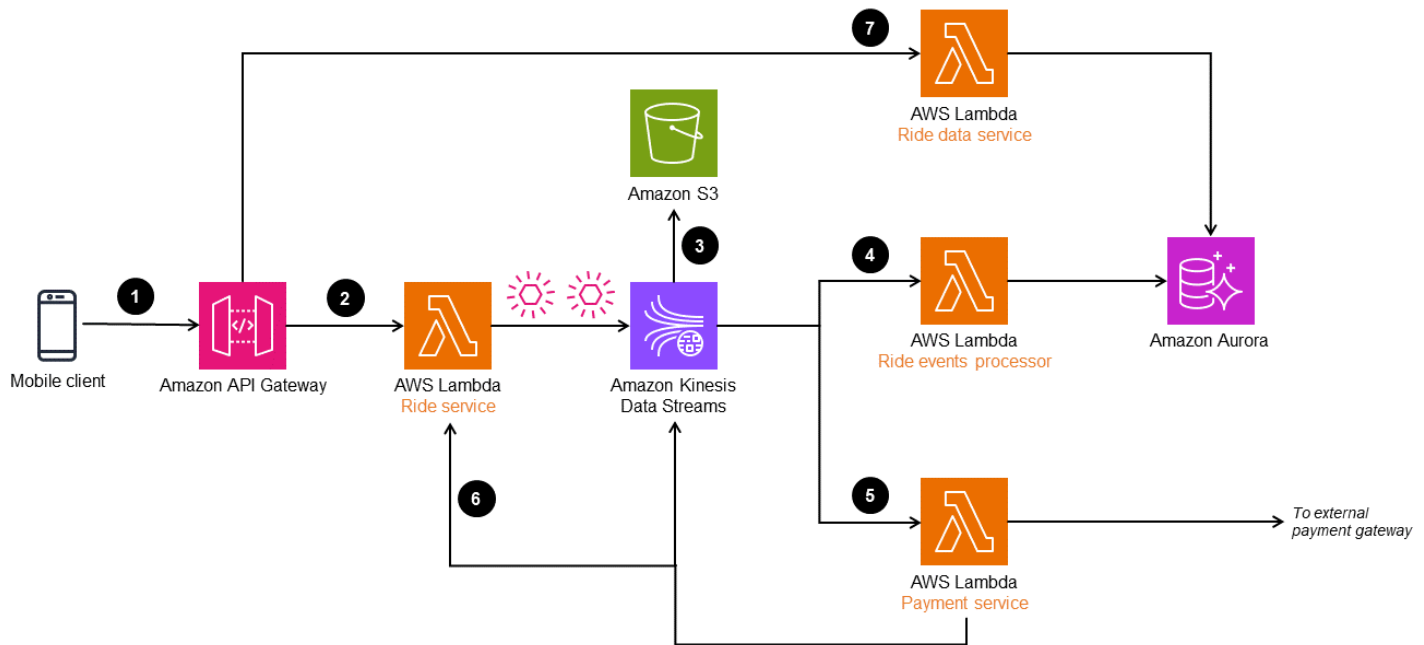
O padrão de fornecimento de eventos é frequentemente usado em aplicativos que exigem uma recuperação pontual e também quando os dados precisam ser projetados em formatos diferentes usando uma única fonte confiável. Ambas as operações exigem um processo de repetição para executar os eventos e derivar o estado final necessário. O processador de repetição também pode exigir um ponto de partida conhecido — de preferência não a partir da abertura do aplicativo, pois esse não seria um processo eficiente. Recomendamos que você tire instantâneos regulares do estado do sistema e aplique um número menor de eventos para obter um estado atualizado.

Implementação usando serviços da AWS

Na arquitetura a seguir, o Amazon Kinesis Data Streams é usado como repositório de eventos. Esse serviço captura e gerencia as alterações do aplicativo como eventos e oferece uma solução de fluxo de dados em tempo real e de alto throughput. Para implementar o padrão de fornecimento de

eventos na AWS, você também pode usar serviços como Amazon EventBridge e Amazon Managed Streaming for Apache Kafka (Amazon MSK) com base nas necessidades do seu aplicativo.

Para aumentar a durabilidade e habilitar a auditoria, você pode arquivar os eventos que são capturados pelo Kinesis Data Streams no Amazon Simple Storage Service (Amazon S3). Essa abordagem de armazenamento duplo ajuda a reter dados históricos de eventos com segurança para fins futuros de análise e conformidade.



O fluxo de trabalho consiste no seguinte:

1. Uma solicitação de reserva de corrida é feita por meio de um cliente móvel para um endpoint do Amazon API Gateway.
2. O microsserviço de corrida (função `Ride service` do Lambda) recebe a solicitação, transforma os objetos e publica no Kinesis Data Streams.
3. Os dados do evento no Kinesis Data Streams são armazenados no Amazon S3 para fins de conformidade e histórico de auditoria.
4. Os eventos são transformados e processados pela função `Ride event processor` do Lambda e armazenados em um banco de dados Amazon Aurora para fornecer uma visão materializada dos dados da corrida.
5. Os eventos de corrida concluídos são filtrados e enviados para processamento de pagamento em um gateway de pagamento externo. Quando o pagamento é concluído, outro evento é enviado ao Kinesis Data Streams para atualizar o banco de dados da Corrida.

6. Quando a corrida é concluída, os eventos da corrida são reproduzidos na função `Ride service` do Lambda para criar rotas e o histórico da corrida.
7. As informações da corrida podem ser lidas por meio do `Ride data service`, que lê a partir do banco de dados Aurora.

O API Gateway também pode enviar o objeto do evento diretamente para o Kinesis Data Streams sem a função `Ride service` do Lambda. No entanto, em um sistema complexo, como um serviço de carona, o objeto do evento pode precisar ser processado e enriquecido antes de ser ingerido no fluxo de dados. Por esse motivo, a arquitetura tem um `Ride service` que processa o evento antes de enviá-lo para o Kinesis Data Streams.

Referências do blog

- [Novidade no AWS Lambda — FIFO do SQS como uma origem de evento](#)

Padrão de arquitetura hexagonal

Intenção

O padrão de arquitetura hexagonal, também conhecido como padrão de portas e adaptadores, foi proposto pelo Dr. Alistair Cockburn em 2005. O objetivo é criar arquiteturas com acoplamento fraco em que os componentes da aplicação possam ser testados de forma independente, sem dependências de armazenamentos de dados ou interfaces de usuário (UIs). Esse padrão ajuda a evitar o bloqueio tecnológico de armazenamentos de dados e interfaces de usuário. Isso facilita a alteração da pilha de tecnologia ao longo do tempo, com impacto limitado ou inexistente na lógica de negócios. Nessa arquitetura com acoplamento fraco, a aplicação se comunica com componentes externos por meio de interfaces chamadas portas e usa adaptadores para converter as trocas técnicas com esses componentes.

Motivação

O padrão de arquitetura hexagonal é usado para isolar a lógica de negócios (lógica de domínio) do código de infraestrutura relacionado, como o código para acessar um banco de dados ou APIs externas. Esse padrão é útil para criar lógica de negócios e código de infraestrutura fracamente acoplados para AWS Lambda funções que exigem integração com serviços externos. Nas arquiteturas tradicionais, uma prática comum é incorporar a lógica de negócios na camada do banco de dados como procedimentos armazenados e na interface do usuário. Essa prática, junto com o uso de UI-specific construções na lógica de negócios, leva a arquiteturas estreitamente acopladas que causam gargalos nas migrações de bancos de dados e nos esforços de modernização da experiência do usuário (UX). O padrão de arquitetura hexagonal permite que você projete seus sistemas e aplicações por propósito e não por tecnologia. Essa estratégia resulta em componentes de aplicações facilmente intercambiáveis, como bancos de dados, UX e componentes de serviço.

Aplicabilidade

Use o padrão de arquitetura hexagonal quando:

- Você deseja desacoplar a arquitetura da sua aplicação para criar componentes que possam ser totalmente testados.
- Vários tipos de clientes podem usar a mesma lógica de domínio.

- Seus componentes de interface de usuário e banco de dados exigem atualizações periódicas de tecnologia que não afetem a lógica da aplicação.
- Sua aplicação requer vários provedores de entrada e consumidores de saída, e personalizar a lógica da aplicação leva à complexidade do código e à falta de extensibilidade.

Problemas e considerações

- Domain-driven design: a arquitetura hexagonal funciona especialmente bem com o design orientado por domínio (DDD). Cada componente da aplicação representa um subdomínio no DDD, e arquiteturas hexagonais podem ser usadas para obter um acoplamento fraco entre os componentes da aplicação.
- Testabilidade: por design, uma arquitetura hexagonal usa abstrações para entradas e saídas. Portanto, gravar testes de unidade e testar isoladamente torna-se mais fácil devido ao acoplamento fraco inerente.
- Complexidade: a complexidade de separar a lógica de negócios do código de infraestrutura, quando tratada com cuidado, pode trazer grandes benefícios, como agilidade, cobertura de testes e adaptabilidade tecnológica. Caso contrário, os problemas podem se tornar complexos de resolver.
- Sobrecarga de manutenção: o código adicional do adaptador que torna a arquitetura conectável é justificado somente se o componente da aplicação exigir várias fontes de entrada e destinos de saída para gravar, ou quando o armazenamento de dados de entrada e saída precisar mudar com o tempo. Caso contrário, o adaptador se tornará outra camada adicional a ser mantida, o que introduz uma sobrecarga de manutenção.
- Problemas de latência: o uso de portas e adaptadores adiciona outra camada, o que pode resultar em latência.

Implementação

As arquiteturas hexagonais são compatíveis com o isolamento da lógica de aplicações e negócios do código de infraestrutura e do código que integra a aplicação a interfaces de usuário, APIs externas, bancos de dados e agentes de mensagens. Você pode conectar facilmente os componentes da lógica de negócios a outros componentes (como bancos de dados) na arquitetura da aplicação por meio de portas e adaptadores.

As portas são pontos de entrada independentes de tecnologia em um componente da aplicação. Essas interfaces personalizadas determinam a interface que permite que atores externos se comuniquem com o componente da aplicação, independentemente de quem ou do que implementa a interface. Isso é semelhante à forma como uma porta USB permite que muitos tipos diferentes de dispositivos se comuniquem com um computador, desde que usem um adaptador USB.

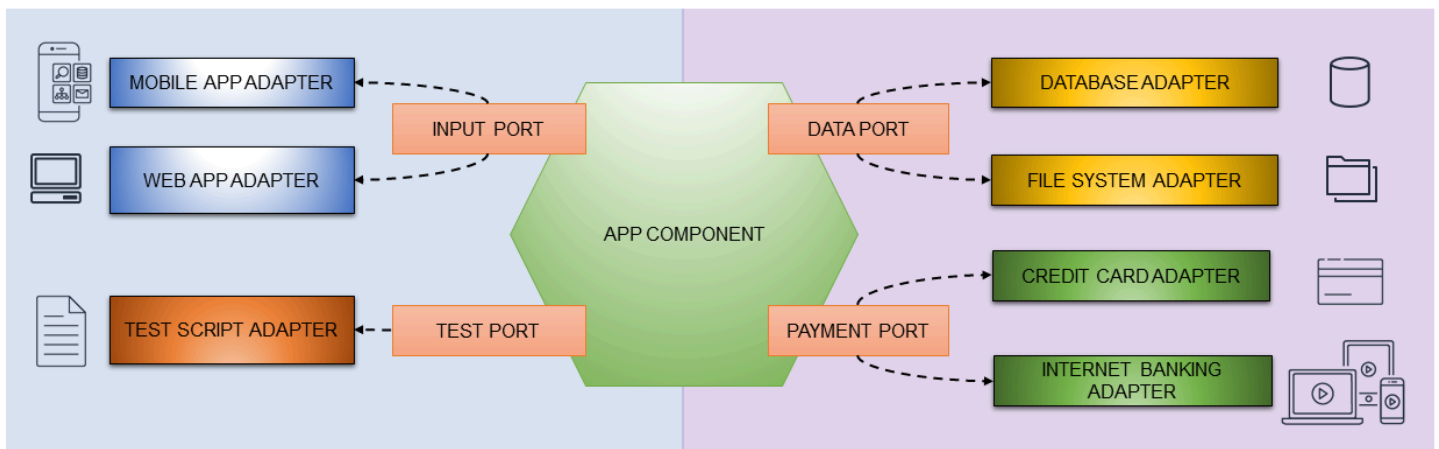
Os adaptadores interagem com a aplicação por meio de uma porta usando uma tecnologia específica. Os adaptadores se conectam a essas portas, recebem ou fornecem dados às portas e transformam os dados para processamento adicional. Por exemplo, um adaptador REST permite que os atores se comuniquem com o componente da aplicação por meio de uma API REST. Uma porta pode ter vários adaptadores sem nenhum risco para a porta ou para o componente da aplicação. Para estender o exemplo anterior, adicionar um adaptador GraphQL à mesma porta fornece um meio adicional para os atores interagirem com a aplicação por meio de uma API do GraphQL sem afetar a API REST, a porta ou a aplicação.

As portas se conectam à aplicação, e os adaptadores servem como uma conexão com o mundo externo. Você pode usar portas para criar componentes de aplicações com acoplamento fraco e trocar componentes dependentes mudando o adaptador. Isso permite que o componente da aplicação interaja com entradas e saídas externas sem precisar ter nenhum conhecimento contextual. Os componentes podem ser trocados em qualquer nível, o que facilita os testes automatizados. Você pode testar componentes de forma independente, sem dependências no código da infraestrutura, em vez de provisionar um ambiente inteiro para realizar testes. A lógica da aplicação não depende de fatores externos, então os testes são simplificados e fica mais fácil simular dependências.

Por exemplo, em uma arquitetura com acoplamento fraco, um componente da aplicação deve ser capaz de ler e gravar dados sem conhecer os detalhes do armazenamento de dados. A responsabilidade do componente da aplicação é fornecer dados para uma interface (porta). Um adaptador define a lógica de gravação em um armazenamento de dados, que pode ser um banco de dados, um sistema de arquivos ou um sistema de armazenamento de objetos, como o Amazon S3, dependendo das necessidades da aplicação.

High-level arquitetura

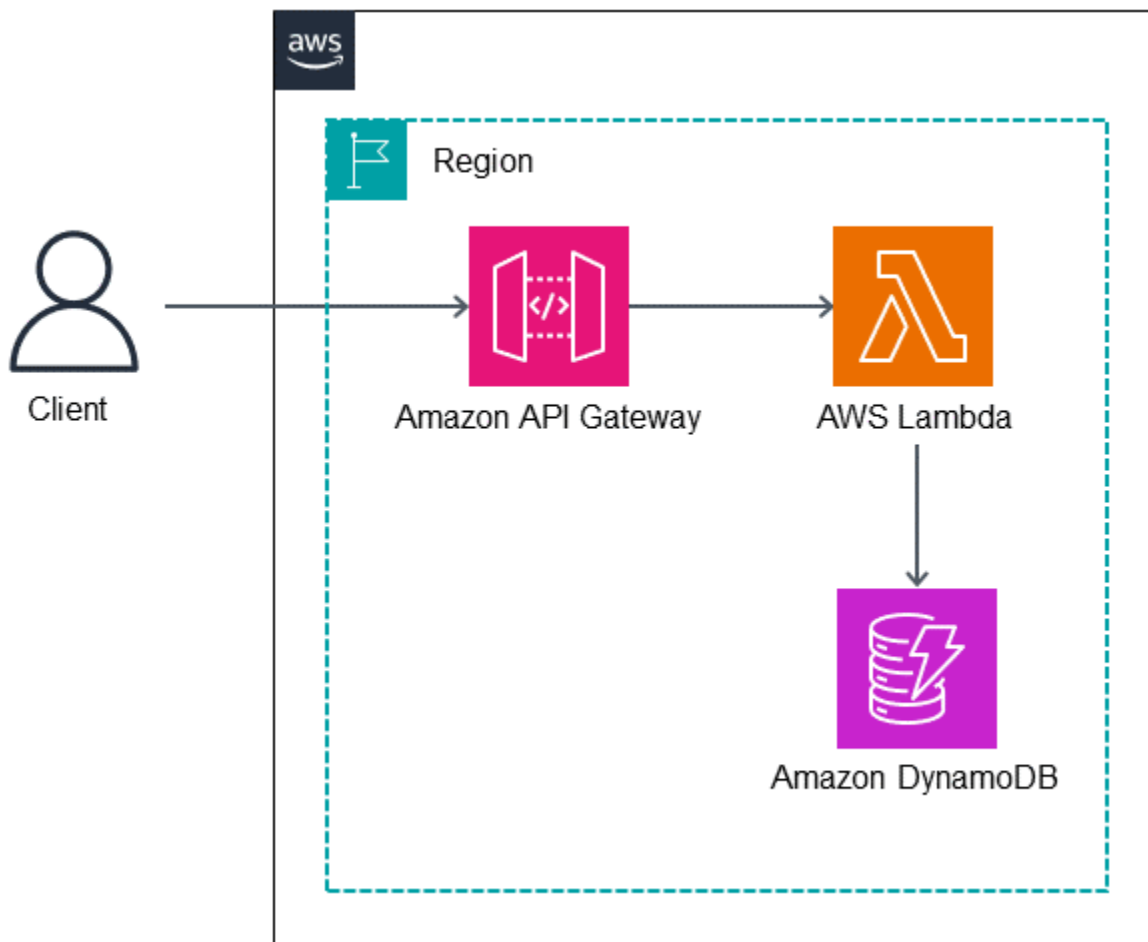
A aplicação ou o componente da aplicação contém a lógica de negócios principal. Ela recebe comandos ou consultas das portas e envia solicitações pelas portas para atores externos, que são implementadas por meio de adaptadores, conforme ilustrado no diagrama a seguir.



Implementação usando Serviços da AWS

AWS Lambda as funções geralmente contêm lógica de negócios e código de integração de banco de dados, que são fortemente acoplados para atingir um objetivo. Você pode usar o padrão de arquitetura hexagonal para separar a lógica de negócios do código de infraestrutura. Essa separação permite o teste de unidade da lógica de negócios sem dependências no código do banco de dados e melhora a agilidade do processo de desenvolvimento.

Na arquitetura a seguir, uma função do Lambda implementa o padrão de arquitetura hexagonal. A função do Lambda é iniciada pela API REST do Amazon API Gateway. A função implementa a lógica de negócios e grava dados nas tabelas do DynamoDB.



Código de exemplo

O código de exemplo nesta seção mostra como implementar o modelo de domínio usando o Lambda, como separá-lo do código de infraestrutura (como o código para acessar o DynamoDB) e como implementar testes de unidade para a função.

Modelo de domínio

A classe do modelo de domínio não tem conhecimento de componentes ou dependências externas, ela apenas implementa a lógica de negócios. No exemplo a seguir, a classe `Recipient` é uma classe de modelo de domínio que verifica se há sobreposições na data da reserva.

```
class Recipient:
    def __init__(self, recipient_id:str, email:str, first_name:str, last_name:str,
age:int):
        self.__recipient_id = recipient_id
        self.__email = email
```

```

        self.__first_name = first_name
        self.__last_name = last_name
        self.__age = age
        self.__slots = []

    @property
    def recipient_id(self):
        return self.__recipient_id
    #.....

    def are_slots_same_date(self, slot:Slot) -> bool:
        for selfslot in self.__slots:
            if selfslot.reservation_date == slot.reservation_date:
                return True
        return False

    def is_slot_counts_equal_or_over_two(self) -> bool:
    #.....

```

Porta de entrada

A classe `RecipientInputPort` se conecta à classe do destinatário e executa a lógica do domínio.

```

class RecipientInputPort(IRecipientInputPort):
    def __init__(self, recipient_output_port: IRecipientOutputPort, slot_output_port:
ISlotOutputPort):
        self.__recipient_output_port = recipient_output_port
        self.__slot_output_port = slot_output_port

    ...
    make reservation: adapting domain model business logic
    ...
    def make_reservation(self, recipient_id:str, slot_id:str) -> Status:
        status = None

        # -----
        # get an instance from output port
        # -----
        recipient = self.__recipient_output_port.get_recipient_by_id(recipient_id)
        slot = self.__slot_output_port.get_slot_by_id(slot_id)

        if recipient == None or slot == None:
            return Status(400, "Request instance is not found. Something wrong!")

```

```

print(f"recipient: {recipient.first_name}, slot date: {slot.reservation_date}")

# -----
# execute domain logic
# -----
ret = recipient.add_reserve_slot(slot)

# -----
# persistent an instance through output port
# -----
if ret == True:
    ret = self.__recipient_output_port.add_reservation(recipient)

if ret == True:
    status = Status(200, "The recipient's reservation is added.")
else:
    status = Status(200, "The recipient's reservation is NOT added!")
return status

```

Classe de adaptador do DynamoDB

A classe DDBRecipientAdapter implementa o acesso às tabelas do DynamoDB.

```

class DDBRecipientAdapter(IRecipientAdapter):
    def __init__(self):
        ddb = boto3.resource('dynamodb')
        self.__table = ddb.Table(table_name)

    def load(self, recipient_id:str) -> Recipient:
        try:
            response = self.__table.get_item(
                Key={'pk': pk_prefix + recipient_id})
            ...

    def save(self, recipient:Recipient) -> bool:
        try:
            item = {
                "pk": pk_prefix + recipient.recipient_id,
                "email": recipient.email,
                "first_name": recipient.first_name,
                "last_name": recipient.last_name,
                "age": recipient.age,

```

```
        "slots": []
    }
    # ...
```

A função do Lambda `get_recipient_input_port` é uma fábrica para instâncias da classe `RecipientInputPort`. Ela cria instâncias de classes de portas de saída com instâncias de adaptador relacionadas.

```
def get_recipient_input_port():
    return RecipientInputPort(
        RecipientOutputPort(DDBRecipientAdapter()),
        SlotOutputPort(DDBSlotAdapter()))

def lambda_handler(event, context):

    body = json.loads(event['body'])
    recipient_id = body['recipient_id']
    slot_id = body['slot_id']

    # get an input port instance
    recipient_input_port = get_recipient_input_port()
    status = recipient_input_port.make_reservation(recipient_id, slot_id)

    return {
        "statusCode": status.status_code,
        "body": json.dumps({
            "message": status.message
        }),
    }
```

Teste de unidade

Você pode testar a lógica de negócios para classes de modelo de domínio injetando classes simuladas. O exemplo a seguir fornece o teste de unidade para a classe `Recipient` do modelo de domínio.

```
def test_add_slot_one(fixture_recipient, fixture_slot):
    slot = fixture_slot
    target = fixture_recipient
    target.add_reserve_slot(slot)
    assert slot != None
    assert target != None
```

```
assert 1 == len(target.slots)
assert slot.slot_id == target.slots[0].slot_id
assert slot.reservation_date == target.slots[0].reservation_date
assert slot.location == target.slots[0].location
assert False == target.slots[0].is_vacant

def test_add_slot_two(fixture_recipient, fixture_slot, fixture_slot_2):
    #.....

def test_cannot_append_slot_more_than_two(fixture_recipient, fixture_slot,
    fixture_slot_2, fixture_slot_3):
    #.....

def test_cannot_append_same_date_slot(fixture_recipient, fixture_slot):
    #.....
```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/aws-lambda-domain-model-sample>.

Conteúdo relacionado

- [Hexagonal architecture](#), artigo de Alistair Cockburn
- [Desenvolvendo arquiteturas evolutivas com AWS Lambda](#) (postagem de AWS blog em japonês)

Vídeos

O vídeo a seguir (em japonês) analisa o uso da arquitetura hexagonal na implementação de um modelo de domínio usando uma função do Lambda.

Publish-subscribe padrão

Intenção

O padrão publicar/assinar, também conhecido como padrão pub-ass, é um padrão de mensagens que separa o remetente da mensagem (publicador) dos destinatários interessados (assinantes). Esse padrão implementa comunicações assíncronas publicando mensagens ou eventos por meio de um intermediário conhecido como agente de mensagens ou roteador (infraestrutura de mensagens). O padrão publicar/assinar aumenta a escalabilidade e a capacidade de resposta dos remetentes, transferindo a responsabilidade da entrega da mensagem para a infraestrutura da mensagem, para que o remetente possa se concentrar no processamento principal da mensagem.

Motivação

Em arquiteturas distribuídas, os componentes do sistema frequentemente precisam fornecer informações a outros componentes à medida que os eventos ocorrem dentro do sistema. O padrão publicar/assinar separa as preocupações para que os aplicativos possam se concentrar em seus recursos principais, enquanto a infraestrutura de mensagens lida com responsabilidades de comunicação, como roteamento de mensagens e entrega confiável. O padrão publicar/assinar permite que mensagens assíncronas dissociem o publicador e os assinantes. Os publicadores também podem enviar mensagens sem o conhecimento dos assinantes.

Aplicabilidade

Use o padrão publicar/assinar quando:

- O processamento paralelo é necessário se uma única mensagem tiver fluxos de trabalho diferentes.
- Não são necessárias a transmissão de mensagens para vários assinantes e respostas em tempo real dos destinatários.
- O sistema ou aplicativo pode tolerar uma consistência eventual dos dados ou do estado.
- O aplicativo ou componente precisa se comunicar com outros aplicativos ou serviços que possam usar linguagens, protocolos ou plataformas diferentes.

Problemas e considerações

- Disponibilidade do assinante: o publicador não sabe se os assinantes estão recebendo, e talvez não estejam. As mensagens publicadas são de natureza transitória e podem resultar no cancelamento se os assinantes não estiverem disponíveis.
- Garantia de entrega de mensagens: normalmente, o padrão publicar/assinar não pode garantir a entrega de mensagens para todos os tipos de assinantes, embora determinados serviços, como o Amazon Simple Notification Service (Amazon SNS), possam fornecer entrega [exatamente uma vez](#) para alguns subconjuntos de assinantes.
- Tempo de vida (TTL): as mensagens têm vida útil e expiram se não forem processadas dentro desse período. Considere adicionar as mensagens publicadas a uma fila para que elas possam persistir e garantir o processamento além do período TTL.
- Relevância da mensagem: os produtores podem definir um intervalo de tempo para a relevância como parte dos dados da mensagem, e a mensagem pode ser descartada após essa data. Considere fazer com que os consumidores examinem essas informações antes de decidir como processar a mensagem.
- Consistência eventual: há um atraso entre o momento em que a mensagem é publicada e o momento em que é consumida pelo assinante. Isso pode fazer com que os armazenamentos de dados do assinante se tornem eventualmente consistentes quando for necessária uma consistência forte. A consistência eventual também pode ser um problema quando produtores e consumidores precisam de interação quase em tempo real.
- Comunicação unidirecional: o padrão publicar/assinar é considerado unidirecional. Os aplicativos que exigem mensagens bidirecionais com um canal de assinatura de retorno devem considerar o uso de um padrão de solicitação-resposta se for necessária uma resposta síncrona.
- Ordem das mensagens: a ordem das mensagens não é garantida. Se os consumidores precisarem de mensagens solicitadas, recomendamos que você use os [tópicos FIFO do Amazon SNS](#) para garantir o pedido.
- Duplicação de mensagens: com base na infraestrutura de mensagens, mensagens duplicadas podem ser entregues aos consumidores. Os consumidores devem ser projetados para serem idempotentes para lidar com o processamento de mensagens duplicadas. Como alternativa, use os [tópicos FIFO do Amazon SNS](#) para garantir uma entrega exata uma vez.
- Filtragem de mensagens: os consumidores geralmente estão interessados apenas em um subconjunto das mensagens publicadas por um publicador. Forneça mecanismos para permitir que os assinantes filtrem ou restrinjam as mensagens que recebem fornecendo tópicos ou filtros de conteúdo.

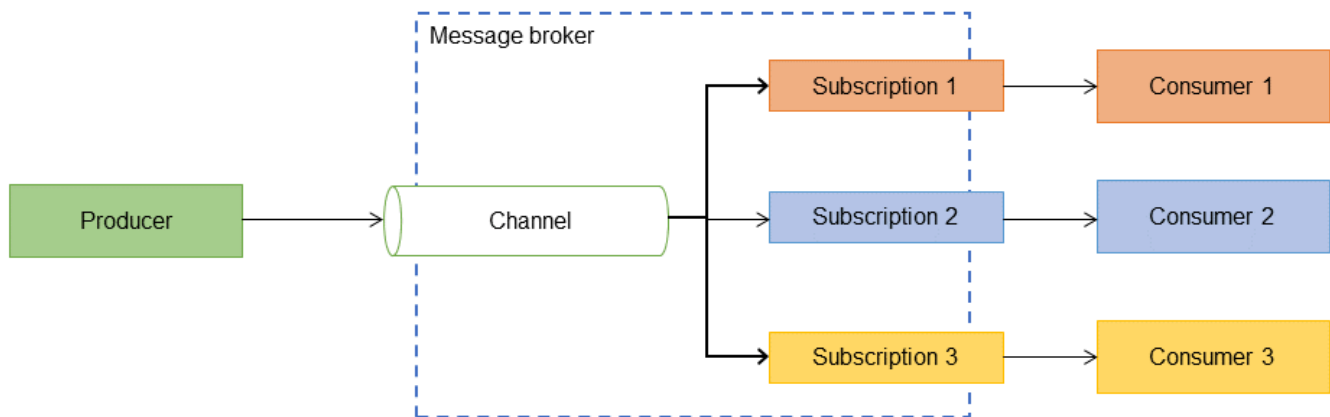
- Reprodução de mensagens: os recursos de reprodução de mensagens podem depender da infraestrutura de mensagens. Você também pode fornecer implementações personalizadas, dependendo do caso de uso.
- Dead-letter filas: em um sistema postal, um escritório de correio morto é uma instalação para processar correspondências não entregues. Em [pub/sub mensagens](#), uma fila de mensagens mortas (DLQ) é uma fila para mensagens que não podem ser entregues a um endpoint inscrito.

Implementação

High-level arquitetura

Em um padrão publicar/assinar, o subsistema de mensagens assíncronas conhecido como agente de mensagens ou roteador acompanha as assinaturas. Quando um produtor publica um evento, a infraestrutura de mensagens envia uma mensagem para cada consumidor. Depois que uma mensagem é enviada aos assinantes, ela é removida da infraestrutura de mensagens para que não possa ser reproduzida e os novos assinantes não vejam o evento. Os agentes de mensagens ou roteadores separam o produtor de eventos dos consumidores de mensagens da seguinte forma:

- Fornecer um canal de entrada para o produtor publicar eventos que são empacotados em mensagens, usando um formato de mensagem definido.
- Criação de um canal de saída individual por assinatura. Uma assinatura é a conexão do consumidor, na qual ele recebe mensagens de eventos associadas a um canal de entrada específico.
- Copiar mensagens do canal de entrada para o canal de saída para todos os consumidores quando o evento for publicado.



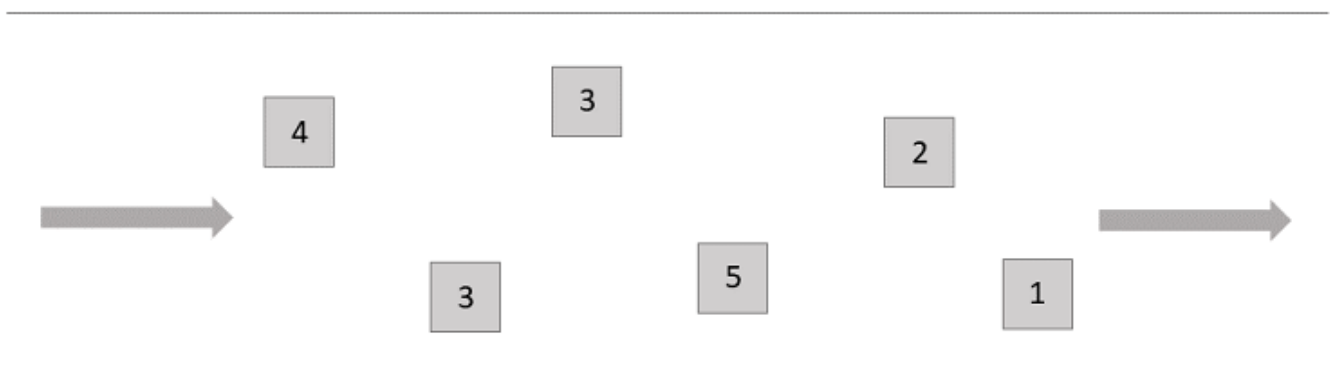
Implementação usando serviços da AWS

Amazon SNS

O Amazon SNS é um serviço de publicador-assinante totalmente gerenciado que fornece mensagens de aplicação para aplicação (A2A) para dissociar aplicações distribuídas. Ele também fornece mensagens de aplicação para pessoa (A2P) para enviar SMS, e-mail e outras notificações push.

O Amazon SNS fornece dois tipos de tópicos: padrão e primeiro a entrar, primeiro a sair (FIFO).

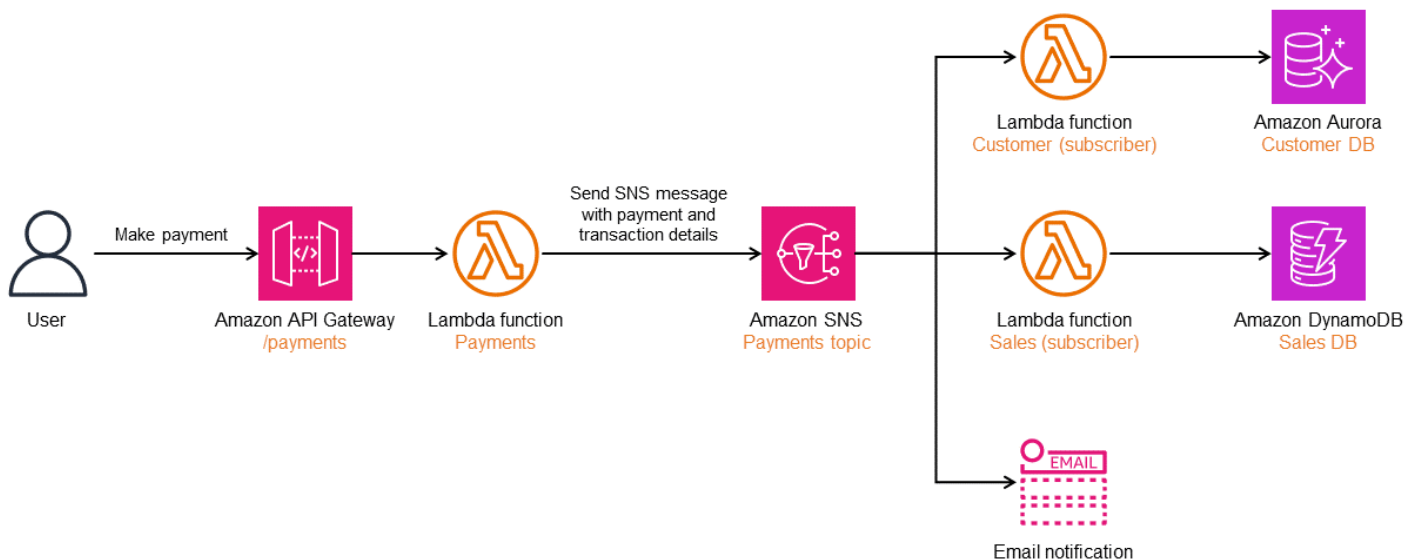
- Os tópicos padrão oferecem suporte a um número ilimitado de mensagens por segundo e oferecem o melhor esforço para ordenação e deduplicação.



- Os tópicos do FIFO fornecem ordenação e deduplicação rigorosas e oferecem suporte a até 300 mensagens por segundo ou 10 MB por segundo por tópico do FIFO (o que ocorrer primeiro).



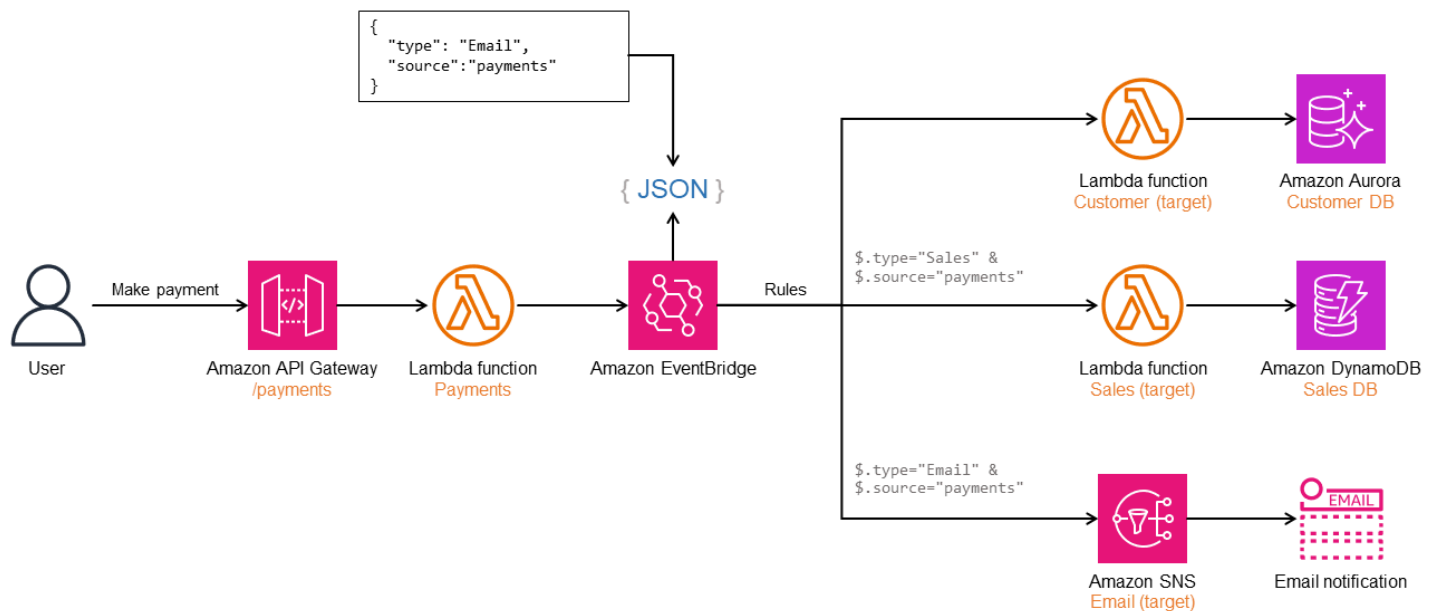
A ilustração a seguir mostra como você pode usar o Amazon SNS para implementar o padrão publicar/assinar. Depois que um usuário faz um pagamento, uma mensagem do SNS é enviada pela função Payments do Lambda para o tópico Payments do SNS. Este tópico do SNS tem três assinantes. Cada assinante recebe uma cópia da mensagem e a processa.



Amazon EventBridge

Você pode usar a Amazon EventBridge quando precisar de um roteamento mais complexo de mensagens de vários produtores em protocolos diferentes para consumidores inscritos ou de assinaturas diretas e distribuídas. EventBridge também oferece suporte a roteamento, filtragem, sequenciamento e divisão ou agregação baseados em conteúdo. Na ilustração a seguir, EventBridge é usado para criar uma versão do padrão publicar-assinar no qual os assinantes são definidos usando regras de eventos. Depois que um usuário faz um pagamento, a função Payments Lambda envia uma mensagem EventBridge usando o barramento de eventos padrão com base em um

esquema personalizado que tem três regras apontando para destinos diferentes. Cada microserviço processa as mensagens e executa as ações necessárias.



Workshop

- [Criação de arquiteturas orientadas por eventos na AWS](#)
- [Enviar Notificações Fanout de Eventos por meio do Amazon Simple Queue Service \(Amazon SQS\) e do Amazon Simple Notification Service \(Amazon SNS\)](#)

Referências do blog

- [Escolher entre serviços de mensagens para aplicativos sem servidor](#)
- [Projetar aplicativos duráveis sem servidor com DLQs para Amazon SNS, Amazon SQS e AWS Lambda](#)
- [Simplifique suas pub/sub mensagens com a filtragem de mensagens do Amazon SNS](#)

Conteúdo relacionado

- [Características do pub/sub sistema de mensagens](#)

Tente novamente com o padrão de recuo

Intenção

O padrão de repetição com recuo melhora a estabilidade da aplicação ao repetir de forma transparente as operações que falham devido a erros transitórios.

Motivação

Em arquiteturas distribuídas, erros transitórios podem ser causados por controle de utilização do serviço, perda temporária de conectividade de rede ou indisponibilidade temporária do serviço. Repetir automaticamente as operações que falham devido a esses erros transitórios melhora a experiência do usuário e a resiliência da aplicação. No entanto, novas tentativas frequentes podem sobrecarregar a largura de banda da rede e causar contenção. O recuo exponencial é uma técnica em que as operações são repetidas aumentando os tempos de espera para um número específico de tentativas de repetição.

Aplicabilidade

Use o padrão de repetição com recuo quando:

- Seus serviços frequentemente controlam a utilização de solicitações para evitar sobrecarga, resultando em uma exceção 429. Muitas solicitações no processo de chamada.
- A rede é um participante invisível em arquiteturas distribuídas, e problemas temporários de rede resultam em falhas.
- O serviço que está sendo chamado está temporariamente indisponível, causando falhas. Tentativas frequentes podem causar degradação do serviço, a menos que você introduza um tempo limite de recuo usando esse padrão.

Problemas e considerações

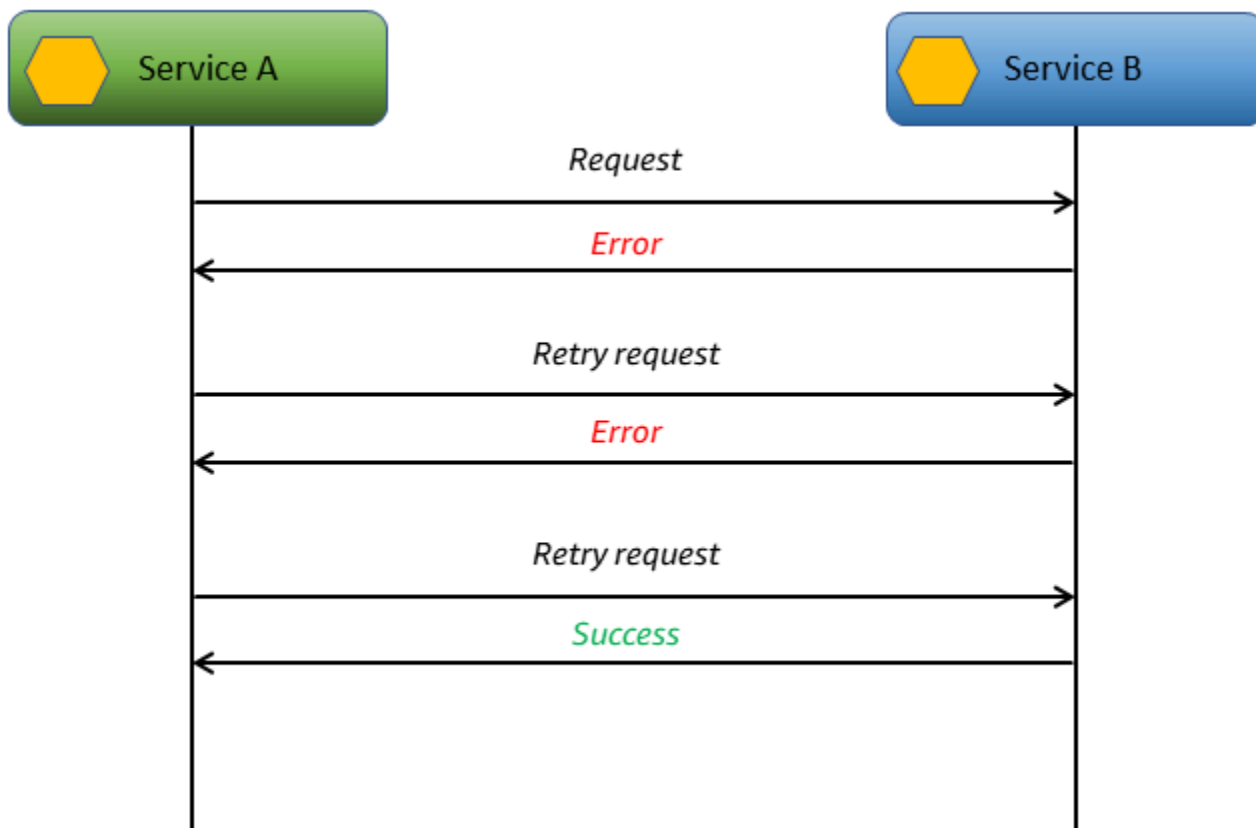
- Idempotência: se várias chamadas para o método tiverem o mesmo efeito de uma única chamada no estado do sistema, a operação será considerada idempotente. As operações devem ser idempotentes quando você usa o padrão de repetição com recuo. Caso contrário, atualizações parciais podem corromper o estado do sistema.

- Largura de banda da rede: a degradação do serviço poderá ocorrer se muitas tentativas ocuparem a largura de banda da rede, levando a tempos de resposta lentos.
- Cenários de falha rápida: para erros não transitórios, se você puder determinar a causa da falha, é mais eficiente antecipar-se à falha usando o padrão do disjuntor.
- Taxa de recuo: a introdução do recuo exponencial pode ter um impacto no tempo limite do serviço, resultando em maiores tempos de espera para o usuário final.

Implementação

High-level arquitetura

O diagrama a seguir ilustra como o Serviço A pode repetir as chamadas para o Serviço B até que uma resposta bem-sucedida seja retornada. Se o Serviço B não retornar uma resposta bem-sucedida após algumas tentativas, o Serviço A poderá parar de tentar novamente e retornar uma falha ao chamador.

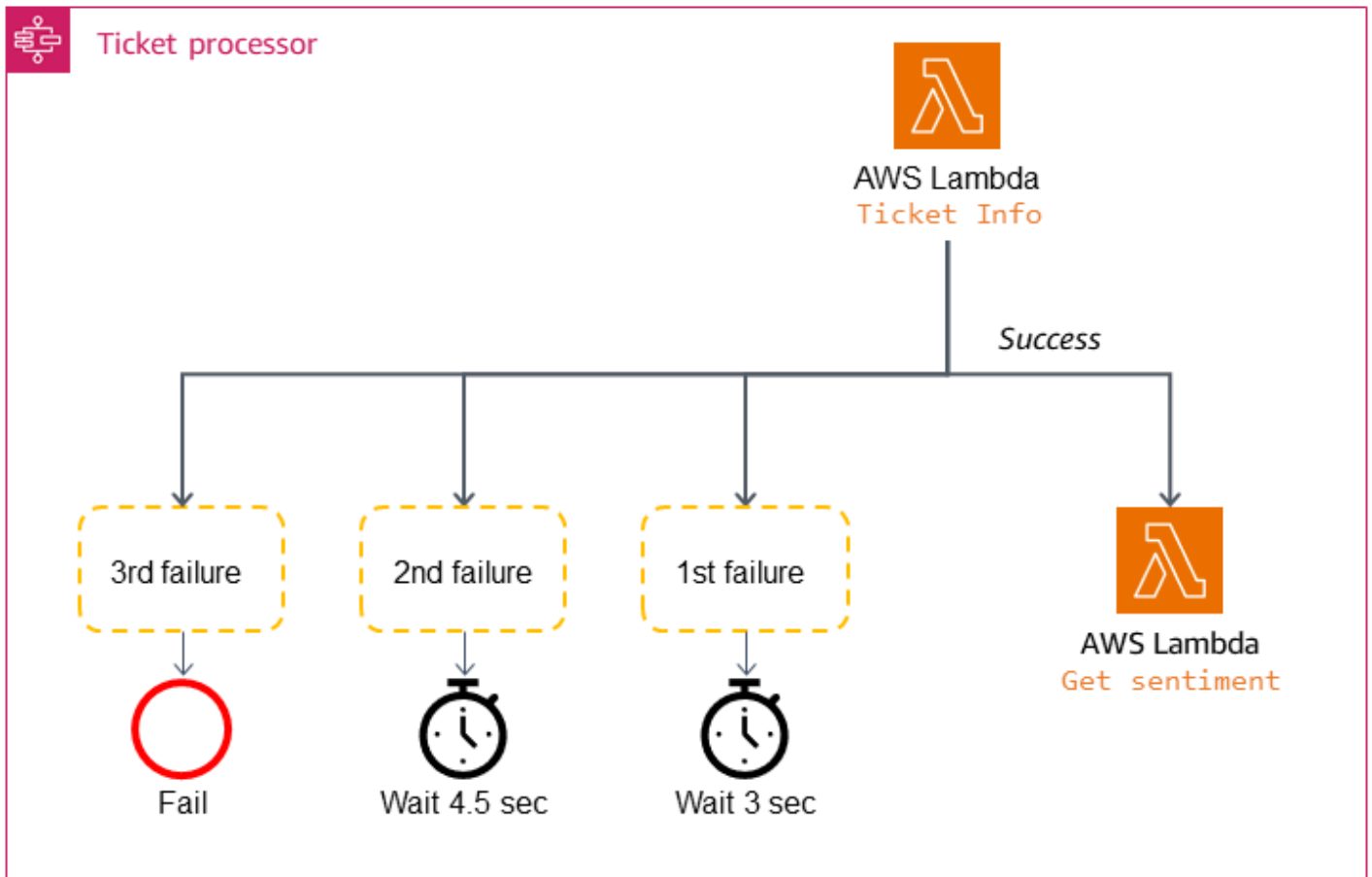


Implementação usando AWS serviços

O diagrama a seguir mostra um fluxo de trabalho de processamento de tíquetes em uma plataforma de suporte ao cliente. Os tíquetes de clientes insatisfeitos são acelerados escalando automaticamente a prioridade dos tíquetes. A função `Ticket info` do Lambda extrai os detalhes do tíquete e chama a função `Get sentiment` do Lambda. A função `Get sentiment` do Lambda verifica os sentimentos do cliente passando a descrição para o [Amazon Comprehend](#) (não mostrado).

Se a chamada para a função `Get sentiment` Lambda falhar, o fluxo de trabalho repetirá a operação três vezes. AWS Step Functions permite o recuo exponencial ao permitir que você configure o valor do recuo.

Neste exemplo, no máximo três novas tentativas são configuradas com um multiplicador de aumento de 1,5 segundo. Se a primeira tentativa ocorrer após 3 segundos, a segunda tentativa ocorrerá após $3 \times 1,5$ segundo = 4,5 segundos, e a terceira tentativa ocorrerá após $4,5 \times 1,5$ segundo = 6,75 segundos. Se a terceira tentativa não for bem-sucedida, o fluxo de trabalho falhará. A lógica de recuo não requer nenhum código personalizado, ela é fornecida como uma configuração pelo AWS Step Functions.



Código de exemplo

O código a seguir mostra a implementação do padrão de repetição com recuo.

```

public async Task DoRetriesWithBackOff()
{
    int retries = 0;
    bool retry;
    do
    {
        //Sample object for sending parameters
        var parameterObj = new InputParameter { SimulateTimeout = "false" };
        var content = new StringContent(JsonConvert.SerializeObject(parameterObj),
                                         System.Text.Encoding.UTF8, "application/json");
        var waitInMilliseconds = Convert.ToInt32((Math.Pow(2, retries) - 1) * 100);
        System.Threading.Thread.Sleep(waitInMilliseconds);
        var response = await _client.PostAsync(_baseURL, content);
        switch (response.StatusCode)
    }
}
  
```

```
{
    //Success
    case HttpStatusCode.OK:
        retry = false;
        Console.WriteLine(response.Content.ReadAsStringAsync().Result);
        break;
    //Throttling, timeouts
    case HttpStatusCode.TooManyRequests:
    case HttpStatusCode.GatewayTimeout:
        retry = true;
        break;
    //Some other error occurred, so stop calling the API
    default:
        retry = false;
        break;
}
retries++;
} while (retry && retries < MAX_RETRIES);
}
```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/retry-with-backoff>.

Conteúdo relacionado

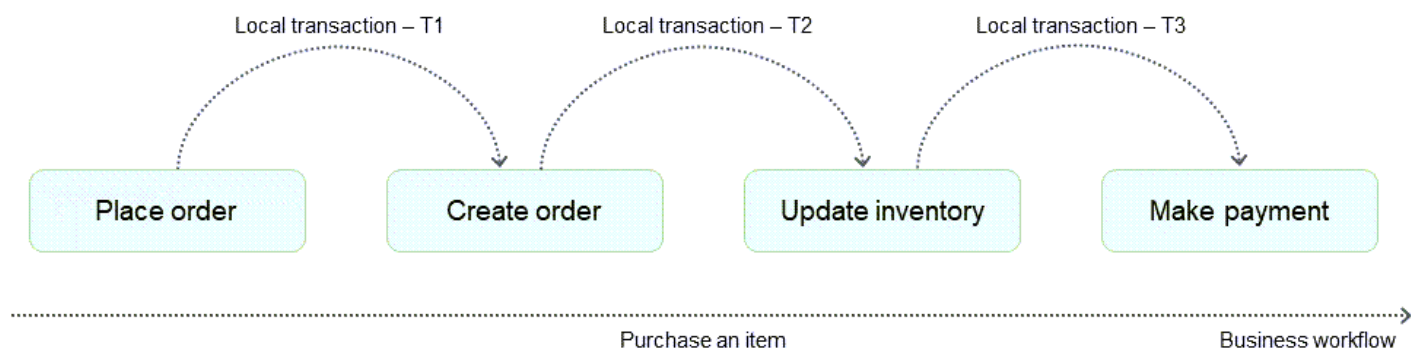
- [Timeouts, retries, and backoff with jitter](#) (Amazon Builders' Library)

Padrões saga

Uma saga consiste em uma sequência de transações locais. Cada transação local em uma saga atualiza o banco de dados e aciona a próxima transação local. Se uma transação falhar, a saga executa transações compensatórias para reverter as alterações do banco de dados feitas pelas transações anteriores.

Essa sequência de transações locais ajuda a alcançar um fluxo de trabalho comercial usando princípios de continuação e compensação. O princípio de continuação decide a recuperação futura do fluxo de trabalho, enquanto o princípio de compensação decide a recuperação retroativa. Se a atualização falhar em qualquer etapa da transação, a saga publica um evento para continuação (para repetir a transação) ou compensação (para voltar ao estado anterior dos dados). Isso garante que a integridade dos dados seja mantida e consistente em todos os repositórios de dados.

Por exemplo, quando um usuário compra um livro de um varejista on-line, o processo consiste em uma sequência de transações, como criação do pedido, atualização do estoque, pagamento e envio, que representa um fluxo de trabalho comercial. Para concluir esse fluxo de trabalho, a arquitetura distribuída emite uma sequência de transações locais para criar um pedido no banco de dados de pedidos, atualizar o banco de dados de estoque e atualizar o banco de dados de pagamento. Quando o processo é bem-sucedido, essas transações são invocadas sequencialmente para concluir o fluxo de trabalho comercial, conforme mostra o diagrama a seguir. No entanto, se alguma dessas transações locais falhar, o sistema deverá ser capaz de decidir sobre a próxima etapa apropriada, ou seja, uma recuperação para a frente ou uma recuperação para trás.



Os dois cenários a seguir ajudam a determinar se a próxima etapa é a recuperação para a frente ou para trás:

- Platform-level falha, quando algo dá errado com a infraestrutura subjacente e faz com que a transação falhe. Nesse caso, o padrão saga pode realizar uma recuperação para a frente tentando novamente a transação local e continuando o processo comercial.
- Application-level falha, quando o serviço de pagamento falha devido a um pagamento inválido. Nesse caso, o padrão saga pode realizar uma recuperação para trás emitindo uma transação compensatória para atualizar o estoque e os bancos de dados de pedidos e restabelecer seu estado anterior.

O padrão saga gerencia o fluxo de trabalho comercial e garante que um estado final desejável seja alcançado por meio da recuperação para a frente. Em caso de falhas, ele reverte as transações locais usando a recuperação reversa para evitar problemas de consistência de dados.

O padrão saga tem duas variantes: coreografia e orquestração.

Coreografia saga

O padrão da coreografia saga depende dos eventos publicados pelos microsserviços. Os participantes da saga (microsserviços) se inscrevem nos eventos e agem com base nos gatilhos do evento. Por exemplo, o serviço de pedidos no diagrama a seguir emite um evento `OrderPlaced`. O serviço de estoque se inscreve nesse evento e atualiza o estoque quando o evento `OrderPlaced` é emitido. Da mesma forma, os serviços participantes atuam com base no contexto do evento emitido.

O padrão coreográfico da saga é adequado quando há apenas alguns participantes na saga e você precisa de uma implementação simples sem um único ponto de falha. Quando mais participantes são adicionados, mais difícil fica rastrear as dependências entre os participantes usando esse padrão.



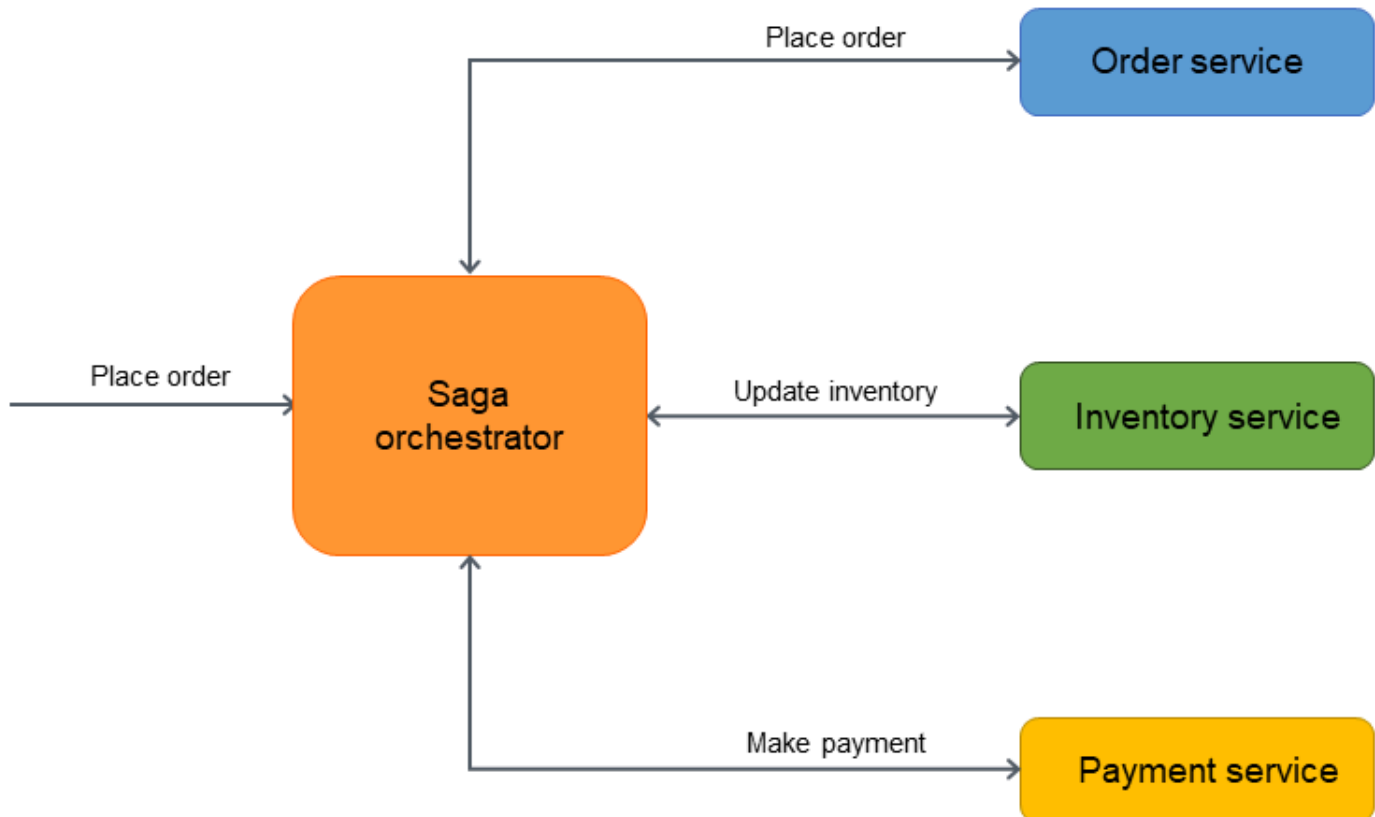
Para uma análise detalhada, consulte a seção de [coreografia de Saga](#) deste guia.

Orquestração da saga

O padrão de orquestração saga tem um coordenador central chamado orquestrador. O orquestrador da saga gerencia e coordena todo o ciclo de vida da transação. Ele está ciente da série de etapas a serem executadas para concluir a transação. Para executar uma etapa, ele envia uma mensagem ao microsserviço participante para realizar a operação. O microsserviço participante conclui a

operação e envia uma mensagem de volta ao orquestrador. Com base na mensagem que recebe, o orquestrador decide qual microserviço executar em seguida na transação.

O padrão de orquestração saga é adequado quando há muitos participantes, e é necessário um acoplamento fraco entre os participantes da saga. O orquestrador encapsula a complexidade da lógica ao tornar os participantes fracamente acoplados. No entanto, o orquestrador pode se tornar um único ponto de falha porque controla todo o fluxo de trabalho.



Para uma análise detalhada, consulte a seção de [orquestração saga](#) deste guia.

Padrão da coreografia saga

Intenção

O padrão de coreografia saga ajuda a preservar a integridade dos dados em transações distribuídas que abrangem vários serviços usando assinaturas de eventos. Em uma transação distribuída, vários serviços podem ser chamados antes que uma transação seja concluída. Quando os serviços armazenam dados em diferentes repositórios de dados, pode ser difícil manter a consistência de dados neles.

Motivação

Uma transação é uma única unidade de trabalho que pode envolver várias etapas, em que todas as etapas são completamente executadas ou nenhuma etapa é executada, resultando em um armazenamento de dados que mantém seu estado consistente. Os termos atomicidade, consistência, isolamento e durabilidade (ACID) definem as propriedades de uma transação. Os bancos de dados relacionais fornecem transações ACID para manter a consistência de dados.

Para manter a consistência em uma transação, os bancos de dados relacionais usam o método de confirmação em duas fases (2PC). Isso consiste em uma fase de preparação e uma fase de confirmação.

- Na fase de preparação, o processo de coordenação solicita que os processos participantes da transação (participantes) prometam confirmar ou reverter a transação.
- Na fase de confirmação, o processo de coordenação solicita que os participantes confirmem a transação. Se os participantes não concordarem em se comprometer na fase de preparação, a transação será revertida.

Em sistemas distribuídos que seguem um padrão de design de banco de dados por serviço, a confirmação em duas fases não é uma opção. Isso ocorre porque cada transação é distribuída em vários bancos de dados e não há um único controlador que possa coordenar um processo semelhante à confirmação em duas fases em repositórios de dados relacionais. Nesse caso, uma solução é usar o padrão de coreografia saga.

Aplicabilidade

Use o padrão de coreografia saga quando:

- Seu sistema exige integridade e consistência de dados em transações distribuídas que abrangem vários repositórios de dados.
- O armazenamento de dados (por ex., um banco de dados NoSQL) não fornece 2PC para fornecer transações ACID; você precisa atualizar várias tabelas em uma única transação, e implementar 2PC dentro dos limites do aplicativo seria uma tarefa complexa.
- Um processo de controle central que gerencia as transações dos participantes pode se tornar um único ponto de falha.
- Os participantes da saga são serviços independentes e precisam ser vagamente acoplados.
- Há comunicação entre contextos limitados em um domínio comercial.

Problemas e considerações

- **Complexidade:** à medida que o número de microsserviços aumenta, a coreografia saga pode tornar-se difícil de gerenciar devido ao número de interações entre os microsserviços. Além disso, transações compensatórias e novas tentativas adicionam complexidades ao código do aplicativo, o que pode resultar em sobrecarga de manutenção. A coreografia estará adequada quando houver poucos participantes na saga e você precisar de uma implementação simples sem um único ponto de falha. Quando mais participantes são adicionados, mais difícil fica rastrear as dependências entre os participantes usando esse padrão.
- **Implementação resiliente:** na coreografia Saga, é mais difícil implementar tempos limite, novas tentativas e outros padrões de resiliência globalmente, em comparação com a orquestração Saga. A coreografia deve ser implementada em componentes individuais em vez de em nível de orquestrador.
- **Dependências cíclicas:** os participantes consomem mensagens publicadas uns pelos outros. Isso pode resultar em dependências cíclicas, levando a complexidades de código e sobrecargas de manutenção, além de possíveis impasses.
- **Problema de gravação dupla:** o microsserviço precisa atualizar atômicamente o banco de dados e publicar um evento. A falha de qualquer operação pode levar a um estado inconsistente. Uma maneira de resolver isso é usar o padrão de caixa de [saída transacional](#).
- **Preservando eventos:** os participantes da saga agem com base nos eventos publicados. É importante salvar os eventos na ordem em que eles ocorrem para fins de auditoria, depuração e repetição. Você pode usar o [padrão de fornecimento de eventos](#) para persistir os eventos em um repositório de eventos, caso seja necessária uma repetição do estado do sistema para restaurar a consistência de dados. Os repositórios de eventos também podem ser usados para fins de auditoria e solução de problemas, pois refletem todas as alterações no sistema.
- **Consistência eventual:** o processamento sequencial de transações locais resulta em consistência eventual, o que pode ser um desafio em sistemas que exigem consistência forte. Você pode resolver esse problema definindo as expectativas de suas equipes comerciais em relação ao modelo de consistência ou reavaliando o caso de uso e migrando para um banco de dados que forneça consistência forte.
- **Idempotência:** os participantes da saga precisam ser idempotentes para permitir a execução repetida em caso de falhas transitórias causadas por falhas inesperadas e falhas do orquestrador.
- **Isolamento de transação:** o padrão saga não tem isolamento de transação, que é uma das quatro propriedades nas transações ACID. O [grau de isolamento](#) de uma transação determina quantas ou quão poucas outras transações simultâneas podem afetar os dados nos quais ela opera. A

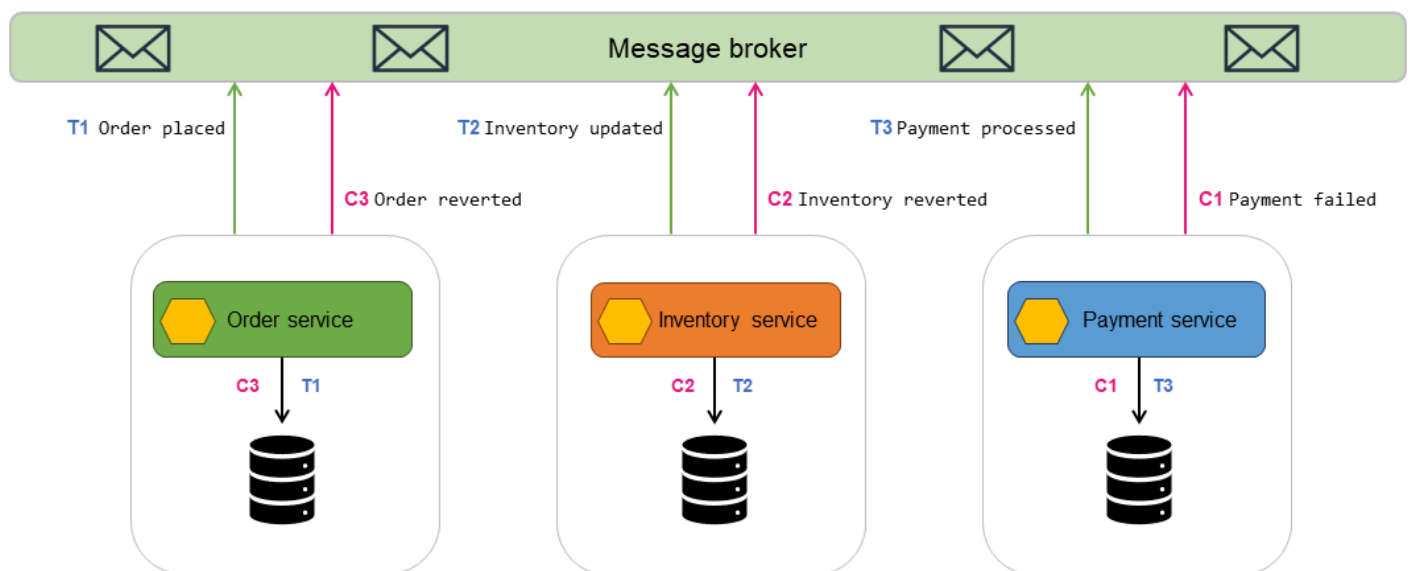
orquestração simultânea de transações pode levar a dados obsoletos. Recomendamos usar o bloqueio semântico para lidar com esses cenários.

- **Observabilidade:** observabilidade se refere ao registro e rastreamento detalhados para solucionar problemas no processo de implementação e orquestração. Isso se torna importante quando o número de participantes da saga aumenta, resultando em complexidades na depuração. End-to-end monitoramento e reportagem são mais difíceis de alcançar na coreografia de saga, em comparação com a orquestração de saga.
- **Problemas de latência:** transações compensatórias podem adicionar latência ao tempo geral de resposta quando a saga consiste em várias etapas. Se as transações fizerem chamadas síncronas, isso pode aumentar ainda mais a latência.

Implementação

High-level arquitetura

No diagrama de arquitetura a seguir, a coreografia da saga tem três participantes: o serviço de pedidos, o serviço de estoque e o serviço de pagamento. São necessárias três etapas para concluir a transação: T1, T2 e T3. Três transações compensatórias restauram os dados ao estado inicial: C1, C2 e C3.



- O serviço de pedidos executa uma transação local, T1, que atualiza atômica o banco de dados e publica uma mensagem `Order placed` no agente de mensagens.

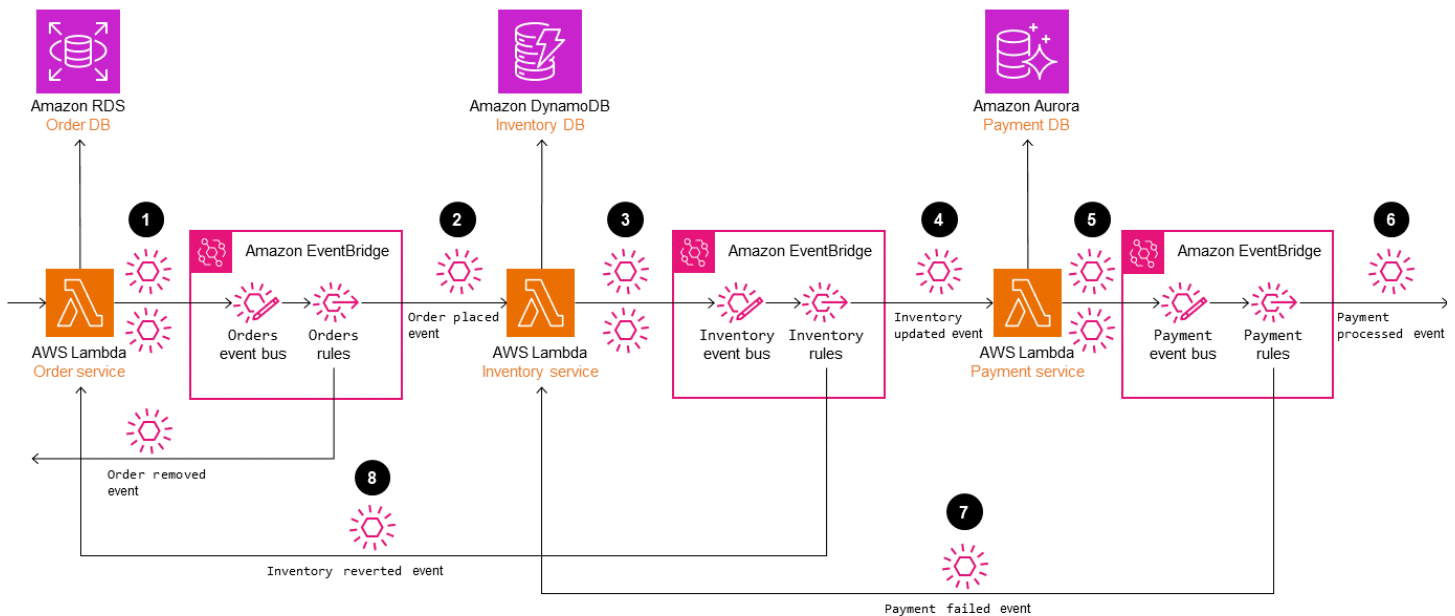
- O serviço de estoque assina as mensagens do serviço de pedidos e recebe a mensagem de que um pedido foi criado.
- O serviço de estoque executa uma transação local, T2, que atualiza atômicamente o banco de dados e publica uma mensagem `Inventory updated` no agente de mensagens.
- O serviço de pagamento assina as mensagens do serviço de estoque e recebe a mensagem de que o estoque foi atualizado.
- O serviço de pagamento executa uma transação local, T3, que atualiza atômicamente o banco de dados com detalhes de pagamento e publica uma mensagem `Payment processed` para o agente de mensagens.
- Se o pagamento falhar, o serviço de pagamento executa uma transação compensatória, C1, que reverte atômicamente o pagamento no banco de dados e publica uma mensagem `Payment failed` para o agente de mensagens.
- As transações compensatórias C2 e C3 são executadas para restaurar a consistência dos dados.

Implementação usando serviços da AWS

Você pode implementar o padrão de coreografia da saga usando a Amazon. EventBridge. EventBridge usa eventos para conectar os componentes do aplicativo. Ele processa eventos por meio de tubos ou barramentos de eventos. Um barramento de eventos é um roteador que recebe [eventos](#) e os entrega a zero ou mais destinos, ou alvos.. [As regras](#) associadas ao barramento de eventos avaliam os eventos à medida que eles chegam e os enviam aos [alvos](#) para processamento.

Na seguinte arquitetura:

- Os microsserviços — serviço de pedidos, serviço de estoque e serviço de pagamento — são implementados como funções do Lambda.
- Existem três EventBridge ônibus personalizados: ônibus para `Orders` eventos, ônibus para `Inventory` eventos e ônibus para `Payment` eventos.
- As regras `Orders`, regras `Inventory` e regras `Payment` correspondem aos eventos que são enviados para o barramento de eventos correspondente e invocam as funções do Lambda.



Em um cenário bem-sucedido, quando um pedido é feito:

1. O serviço de pedidos processa a solicitação e envia o evento para o barramento de eventos Orders.
2. As regras Orders correspondem aos eventos e iniciam o serviço de estoque.
3. O serviço de estoque atualiza o estoque e envia o evento para o barramento de eventos Inventory.
4. As regras Inventory correspondem aos eventos e iniciam o serviço de pagamento.
5. O serviço de pagamento processa o pagamento e envia o evento para o barramento de eventos Payment.
6. As regras Payment correspondem aos eventos e enviam a notificação do evento Payment processed ao receptor.

Como alternativa, quando há um problema no processamento do pedido, EventBridge as regras iniciam as transações compensatórias para reverter as atualizações de dados para manter a consistência e a integridade dos dados.

7. Se o pagamento falhar, as regras Payment processam o evento e iniciam o serviço de estoque. O serviço de inventário executa transações compensatórias para reverter o estoque.

8. Quando o estoque é revertido, o serviço de estoque envia o evento `Inventory reverted` para o barramento de eventos `Inventory`. Esse evento é processado por regras `Inventory`. Ele inicia o serviço de pedidos, que executa a transação compensatória para remover o pedido.

Conteúdo relacionado

- [Padrão de orquestração da saga](#)
- [Padrão de caixa de saída transacional](#)
- [Tente novamente com o padrão de recuo](#)

Padrão de orquestração saga

Intenção

O padrão de orquestração saga usa um coordenador central (orquestrador) para ajudar a preservar a integridade dos dados em transações distribuídas que abrangem vários serviços. Em uma transação distribuída, vários serviços podem ser chamados antes que uma transação seja concluída. Quando os serviços armazenam dados em diferentes repositórios de dados, pode ser difícil manter a consistência de dados neles.

Motivação

Uma transação é uma única unidade de trabalho que pode envolver várias etapas, em que todas as etapas são completamente executadas ou nenhuma etapa é executada, resultando em um armazenamento de dados que mantém seu estado consistente. Os termos atomicidade, consistência, isolamento e durabilidade (ACID) definem as propriedades de uma transação. Os bancos de dados relacionais fornecem transações ACID para manter a consistência de dados.

Para manter a consistência em uma transação, os bancos de dados relacionais usam o método de confirmação em duas fases (2PC). Isso consiste em uma fase de preparação e uma fase de confirmação.

- Na fase de preparação, o processo de coordenação solicita que os processos participantes da transação (participantes) prometam confirmar ou reverter a transação.
- Na fase de confirmação, o processo de coordenação solicita que os participantes confirmem a transação. Se os participantes não concordarem em se comprometer na fase de preparação, a transação será revertida.

Em sistemas distribuídos que seguem um padrão de design de banco de dados por serviço, a confirmação em duas fases não é uma opção. Isso ocorre porque cada transação é distribuída em vários bancos de dados e não há um único controlador que possa coordenar um processo semelhante à confirmação em duas fases em repositórios de dados relacionais. Nesse caso, uma solução é usar o padrão de orquestração saga.

Aplicabilidade

Use o padrão de orquestração saga quando:

- Seu sistema exige integridade e consistência de dados em transações distribuídas que abrangem vários repositórios de dados.
- O armazenamento de dados não fornece 2PC para fornecer transações ACID, e implementar 2PC dentro dos limites do aplicativo é uma tarefa complexa.
- Você tem bancos de dados NoSQL, que não fornecem transações ACID, e precisa atualizar várias tabelas em uma única transação.

Problemas e considerações

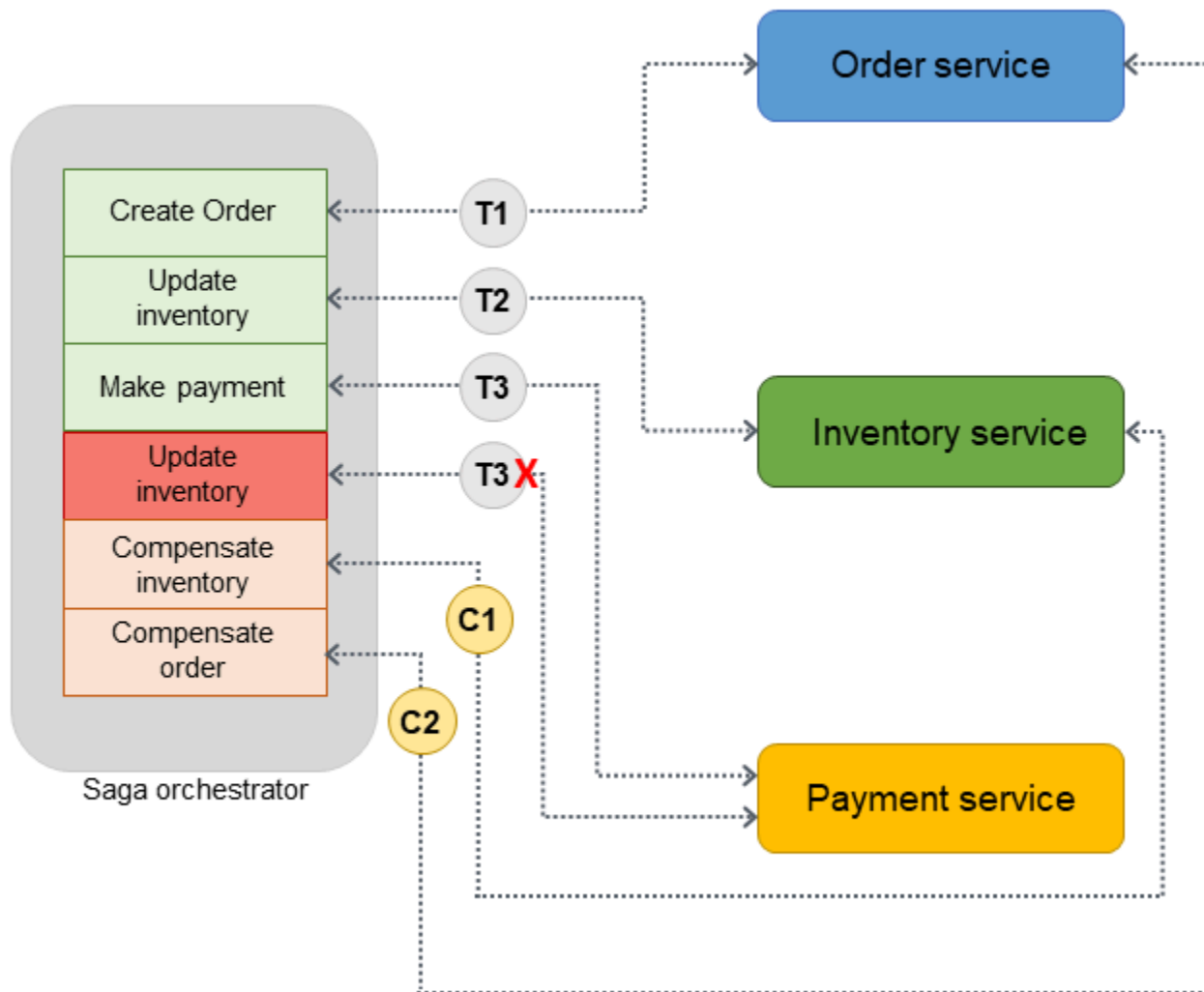
- Complexidade: transações compensatórias e novas tentativas adicionam complexidades ao código do aplicativo, o que pode resultar em sobrecarga de manutenção.
- Consistência eventual: o processamento sequencial de transações locais resulta em consistência eventual, o que pode ser um desafio em sistemas que exigem consistência forte. Você pode resolver esse problema definindo as expectativas de suas equipes comerciais em relação ao modelo de consistência ou mudando para um armazenamento de dados que forneça uma consistência forte.
- Idempotência: os participantes da saga precisam ser idempotentes para permitir a execução repetida em caso de falhas transitórias causadas por falhas inesperadas e falhas do orquestrador.
- Isolamento de transações: saga não tem isolamento de transações. A orquestração simultânea de transações pode levar a dados obsoletos. Recomendamos usar o bloqueio semântico para lidar com esses cenários.
- Observabilidade: observabilidade se refere ao registro e ao rastreamento detalhados para solucionar problemas no processo de execução e orquestração. Isso torna-se importante quando o número de participantes da saga aumenta, resultando em complexidades na depuração.
- Problemas de latência: transações compensatórias podem adicionar latência ao tempo geral de resposta quando a saga consiste em várias etapas. Evite chamadas síncronas nesses casos.

- Ponto único de falha: o orquestrador pode se tornar um ponto único de falha porque coordena toda a transação. Em alguns casos, o padrão de coreografia da saga é preferido por causa dessa questão.

Implementação

High-level arquitetura

No diagrama de arquitetura a seguir, o orquestrador saga tem três participantes: o serviço de pedidos, o serviço de estoque e o serviço de pagamento. São necessárias três etapas para concluir a transação: T1, T2 e T3. O orquestrador saga está ciente das etapas e as executa na ordem necessária. Quando a etapa T3 falha (falha no pagamento), o orquestrador executa as transações compensatórias C1 e C2 para restaurar os dados ao estado inicial.



Você pode usar [AWS Step Functions](#) para implementar a orquestração saga quando a transação é distribuída em vários bancos de dados.

Implementação usando AWS serviços

A solução de amostra usa o fluxo de trabalho padrão em Step Functions para implementar o padrão de orquestração saga.



Quando um cliente chama a API, a função do Lambda é invocada e o pré-processamento ocorre na função do Lambda. A função inicia o fluxo de trabalho do Step Functions para começar a processar a transação distribuída. Se o pré-processamento não for necessário, você poderá [iniciar o fluxo de trabalho do Step Functions diretamente](#) do API Gateway sem usar a função do Lambda.

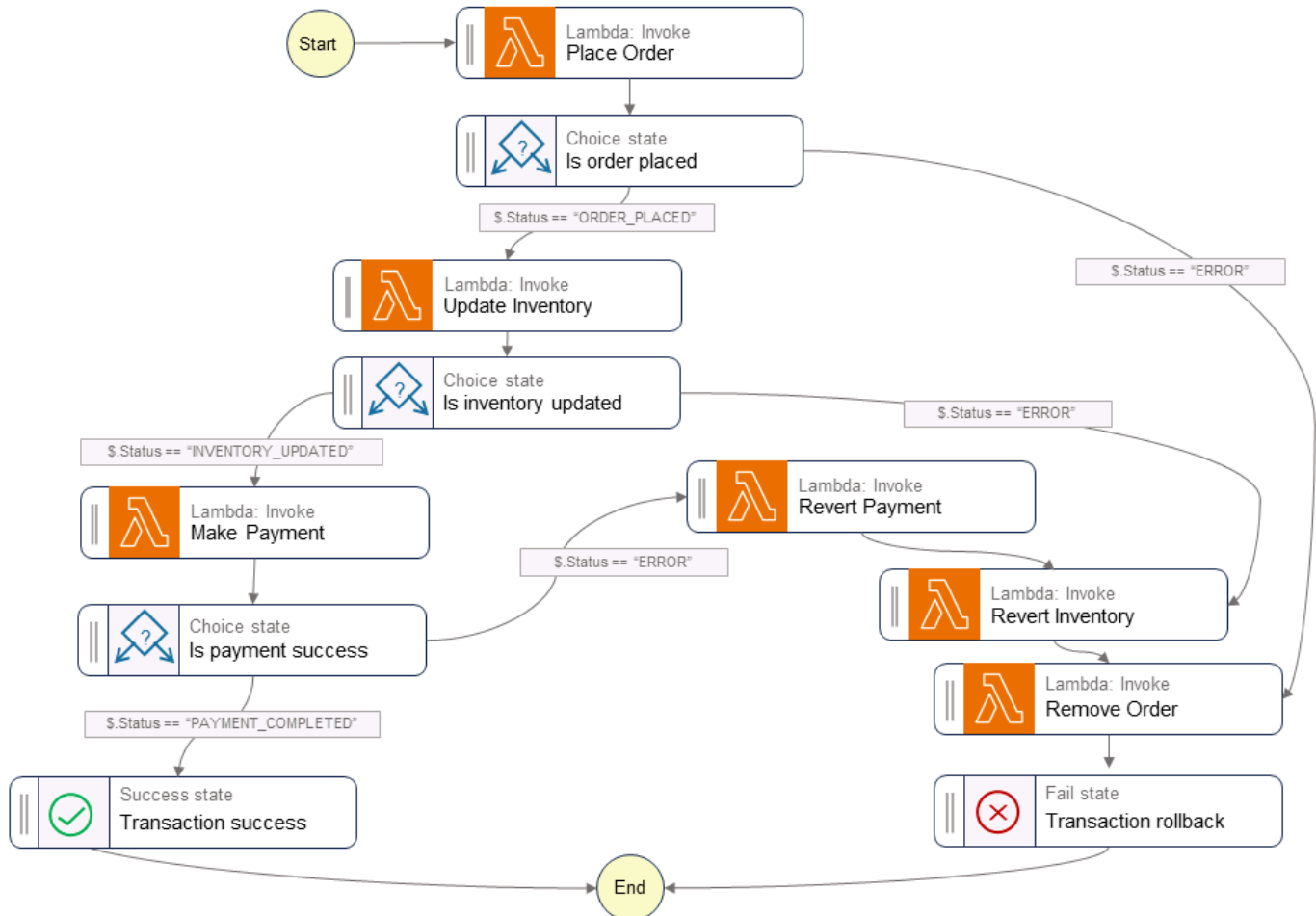
O uso do Step Functions atenua o problema do ponto único de falha, que é inerente à implementação do padrão de orquestração saga. O Step Functions tem a tolerância a falhas integrada e mantém a capacidade de serviço em várias zonas de disponibilidade em cada região da AWS para proteger as aplicações contra falhas individuais em máquinas ou data centers. Isso ajuda a garantir a alta disponibilidade do serviço em si e do fluxo de trabalho do aplicativo que ele opera.

O fluxo de trabalho do Step Functions

A máquina de estado Step Functions permite que você configure os requisitos de fluxo de controle com base em decisão para a implementação do padrão. O fluxo de trabalho do Step Functions chama os serviços individuais para colocação de pedidos, atualização de estoque e processamento de pagamentos para concluir a transação e envia uma notificação de evento para processamento adicional. O fluxo de trabalho do Step Functions atua como orquestrador para coordenar as transações. Se o fluxo de trabalho contiver algum erro, o orquestrador executará as transações compensatórias para garantir que a integridade dos dados seja mantida em todos os serviços.

O diagrama a seguir mostra as etapas que são executadas no fluxo de trabalho do Step Functions. As etapas Place Order, Update Inventory, e Make Payment indicam o caminho do sucesso. O pedido é feito, o estoque é atualizado e o pagamento é processado antes que um Success estado seja devolvido ao chamador.

As funções `Revert Payment`, `Revert Inventory`, e `Remove Order` do Lambda indicam as transações compensatórias que o orquestrador executa quando alguma etapa do fluxo de trabalho falha. Se o fluxo de trabalho falhar na etapa `Update Inventory`, o orquestrador chama as etapas `Revert Inventory` e `Remove Order` e antes de retornar um estado `Fail` ao chamador. Essas transações compensatórias garantem que a integridade dos dados seja mantida. O estoque volta ao nível original e o pedido é revertido.



Código de exemplo

O código de exemplo a seguir mostra como você pode criar um orquestrador saga usando Step Functions. Para ver o código completo, consulte o [GitHub repositório](#) deste exemplo.

Definições de tarefa

```
var successState = new Succeed(this, "SuccessState");
var failState = new Fail(this, "Fail");
```



```
var placeOrderTask = new LambdaInvoke(this, "Place Order", new LambdaInvokeProps
{
    LambdaFunction = placeOrderLambda,
    Comment = "Place Order",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
});

var updateInventoryTask = new LambdaInvoke(this, "Update Inventory", new
    LambdaInvokeProps
{
    LambdaFunction = updateInventoryLambda,
    Comment = "Update inventory",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
});

var makePaymentTask = new LambdaInvoke(this, "Make Payment", new LambdaInvokeProps
{
    LambdaFunction = makePaymentLambda,
    Comment = "Make Payment",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
});

var removeOrderTask = new LambdaInvoke(this, "Remove Order", new LambdaInvokeProps
{
    LambdaFunction = removeOrderLambda,
    Comment = "Remove Order",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
}).Next(failState);

var revertInventoryTask = new LambdaInvoke(this, "Revert Inventory", new
    LambdaInvokeProps
{
    LambdaFunction = revertInventoryLambda,
    Comment = "Revert inventory",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
}).Next(removeOrderTask);

var revertPaymentTask = new LambdaInvoke(this, "Revert Payment", new LambdaInvokeProps
{
```

```

    LambdaFunction = revertPaymentLambda,
    Comment = "Revert Payment",
    RetryOnServiceExceptions = false,
    PayloadResponseOnly = true
  }).Next(revertInventoryTask);

var waitState = new Wait(this, "Wait state", new WaitProps
{
    Time = WaitTime.Duration(Duration.Seconds(30))
}).Next(revertInventoryTask);

```

Definições de Step function e máquina de estado

```

var stepDefinition = placeOrderTask
    .Next(new Choice(this, "Is order placed")
        .When(Condition.StringEquals("$.Status", "ORDER_PLACED"),
            updateInventoryTask
                .Next(new Choice(this, "Is inventory updated")
                    .When(Condition.StringEquals("$.Status",
                        "INVENTORY_UPDATED"),
                        makePaymentTask.Next(new Choice(this, "Is payment
                            success")
                                .When(Condition.StringEquals("$.Status",
                                    "PAYMENT_COMPLETED"), successState)
                                .When(Condition.StringEquals("$.Status", "ERROR"),
                                    revertPaymentTask)))
                    .When(Condition.StringEquals("$.Status", "ERROR"),
                        waitState)))
        .When(Condition.StringEquals("$.Status", "ERROR"), failState));

var stateMachine = new StateMachine(this, "DistributedTransactionOrchestrator", new
    StateMachineProps {
        StateMachineName = "DistributedTransactionOrchestrator",
        StateMachineType = StateMachineType.STANDARD,
        Role = iamStepFunctionRole,
        TracingEnabled = true,
        Definition = stepDefinition
    });

```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/saga-orchestration-netcore-blog>.

Referências do blog

- [Construir um aplicativo distribuído sem servidor usando o padrão Saga Orchestration](#)

Conteúdo relacionado

- [Padrão de coreografia saga](#)
- [Padrão de caixa de saída transacional](#)

Vídeos

O vídeo a seguir analisa como implementar o padrão de orquestração saga usando o AWS Step Functions.

Scatter-gather padrão

Intenção

O padrão scatter-gather é um padrão de roteamento de mensagens que envolve a transmissão de solicitações semelhantes ou relacionadas para vários destinatários e a agregação de suas respostas em uma única mensagem usando um componente chamado agregador. Esse padrão ajuda a alcançar a paralelização, reduz a latência de processamento e gerencia a comunicação assíncrona. É fácil implementar o padrão scatter-gather usando uma abordagem síncrona, mas uma abordagem mais robusta envolve implementá-lo como roteamento de mensagens em comunicação assíncrona, com ou sem um serviço de mensagens.

Motivação

No processamento de aplicações, uma solicitação que pode levar muito tempo para ser processada sequencialmente pode ser dividida em várias solicitações processadas paralelamente. Você também pode enviar solicitações para vários sistemas externos por meio de chamadas de API para obter uma resposta. O padrão scatter-gather é útil quando você precisa de informações de várias fontes. Scatter-gather agrega os resultados para ajudá-lo a tomar uma decisão informada ou selecionar a melhor resposta para a solicitação.

O padrão scatter-gather consiste em duas fases, como o próprio nome indica:

- A fase de dispersão processa a mensagem de solicitação e a envia para vários destinatários em paralelo. Durante essa fase, a aplicação dispersa as solicitações pela rede e continua sendo executada sem esperar por respostas imediatas.
- Durante a fase de coleta, a aplicação coleta as respostas dos destinatários e as filtra ou combina em uma resposta unificada. Quando todas as respostas tiverem sido coletadas, elas poderão ser agregadas em uma única resposta, ou a melhor resposta poderá ser escolhida para processamento posterior.

Aplicabilidade

Use o padrão scatter-gather quando:

- Você planeja agregar e consolidar dados de várias APIs para criar uma resposta precisa. O padrão consolida informações de fontes diferentes em um todo coeso. Por exemplo, um sistema de agendamento pode fazer uma solicitação a vários destinatários para obter cotações de vários parceiros externos.
- A mesma solicitação precisa ser enviada para vários destinatários simultaneamente para concluir uma transação. Por exemplo, você pode usar esse padrão para consultar dados de inventário em paralelo e verificar a disponibilidade de um produto.
- Você deseja implementar um sistema confiável e escalável em que o balanceamento de carga possa ser obtido distribuindo solicitações entre vários destinatários. Se um destinatário falhar ou enfrentar uma carga alta, outros destinatários ainda poderão processar solicitações.
- Você quer otimizar a performance ao implementar consultas complexas que envolvem várias fontes de dados. Você pode dispersar a consulta em bancos de dados relevantes, reunir os resultados parciais e combiná-los em uma resposta abrangente.
- Você está implementando um tipo de processamento de redução de mapas em que a solicitação de dados é roteada para vários endpoints de processamento de dados para fragmentação e replicação. Os resultados parciais são filtrados e combinados para compor a resposta correta.
- Você deseja distribuir as operações de gravação em um espaço de chave de partição em workloads com gravação intensa em banco de dados de chave-valor. O agregador lê os resultados consultando os dados em cada fragmento e os consolida em uma única resposta.

Problemas e considerações

- Tolerância a falhas: esse padrão depende de vários destinatários que trabalham em paralelo, por isso é essencial lidar com as falhas normalmente. Para mitigar o impacto das falhas do destinatário no sistema geral, você pode implementar estratégias como redundância, replicação e detecção de falhas.
- Scale-out limites: à medida que o número total de nós de processamento aumenta, a sobrecarga da rede associada também aumenta. Cada solicitação que envolve comunicação pela rede pode aumentar a latência e afetar negativamente os benefícios da paralelização.
- Gargalos de tempo de resposta: para operações que exigem que todos os destinatários sejam processados antes que o processamento final seja concluído, a performance geral do sistema é limitada pelo tempo de resposta mais lento do destinatário.
- Respostas parciais: quando as solicitações são dispersadas para vários destinatários, alguns destinatários podem atingir o tempo limite. Nesses casos, a implementação deve comunicar ao

cliente que a resposta está incompleta. Você também pode exibir os detalhes da agregação de respostas usando um frontend de interface de usuário.

- **Consistência de dados:** ao processar dados de vários destinatários, você deve considerar cuidadosamente as técnicas de sincronização de dados e resolução de conflitos, para garantir que os resultados agregados finais sejam precisos e consistentes.

Implementação

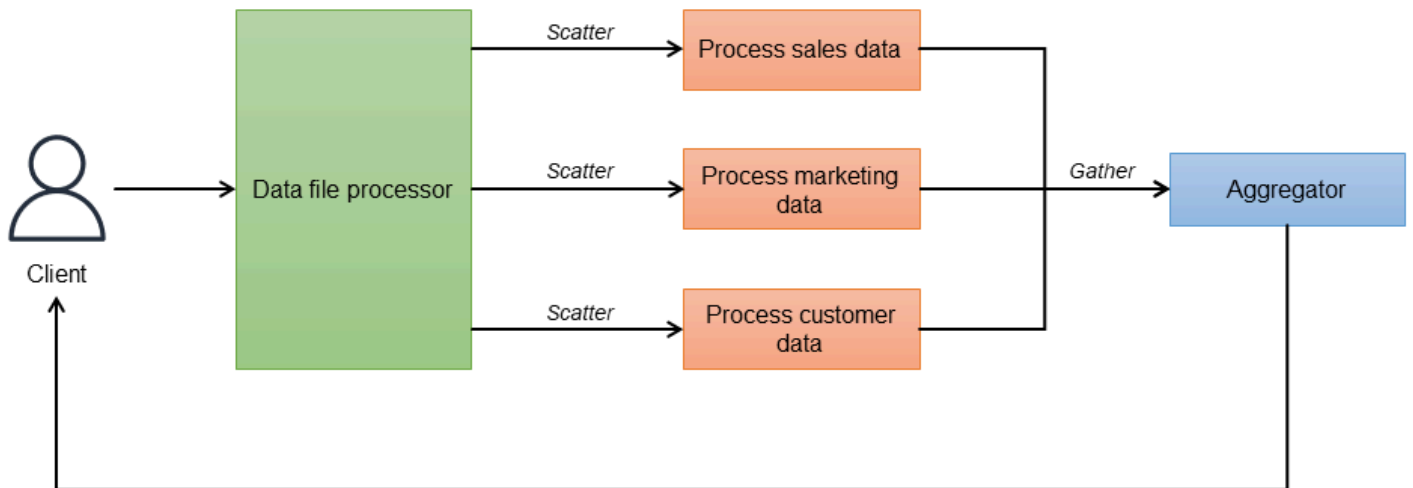
High-level arquitetura

O padrão scatter-gather usa um controlador raiz para distribuir solicitações aos destinatários que processarão as solicitações. Durante a fase de dispersão, esse padrão pode usar dois mecanismos para enviar mensagens aos destinatários:

- **Dispersão por distribuição:** a aplicação tem uma lista conhecida de destinatários que devem ser chamados para obter os resultados. Os destinatários podem ser processos diferentes com funções exclusivas ou um único processo que teve a escala aumentada horizontalmente para distribuir a carga de processamento. Se algum dos nós de processamento atingir o tempo limite ou mostrar atrasos na resposta, o controlador poderá redistribuir o processamento para outro nó.
- **Dispersão por leilão:** a aplicação transmite a mensagem aos destinatários interessados usando um [padrão de publicação-assinatura](#). Nesse caso, os destinatários podem assinar a mensagem ou cancelar a assinatura a qualquer momento.

Dispersão por distribuição

No método de dispersão por distribuição, o controlador raiz divide a solicitação recebida em tarefas independentes e as atribui aos destinatários disponíveis (a fase de dispersão). Cada destinatário (processo, contêiner ou função do Lambda) trabalha de forma independente e paralela em sua computação e produz uma parte da resposta. Quando os destinatários concluem suas tarefas, eles enviam suas respostas para um agregador (a fase de coleta). O agregador combina as respostas parciais e retorna o resultado final ao cliente. O diagrama a seguir ilustra esse fluxo de trabalho.



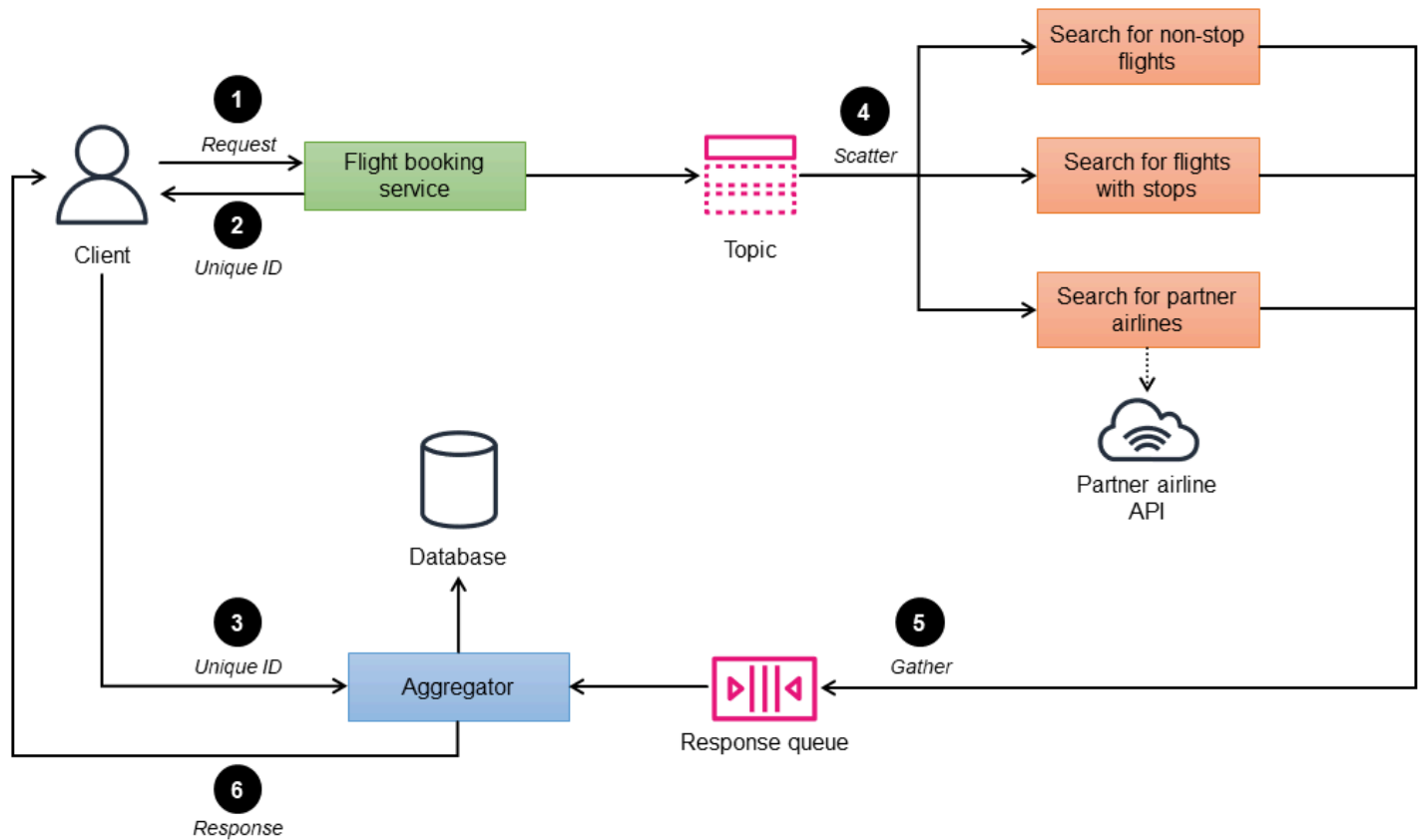
O controlador (processador de arquivos de dados) orquestra todo o conjunto de invocações e está ciente de todos os endpoints de reserva a serem chamados. Ele pode configurar um parâmetro de tempo limite para ignorar respostas que demoram muito. Quando as solicitações são enviadas, o agregador aguarda as respostas de cada endpoint. Para implementar a resiliência, cada microsserviço pode ser implantado com várias instâncias para balanceamento de carga. O agregador obtém os resultados, os combina em uma única mensagem de resposta e remove dados duplicados antes de continuar o processamento. As respostas que expiram são ignoradas. O controlador também pode atuar como um agregador em vez de usar um serviço agregador separado.

Dispersão por leilão

Se o controlador não estiver ciente dos destinatários, ou se os destinatários estiverem com acoplamento fraco, você poderá usar o método de dispersão por leilão. Nesse método, os destinatários se inscrevem em um tópico e o controlador publica a solicitação no tópico. Os destinatários publicam os resultados em uma fila de respostas. Como o controlador raiz não conhece os destinatários, o processo de coleta usa um agregador (outro padrão de mensagens) para coletar as respostas e transformá-las em uma única mensagem de resposta. O agregador usa um ID exclusivo para identificar um grupo de solicitações.

Por exemplo, no diagrama a seguir, o método de dispersão por leilão é usado para implementar um serviço de reserva de voos no site de uma companhia aérea. O site permite que os usuários pesquisem e exibam voos da própria companhia aérea e das companhias aéreas parceiras, e deve exibir o status da pesquisa em tempo real. O serviço de reserva de voos consiste em três microsserviços de busca: voos sem escala, voos com escalas e companhias aéreas parceiras.

A pesquisa da companhia aérea parceira chama os endpoints da API do parceiro para obter as respostas.

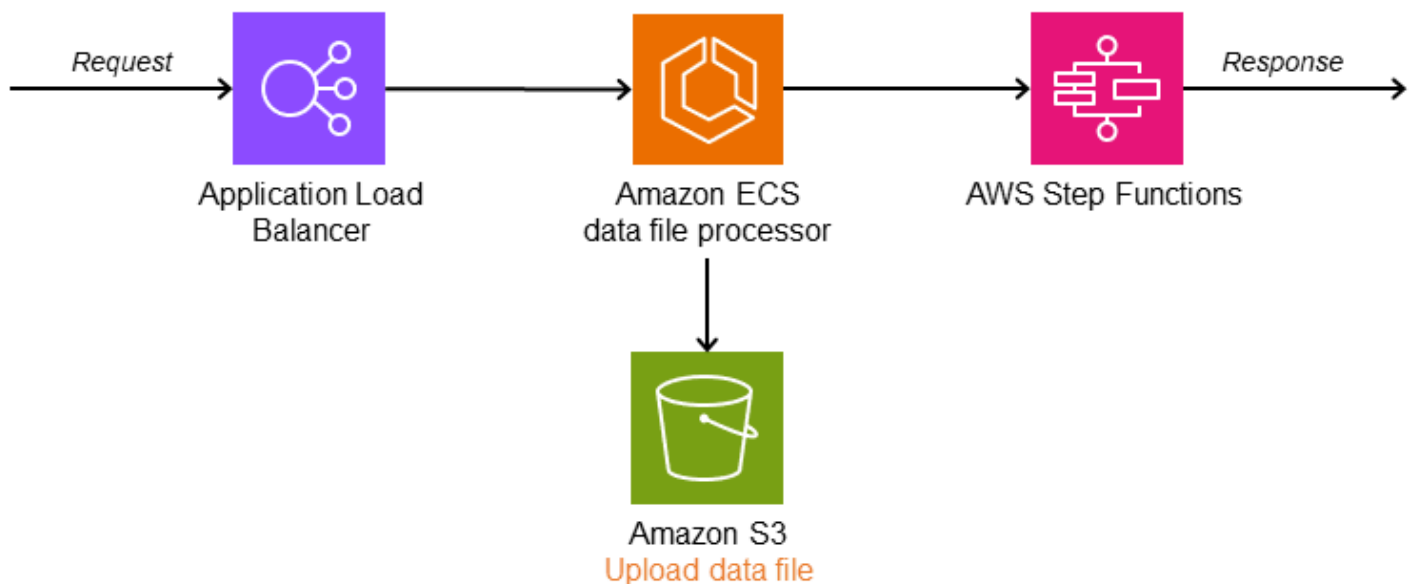


1. O serviço de reserva de voos (controlador) usa os critérios de pesquisa como entrada do cliente e processa e publica a solicitação no tópico.
2. O controlador usa uma ID exclusiva para identificar cada grupo de solicitações.
3. O cliente envia a ID exclusiva para o agregador na etapa 6.
4. Os microserviços de pesquisa de reservas que se inscreveram no tópico de reserva recebem a solicitação.
5. Os microserviços processam a solicitação e devolvem a disponibilidade de assentos para os critérios de pesquisa fornecidos em uma fila de resposta.
6. O agregador agrupa todas as mensagens de resposta armazenadas em um banco de dados temporário, agrupa os voos por ID exclusiva, cria uma única resposta unificada e a envia de volta ao cliente.

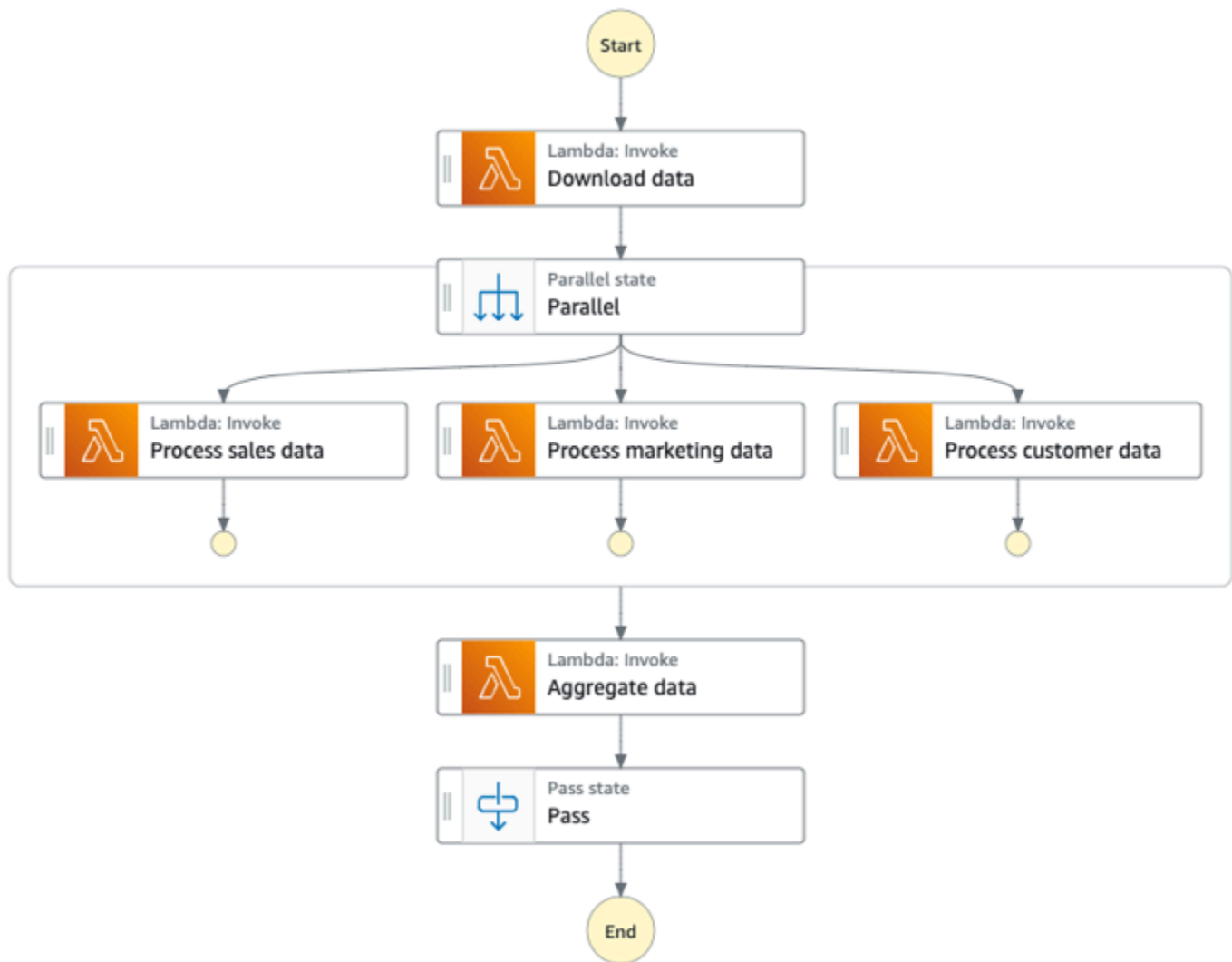
Implementação usando Serviços da AWS

Dispersão por distribuição

Na arquitetura a seguir, o controlador raiz é um processador de arquivos de dados (Amazon ECS) que divide os dados da solicitação recebida em buckets individuais do Amazon Simple Storage Service (Amazon S3) e inicia um fluxo de trabalho. AWS Step Functions O fluxo de trabalho baixa os dados e inicia o processamento paralelo dos arquivos. O estado `Parallel` espera que todas as tarefas retornem uma resposta. Uma AWS Lambda função agrega os dados e os salva de volta no Amazon S3.

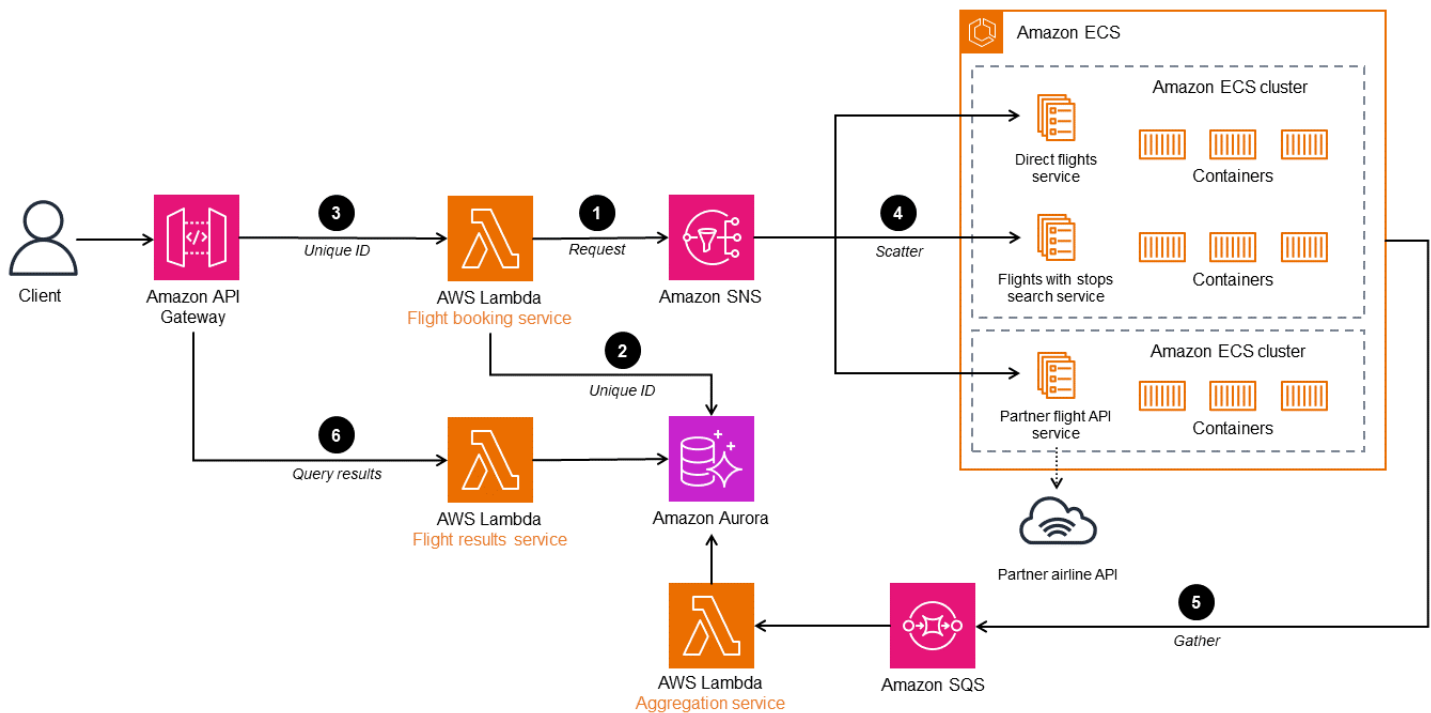


O diagrama a seguir ilustra o fluxo de trabalho do Step Functions com o estado `Parallel`.



Dispersão por leilão

O diagrama a seguir mostra uma AWS arquitetura para o método de dispersão por leilão. O serviço de reserva de voos do controlador raiz distribui a solicitação de pesquisa de voos em vários microsserviços. Um canal de publicação/assinatura é implementado com o Amazon Simple Notification Service (Amazon SNS), que é um serviço de mensagens gerenciado para comunicações. O Amazon SNS oferece suporte a mensagens entre aplicações de microsserviços desacoplados ou comunicações diretas com os usuários. Você pode implantar os microsserviços do destinatário no Amazon Elastic Kubernetes Service (Amazon EKS) ou no Amazon Elastic Container Service (Amazon ECS) para um melhor gerenciamento e escalabilidade. O serviço de resultados do voo retorna os resultados para o cliente. Ele pode ser implementado em AWS Lambda ou em outros serviços de orquestração de contêineres, como Amazon ECS ou Amazon EKS.



1. O serviço de reserva de voos (controlador) usa os critérios de pesquisa como entrada do cliente e processa e publica a solicitação no tópico do SNS.
2. O controlador publica o ID exclusivo em um banco de dados Amazon Aurora para identificar a solicitação.
3. O cliente envia a ID exclusiva para o cliente na etapa 6.
4. Os microsserviços de pesquisa de reservas que se inscreveram no tópico de reserva recebem a solicitação.
5. Os microsserviços processam a solicitação e devolvem a disponibilidade de assentos para os critérios de pesquisa fornecidos para uma fila de resposta no Amazon Simple Queue Service (Amazon SQS). O agregador agrupa todas as mensagens de resposta e as armazena em um banco de dados temporário.
6. O serviço de resultados de voos agrupa os voos por ID exclusivo, cria uma única resposta unificada e a envia de volta ao cliente.

Se você quiser adicionar outra pesquisa aérea a essa arquitetura, adicione um microsserviço que assina o tópico do SNS e publica na fila do SQS.

Resumindo, o padrão scatter-gather permite que os sistemas distribuídos alcancem uma paralelização eficiente, reduzam a latência e lidem perfeitamente com a comunicação assíncrona.

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/asynchronous-messaging-workshop/tree/master/code/lab-3>.

Workshop

- [Scatter-gather laboratório](#) no workshop de microsserviços desacoplados

Referências do blog

- [Application integration patterns for microservices](#)

Conteúdo relacionado

- Padrão [Publish-subscribe](#)

padrão strangler fig

Intenção

O padrão strangler fig ajuda a migrar incrementalmente uma aplicação monolítica para uma arquitetura de microsserviços, com risco de transformação e interrupção dos negócios reduzidos.

Motivação

As aplicações monolíticas são desenvolvidas para fornecer a maior parte de suas funcionalidades em um único processo ou contêiner. O código está fortemente acoplado. Como resultado, as alterações na aplicação exigem um novo teste completo para evitar problemas de regressão. As mudanças não podem ser testadas isoladamente, o que afeta o tempo do ciclo. À medida que a aplicação é aprimorada com mais recursos, a alta complexidade pode levar a mais tempo gasto em manutenção, maior tempo de lançamento no mercado e, conseqüentemente, retardar a inovação do produto.

Quando a aplicação aumenta de tamanho, ela aumenta a carga cognitiva da equipe e pode ocasionar limites pouco claros de propriedade da equipe. Não é possível escalar recursos individuais com base na carga. Toda a aplicação precisa ser escalada para suportar picos de carga. À medida que os sistemas envelhecem, a tecnologia pode se tornar obsoleta, o que aumenta os custos de suporte. As aplicações monolíticas legadas seguem as práticas recomendadas que estavam disponíveis no momento do desenvolvimento e não foram projetadas para serem distribuídas.

Quando uma aplicação monolítica é migrada para uma arquitetura de microsserviços, ela pode ser dividida em componentes menores. Esses componentes podem ser escalados de forma independente, podem ser lançados de forma independente e podem ser de propriedade de equipes individuais. Isso resulta em uma maior velocidade de mudança, porque as alterações são localizadas e podem ser testadas e liberadas rapidamente. As mudanças têm um escopo de impacto menor porque os componentes têm acoplamento fraco e podem ser implantados individualmente.

Substituir completamente um monólito por uma aplicação de microsserviços reescrevendo ou refatorando o código é uma grande tarefa e um grande risco. Uma grande migração, em que o monólito é migrado em uma única operação, introduz risco de transformação e interrupção nos negócios. Enquanto a aplicação está sendo refatorada, é extremamente difícil ou mesmo impossível adicionar novos recursos.

Uma forma de resolver esse problema é usar o padrão fig pattern, introduzido por Martin Fowler. Esse padrão envolve migrar para microsserviços extraindo gradualmente recursos e criando uma nova aplicação em torno do sistema existente. Os recursos do monólito são substituídos gradualmente por microsserviços, e os usuários da aplicação podem usar os recursos recém-migrados progressivamente. Quando todos os recursos são transferidos para o novo sistema, a aplicação monolítica pode ser desativada com segurança.

Aplicabilidade

Use o padrão strangler fig quando:

- Você deseja migrar gradualmente sua aplicação monolítica para uma arquitetura de microsserviços.
- Uma abordagem de migração do big bang é arriscada devido ao tamanho e à complexidade do monólito.
- A empresa quer adicionar novos recursos e não pode esperar que a transformação seja concluída.
- Os usuários finais devem ser minimamente afetados durante a transformação.

Problemas e considerações

- Acesso à base de código: para implementar o padrão strangler fig, você deve ter acesso à base de código da aplicação monolítica. À medida que os recursos são migrados do monólito, você precisará fazer pequenas alterações no código e implementar uma camada anticorrupção dentro do monólito para encaminhar chamadas para novos microsserviços. Você não pode interceptar chamadas sem acesso à base de código. O acesso à base de código também é essencial para redirecionar as solicitações recebidas. Talvez seja necessária alguma refatoração de código para que a camada de proxy possa interceptar as chamadas dos recursos migrados e roteá-las para microsserviços.
- Domínio pouco claro: a decomposição prematura dos sistemas pode ser cara, especialmente quando o domínio não está claro e é possível errar os limites do serviço. Domain-driven design (DDD) é um mecanismo para entender o domínio, e o armazenamento de eventos é uma técnica para determinar os limites do domínio.
- Identificação de microsserviços: você pode usar o DDD como uma ferramenta essencial para identificar microsserviços. Para identificar microsserviços, procure as divisões naturais entre as classes de serviço. Muitos serviços terão seu próprio objeto de acesso a dados e se separarão

facilmente. Serviços que têm lógica de negócios relacionada e classes que têm poucas ou nenhuma dependência são bons candidatos para microsserviços. Você pode refatorar o código antes de quebrar o monólito para evitar um acoplamento forte. Você também deve considerar os requisitos de conformidade, o ritmo de lançamento, a localização geográfica das equipes, as necessidades de escalabilidade, as necessidades de tecnologia orientadas por casos de uso e a carga cognitiva das equipes.

- **Anti-corruption camada:** durante o processo de migração, quando os recursos dentro do monólito precisam chamar os recursos que foram migrados como microsserviços, você deve implementar uma camada anticorrupção (ACL) que roteie cada chamada para o microsserviço apropriado. Para desacoplar e evitar alterações nos chamadores existentes dentro do monólito, a ACL funciona como um adaptador ou uma fachada que converte as chamadas na interface mais recente. Isso é discutido em detalhes na [seção Implementação](#) do padrão da ACL, anteriormente neste guia.
- **Falha na camada de proxy:** durante a migração, uma camada de proxy intercepta as solicitações que vão para a aplicação monolítica e as roteia para o sistema legado ou para o novo sistema. No entanto, essa camada de proxy pode se tornar um único ponto de falha ou um gargalo de performance.
- **Complexidade da aplicação:** monólitos grandes são os que mais se beneficiam do padrão strangler fig. Para aplicações pequenas, em que a complexidade da refatoração completa é baixa, talvez seja mais eficiente reescrever a aplicação na arquitetura de microsserviços em vez de migrá-la.
- **Interações de serviço:** os microsserviços podem se comunicar de forma síncrona ou assíncrona. Quando a comunicação síncrona for necessária, considere se os tempos limite podem causar o consumo da conexão ou do grupo de threads, resultando em problemas de performance da aplicação. Nesses casos, use o [padrão do disjuntor](#) para retornar a falha imediata de operações que provavelmente falharão por longos períodos de tempo. A comunicação assíncrona pode ser obtida usando eventos e filas de mensagens.
- **Agregação de dados:** em uma arquitetura de microsserviços, os dados são distribuídos entre bancos de dados. Quando a agregação de dados é necessária, você pode usar o [AWS AppSync](#) no frontend ou o padrão de segmentação de responsabilidade de consulta de comando (CQRS) no backend.
- **Consistência de dados:** os microsserviços possuem seu armazenamento de dados, e a aplicação monolítica também pode provavelmente usar esses dados. Para permitir o compartilhamento, você pode sincronizar o armazenamento de dados dos novos microsserviços com o banco de dados da aplicação monolítica usando uma fila e um agente. No entanto, isso pode causar redundância de dados e eventual consistência entre dois armazenamentos de dados, por isso recomendamos

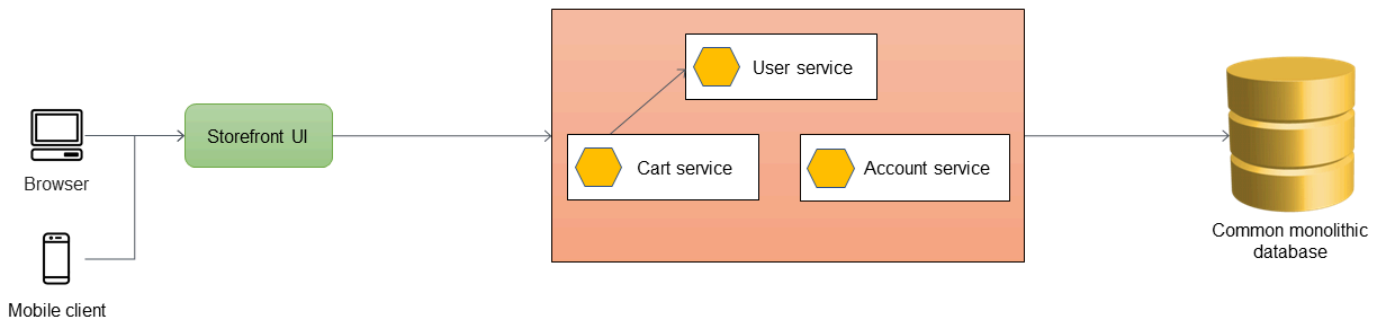
que você a trate como uma solução tática até que você possa estabelecer uma solução de longo prazo, como um data lake.

Implementação

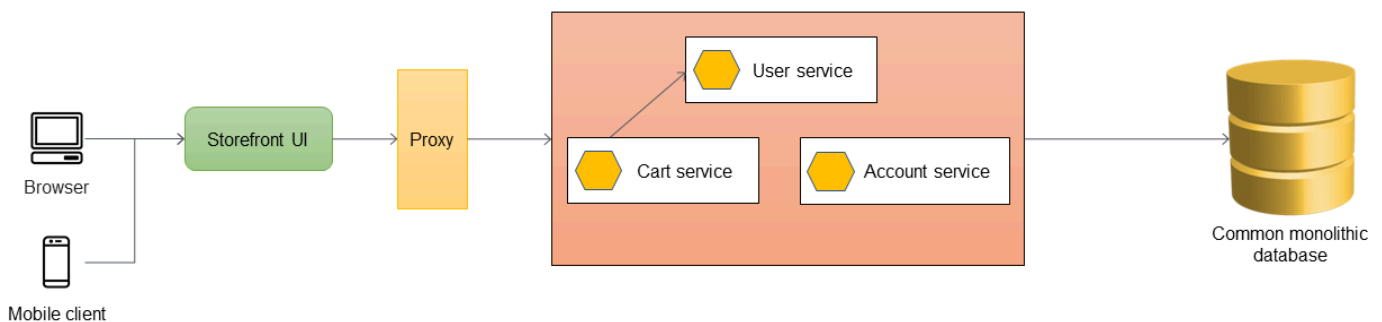
No padrão strangler fig, você substitui uma funcionalidade específica por um novo serviço ou aplicação, um componente por vez. Uma camada de proxy intercepta as solicitações que vão para a aplicação monolítica e as roteia para o sistema legado ou para o novo sistema. Como a camada de proxy roteia os usuários para a aplicação correta, você pode adicionar recursos ao novo sistema e, ao mesmo tempo, garantir que o monólito continue funcionando. O novo sistema eventualmente substitui todos os recursos do sistema antigo, e você pode desativá-lo.

High-level arquitetura

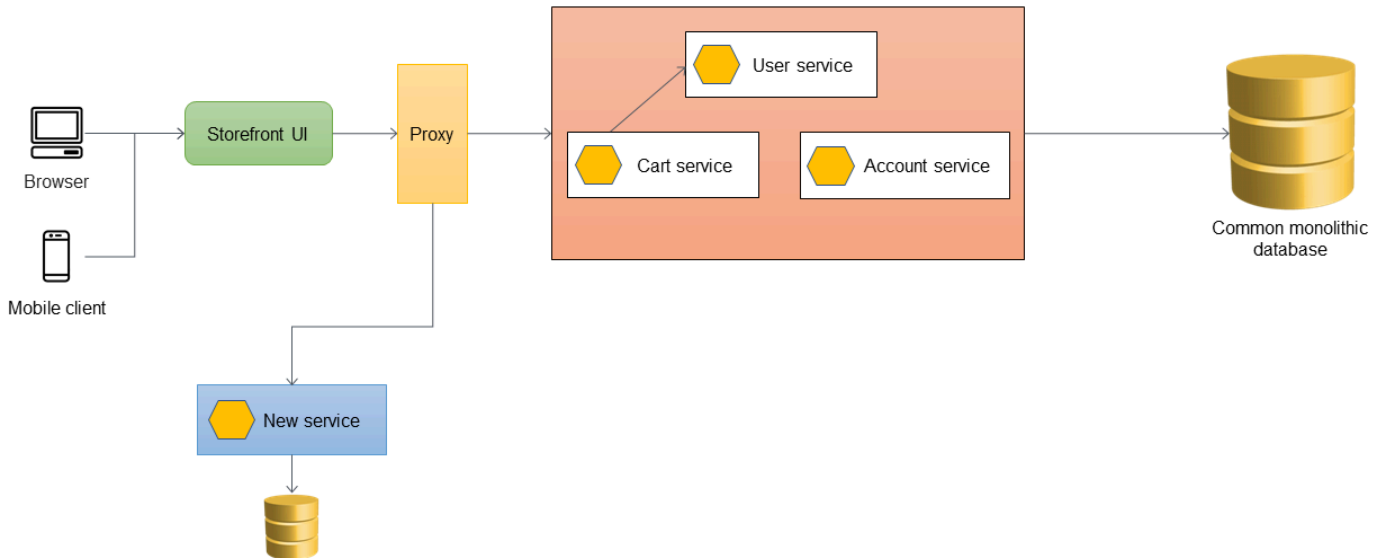
No diagrama a seguir, uma aplicação monolítica tem três serviços: serviço de usuário, serviço de carrinho e serviço de conta. O serviço de carrinho depende do serviço do usuário, e a aplicação usa um banco de dados relacional monolítico.



A primeira etapa é adicionar uma camada de proxy entre a interface do usuário do storefront e a aplicação monolítica. No início, o proxy roteia todo o tráfego para a aplicação monolítica.

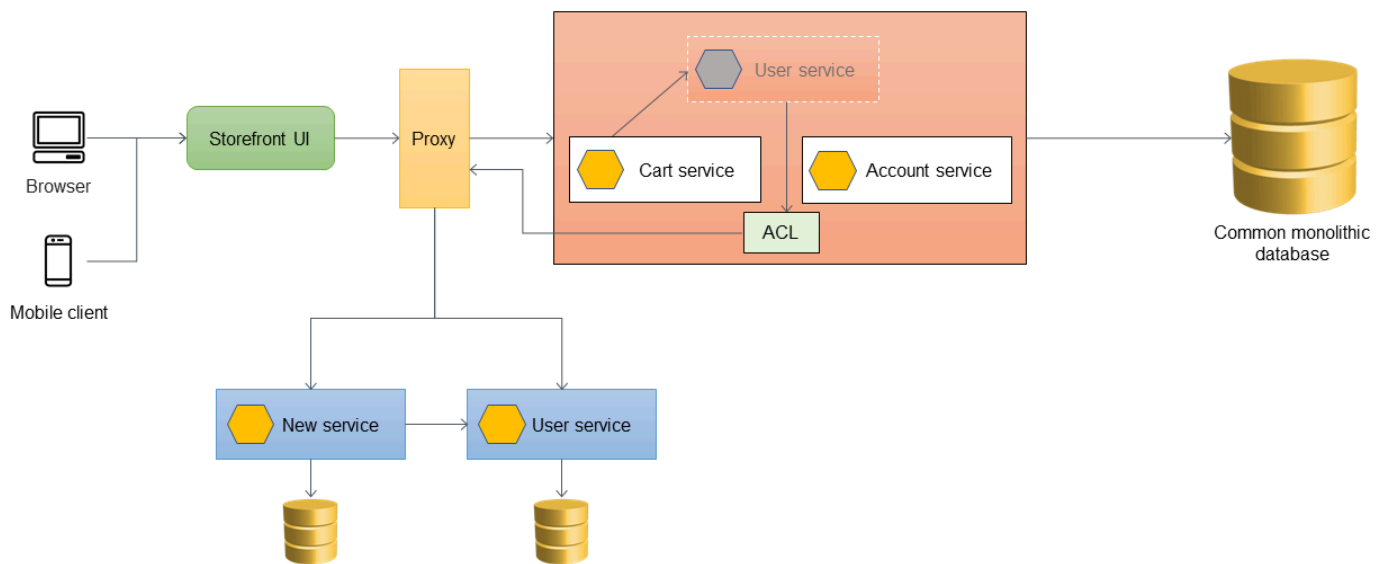


Quando quiser adicionar novos recursos à sua aplicação, você os implementa como novos microsserviços em vez de adicionar recursos ao monólito existente. No entanto, você continua corrigindo bugs no monólito para garantir a estabilidade da aplicação. No diagrama a seguir, a camada de proxy roteia as chamadas para o monólito ou para o novo microsserviço com base no URL da API.



Adição de uma camada anticorrupção

Na arquitetura a seguir, o serviço do usuário foi migrado para um microsserviço. O serviço de carrinho chama o serviço de usuário, mas a implementação não está mais disponível no monólito. Além disso, a interface do serviço recém-migrado pode não corresponder à interface anterior dentro da aplicação monolítica. Para lidar com essas alterações, você implementa uma ACL. Durante o processo de migração, quando os recursos do monólito precisam chamar os recursos que foram migrados como microsserviços, a ACL converte as chamadas na nova interface e as roteia para o microsserviço apropriado.

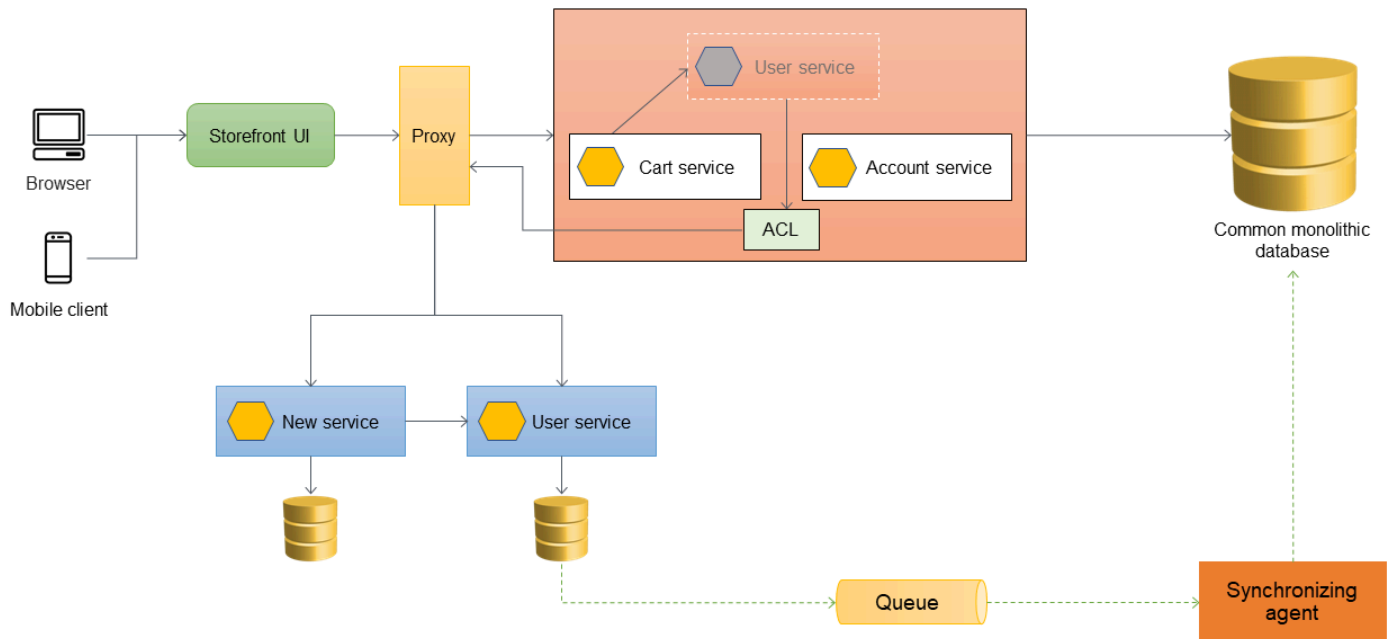


Você pode implementar a ACL dentro da aplicação monolítica como uma classe específica do serviço que foi migrado, por exemplo, `UserServiceFacade` ou `UserServiceAdapter`. A ACL deve ser desativada após a migração de todos os serviços dependentes para a arquitetura de microserviços.

Quando você usa a ACL, o serviço de carrinho ainda chama o serviço do usuário dentro do monólito, e o serviço do usuário redireciona a chamada para o microserviço por meio da ACL. O serviço de carrinho ainda deve chamar o serviço do usuário sem estar ciente da migração do microserviço. Esse acoplamento fraco é necessário para reduzir a regressão e a interrupção dos negócios.

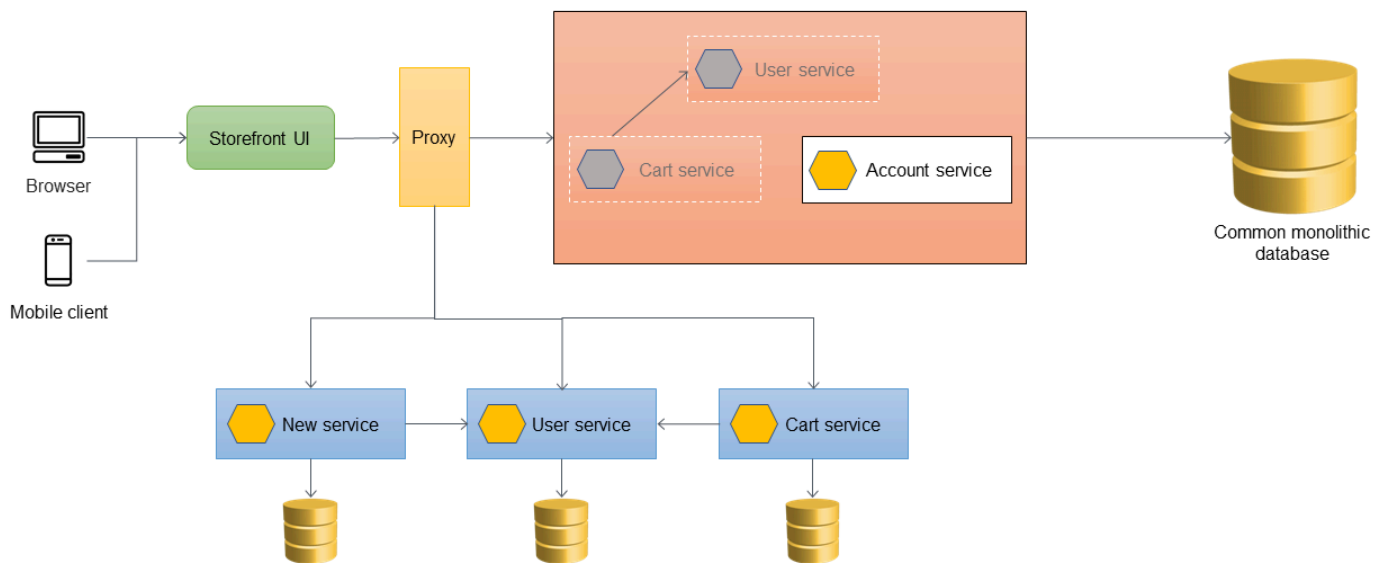
Gerenciamento da sincronização de dados

Como prática recomendada, o microserviço deve possuir os dados. O serviço do usuário armazena seus dados em seu próprio armazenamento de dados. Talvez seja necessário sincronizar dados com o banco de dados monolítico para lidar com dependências, como relatórios, e oferecer suporte a aplicações downstream que ainda não estão prontas para acessar diretamente os microserviços. A aplicação monolítica também pode exigir os dados de outras funções e componentes que ainda não foram migrados para microserviços. Portanto, a sincronização de dados é necessária entre o novo microserviço e o monólito. Para sincronizar os dados, você pode introduzir um agente de sincronização entre o microserviço do usuário e o banco de dados monolítico, conforme mostrado no diagrama a seguir. O microserviço do usuário envia um evento para a fila sempre que seu banco de dados é atualizado. O agente de sincronização escuta a fila e atualiza continuamente o banco de dados monolítico. Os dados no banco de dados monolítico acabam sendo consistentes com os dados que estão sendo sincronizados.



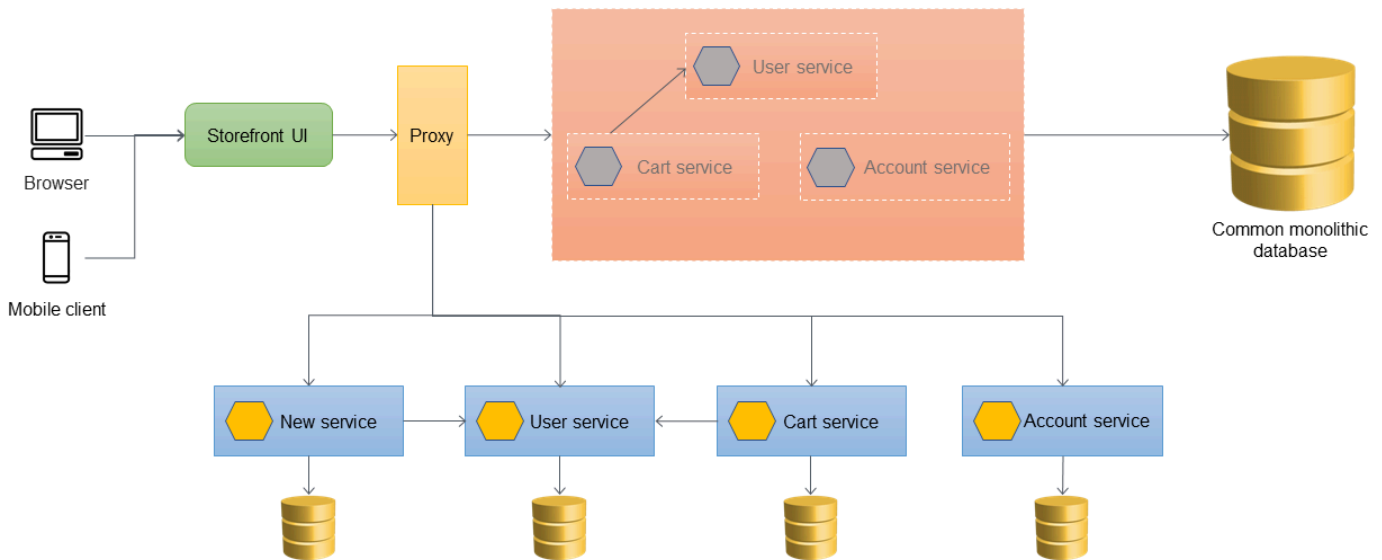
Migração de serviços adicionais

Quando o serviço de carrinho é migrado da aplicação monolítica, seu código é revisado para chamar o novo serviço diretamente, de forma que a ACL não roteie mais essas chamadas. O diagrama a seguir ilustra esse cenário

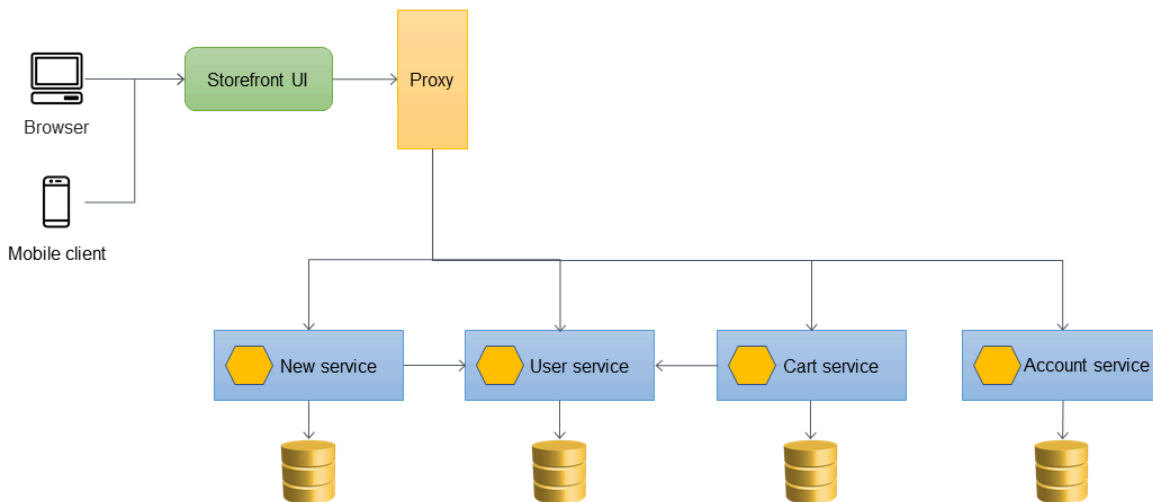


O diagrama a seguir mostra o estado final do processo de estrangulamento, em que todos os serviços foram migrados para fora do monólito, restando apenas o seu esqueleto. Os dados

históricos podem ser migrados para armazenamentos de dados pertencentes a serviços individuais. A ACL pode ser removida e o monólito está pronto para ser desativado neste estágio.



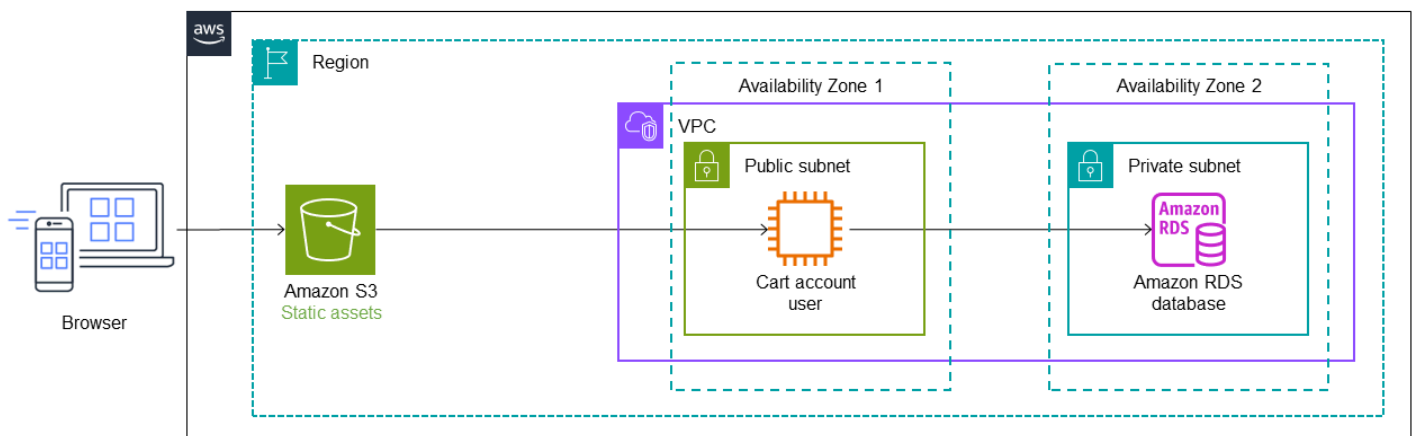
O diagrama a seguir mostra a arquitetura final após a desativação da aplicação monolítica. Você pode hospedar os microsserviços individuais por meio de um URL baseado em recursos (como `http://www.storefront.com/user`) ou por meio de seu próprio domínio (por exemplo, `http://user.storefront.com`) com base nos requisitos da sua aplicação. Para obter mais informações sobre os principais métodos para expor as APIs HTTP aos consumidores upstream usando nomes de host e caminhos, consulte a seção [Padrões de roteamento de API](#).



Implementação usando AWS serviços

Uso do API Gateway como proxy da aplicação

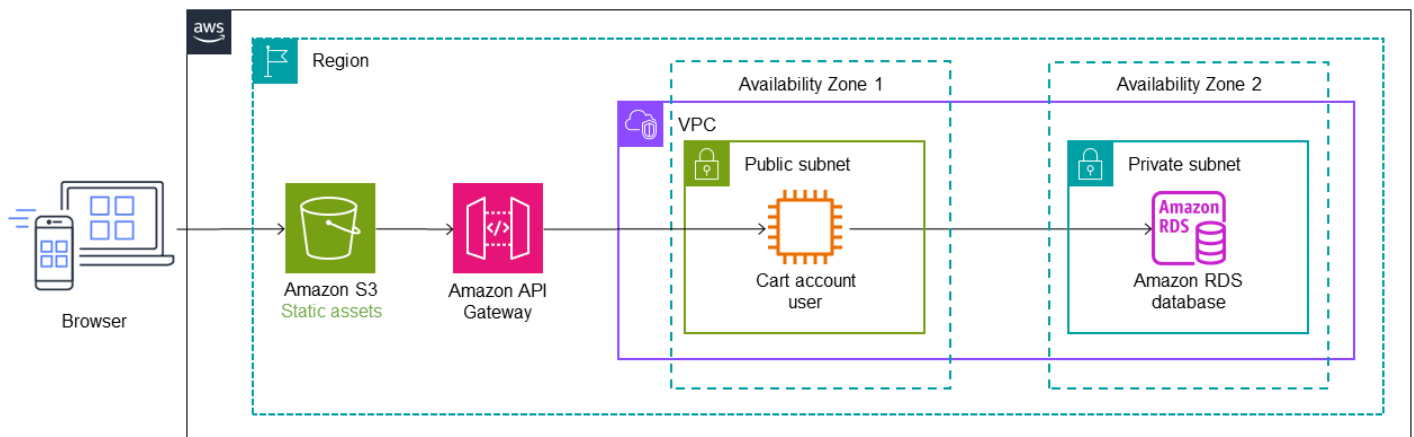
O diagrama a seguir mostra o estado inicial da aplicação monolítica. Vamos supor que ele foi migrado usando uma estratégia de elevação e mudança, portanto, está sendo executado em uma instância do Amazon [Elastic Compute Cloud \(Amazon EC2\)](#) e usa um banco de dados do [Amazon Relational Database Service \(Amazon RDS\)](#). Para simplificar, a arquitetura usa uma única nuvem privada virtual (VPC) com uma sub-rede privada e uma pública, e vamos supor que os microsserviços serão inicialmente implantados na mesma Conta da AWS. (A melhor prática em ambientes de produção é usar uma arquitetura de várias contas para garantir a independência da implantação.) A instância do EC2 reside em uma única zona de disponibilidade na sub-rede pública, e a instância do RDS reside em uma única zona de disponibilidade na sub-rede privada. [O Amazon Simple Storage Service \(Amazon S3\)](#) armazena ativos estáticos, como arquivos CSS e React, para JavaScript o site.



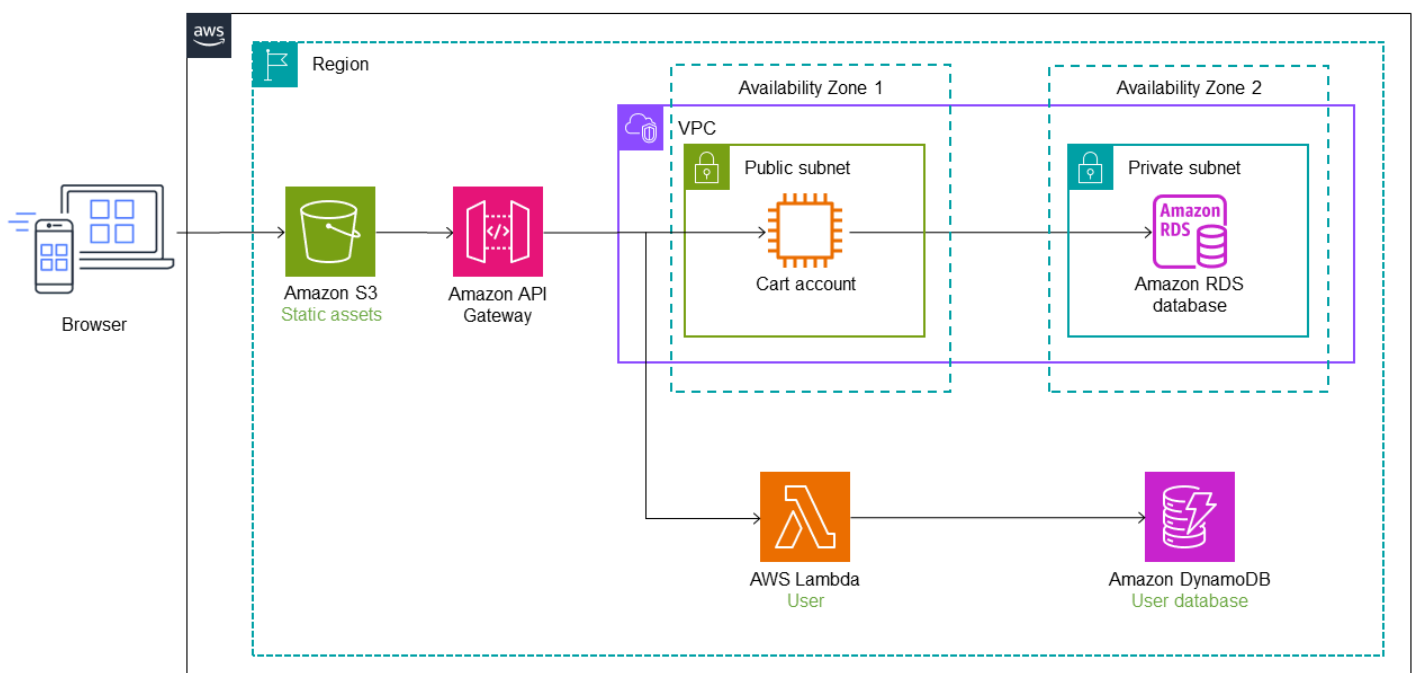
Na arquitetura a seguir, o [AWS Migration Hub Refactor Spaces](#) implanta o [Amazon API Gateway](#) na frente da aplicação monolítica. O Refactor Spaces cria uma infraestrutura de refatoração dentro da sua conta, e o API Gateway atua como a camada de proxy para rotear chamadas para o monólito. Inicialmente, todas as chamadas são roteadas para a aplicação monolítica por meio da camada de proxy. Conforme analisado anteriormente, as camadas de proxy podem se tornar um único ponto de falha. No entanto, usar o API Gateway como proxy reduz o risco, pois é um serviço sem servidor. Multi-AZ

Note

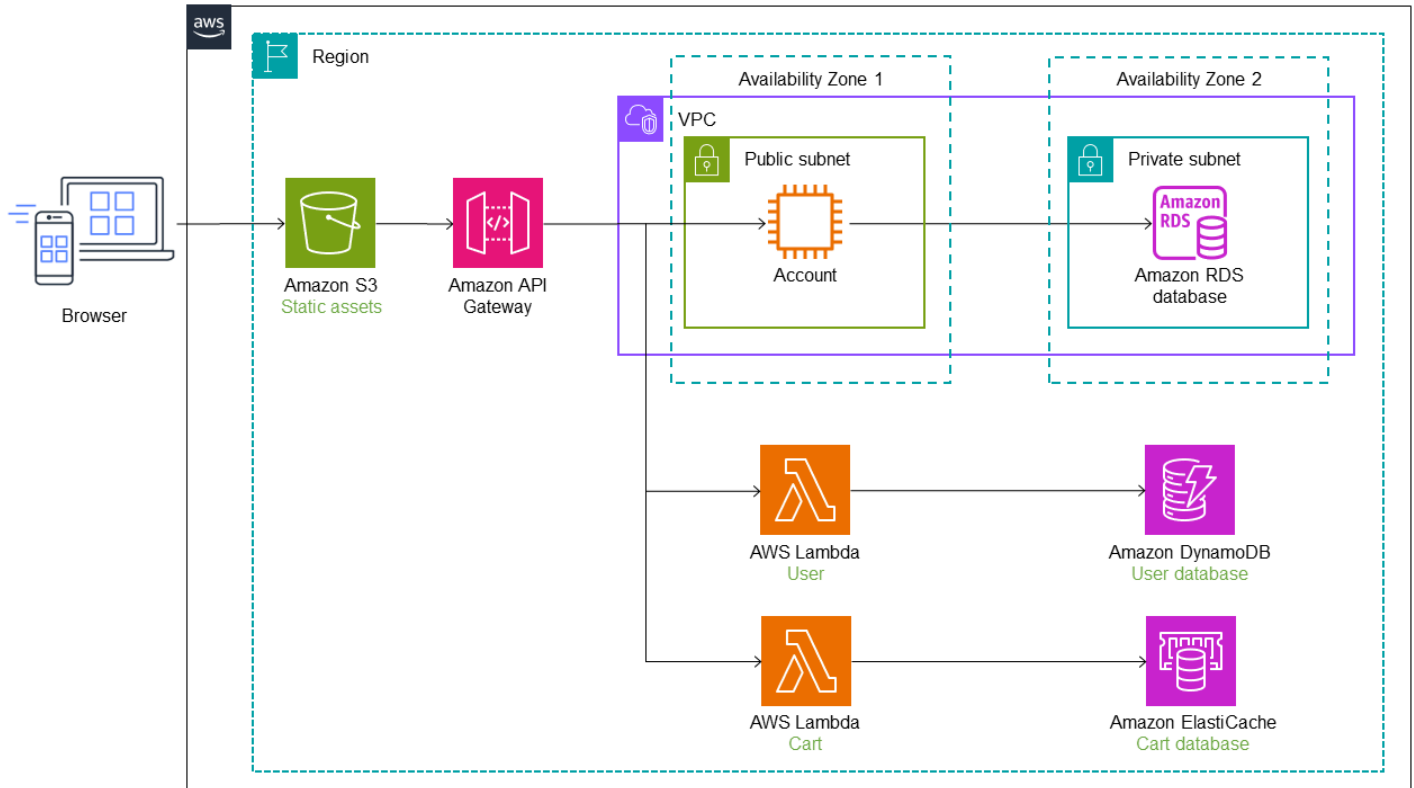
AWS Migration Hub Refactor Spaces não está mais aberto a novos clientes a partir de 7 de novembro de 2025. Para recursos semelhantes a AWS Migration Hub Refactor Spaces, explore [AWS Transform](#).



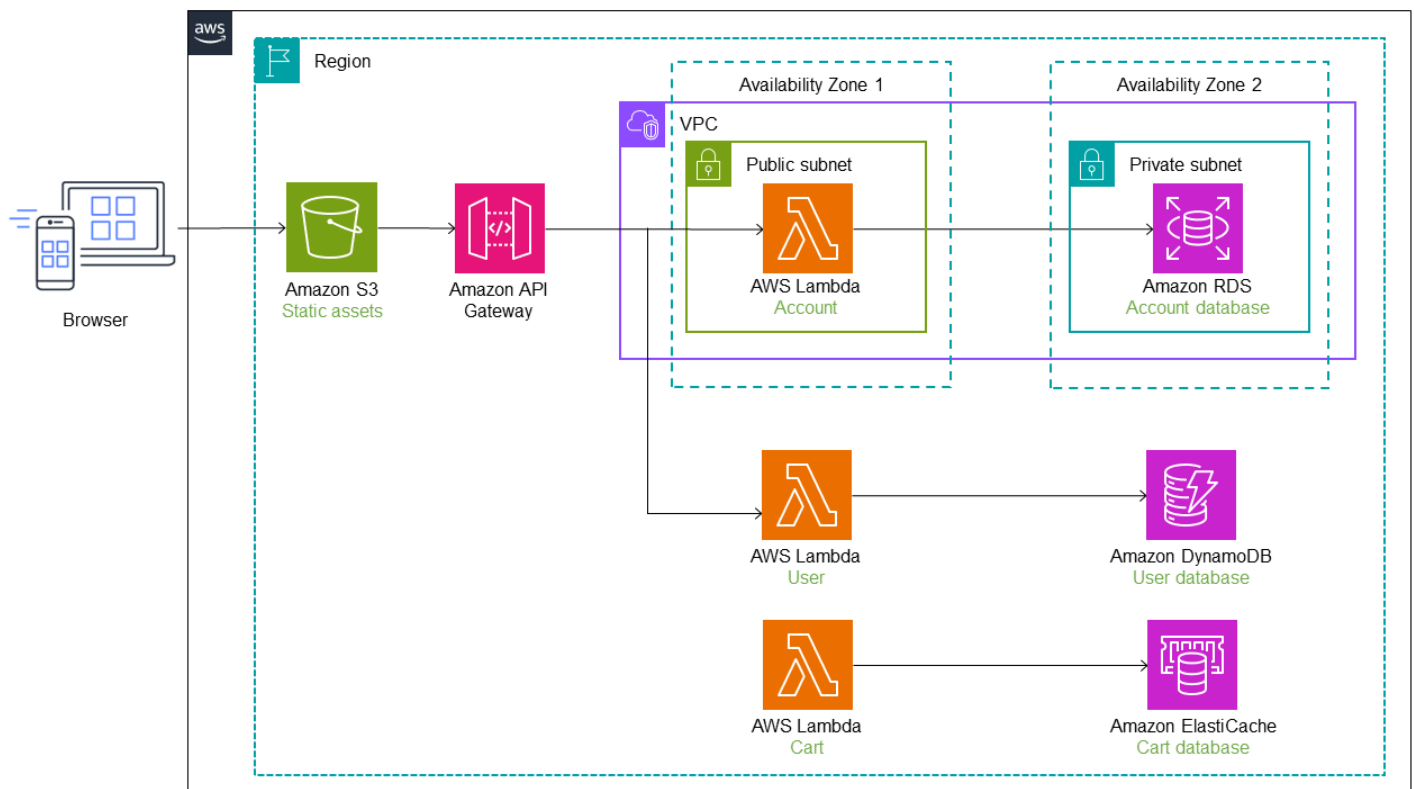
O serviço do usuário é migrado para uma função do Lambda, e um banco de dados do [Amazon DynamoDB](#) armazena seus dados. Um endpoint de serviço do Lambda e uma rota padrão são adicionados ao Refactor Spaces, e o API Gateway é configurado automaticamente para rotear as chamadas para a função do Lambda.



No diagrama a seguir, o serviço de carrinho também foi migrado do monólito para uma função do Lambda. Uma rota adicional e um endpoint de serviço são adicionados ao Refactor Spaces, e o tráfego passa automaticamente para a função Cart do Lambda. [O armazenamento de dados da função Lambda é gerenciado pela Amazon. ElastiCache](#) A aplicação monolítica ainda permanece na instância do EC2 junto com o banco de dados do Amazon RDS.



No diagrama a seguir, o último serviço (conta) é migrado do monólito para uma função do Lambda. Ele continua usando o banco de dados original do Amazon RDS. A nova arquitetura agora tem três microsserviços com bancos de dados separados. Cada serviço usa um tipo diferente de banco de dados. Esse conceito de usar banco de dados com propósito específico para atender às necessidades específicas dos microsserviços é denominado persistência poliglota. As funções do Lambda também podem ser implementadas em diferentes linguagens de programação, conforme determinado pelo caso de uso. Durante a refatoração, o Refactor Spaces automatiza a substituição e o roteamento do tráfego para o Lambda. Isso economiza para seus criadores o tempo necessário para arquitetar, implantar e configurar a infraestrutura de roteamento.



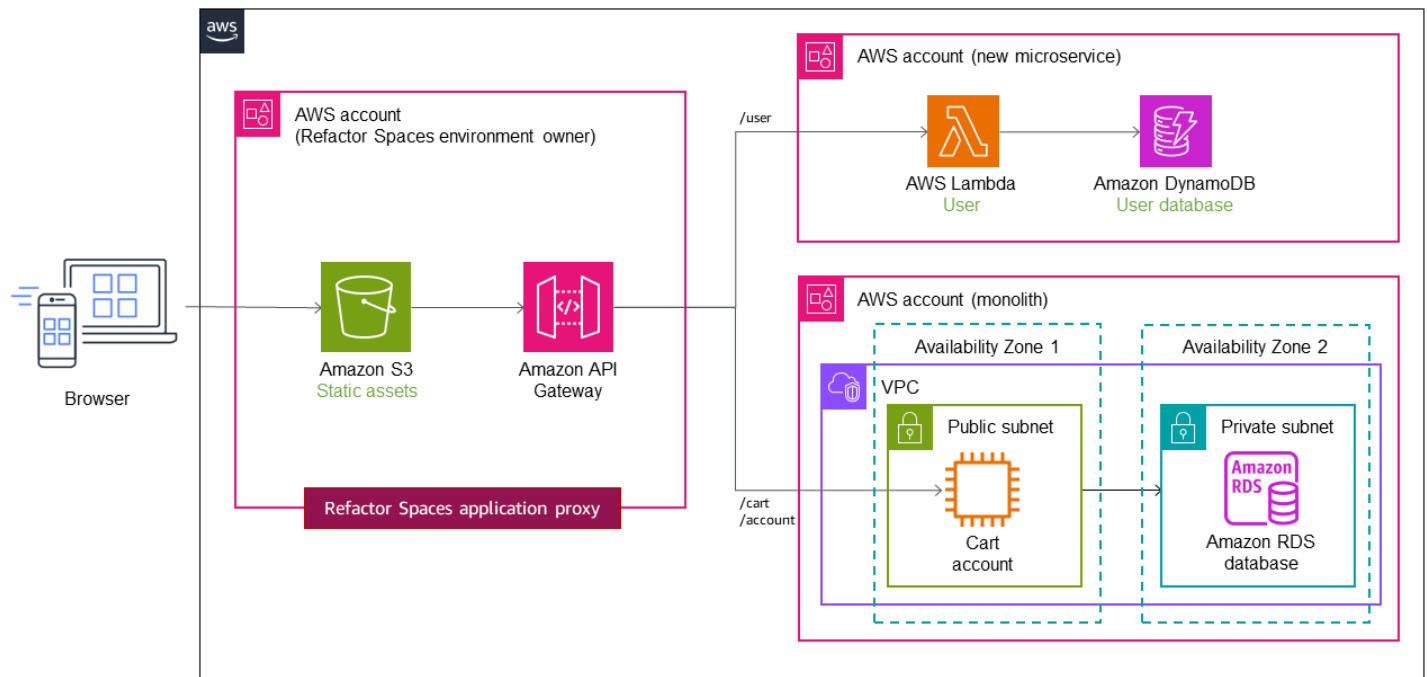
Uso de várias contas

Na implementação anterior, usamos uma única VPC com uma sub-rede pública e uma privada para a aplicação monolítica, e implantamos os microserviços dentro da mesma Conta da AWS por uma questão de simplicidade. No entanto, esse raramente é o caso em cenários do mundo real, em que os microserviços geralmente são implantados em várias Contas da AWS para independência de implantação. Em uma estrutura de várias contas, você precisa configurar o tráfego de roteamento do monólito para os novos serviços em contas diferentes.

O [Refactor Spaces](#) ajuda você a criar e configurar a AWS infraestrutura para rotear chamadas de API fora do aplicativo monolítico. O Refactor Spaces orquestra o [API Gateway](#), o [Network Load Balancer](#) e as políticas do [AWS Identity and Access Management \(IAM\)](#) baseadas em recursos nas suas contas da AWS como parte de seu recurso de aplicação. Você pode adicionar novos serviços de forma transparente em uma única conta Conta da AWS ou em várias contas a um endpoint HTTP externo. Todos esses recursos são orquestrados dentro do seu Conta da AWS e podem ser personalizados e configurados após a implantação.

Suponhamos que os serviços de usuário e carrinho de compras sejam implantados em duas contas diferentes, conforme mostrado no diagrama a seguir. Ao usar o Refactor Spaces, você só precisa configurar o endpoint do serviço e a rota. O Refactor Spaces automatiza a integração [API Gateway](#)–

[Lambda](#) e a criação de políticas de recursos do Lambda, para que você possa se concentrar na refatoração segura de serviços fora do monólito.



Para ver um tutorial em vídeo sobre como usar o Refactor Spaces, consulte [Refactor Apps Incrementally with AWS Migration Hub Refactor Spaces](#).

Workshop

- [Iterative app modernization workshop](#)

Referências do blog

- [AWS Migration Hub Refactor Spaces](#)
- [Deep Dive on an AWS Migration Hub Refactor Spaces](#)
- [Deployment Pipelines Reference Architecture and Reference Implementations](#)

Conteúdo relacionado

- [Padrões de roteamento da API](#)
- [Documentação do Refactor Spaces](#)

Padrão de caixa de saída transacional

Intenção

O padrão de caixa de saída transacional resolve o problema de operações de gravação dupla que ocorre em sistemas distribuídos quando uma única operação envolve uma operação de gravação no banco de dados e uma notificação de mensagem ou evento. Uma operação de gravação dupla ocorre quando um aplicativo grava em dois sistemas diferentes; por exemplo, quando um microsserviço precisa manter os dados no banco de dados e enviar uma mensagem para notificar outros sistemas. Uma falha em uma dessas operações pode resultar em dados inconsistentes.

Motivação

Quando um microsserviço envia uma notificação de evento após uma atualização do banco de dados, essas duas operações devem ser executadas atomicamente para garantir a consistência e a confiabilidade dos dados.

- Se a atualização do banco de dados for bem-sucedida, mas a notificação do evento falhar, o serviço seguinte não estará ciente da alteração e o sistema poderá entrar em um estado inconsistente.
- Se a atualização do banco de dados falhar, mas a notificação do evento for enviada, os dados poderão ser corrompidos, o que poderá afetar a confiabilidade do sistema.

Aplicabilidade

Use o padrão de caixa de saída transacional quando:

- Você estiver criando um aplicativo orientado por eventos em que uma atualização do banco de dados inicia uma notificação de evento.
- Você quiser garantir a atomicidade em operações que envolvem dois serviços.
- Você quiser implementar o [padrão de fornecimento de eventos](#).

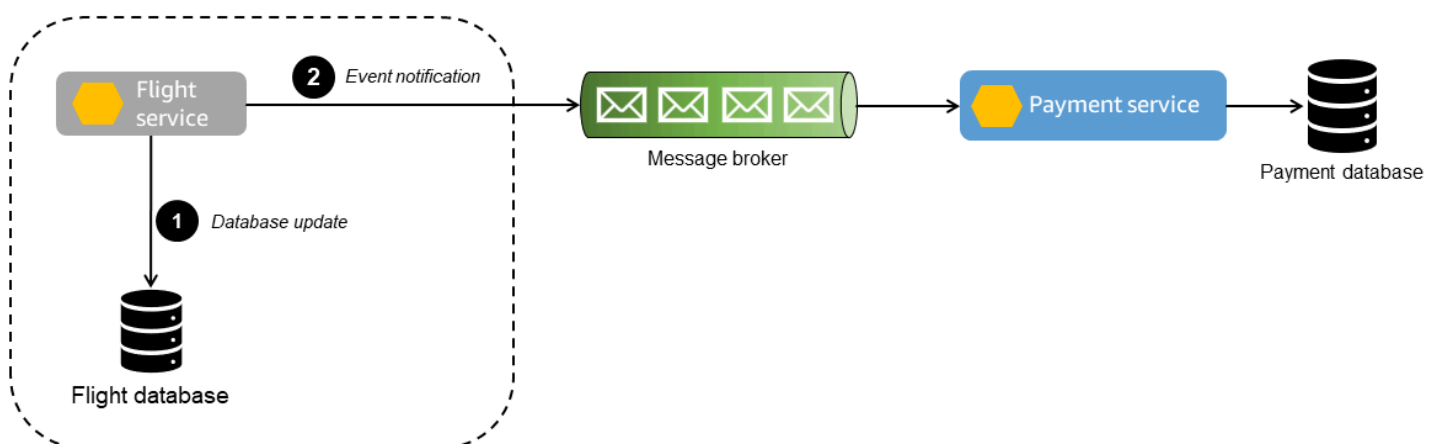
Problemas e considerações

- Mensagens duplicadas: o serviço de processamento de eventos pode enviar mensagens ou eventos duplicados, portanto, recomendamos que você torne o serviço consumidor idempotente rastreando as mensagens processadas.
- Ordem de notificação: envie mensagens ou eventos na mesma ordem em que o serviço atualiza o banco de dados. Isso é fundamental para o padrão de fornecimento de eventos, em que você pode usar um repositório de eventos para a recuperação pontual do armazenamento de dados. Se a ordem estiver incorreta, isso poderá comprometer a qualidade dos dados. A consistência eventual e a reversão do banco de dados podem agravar o problema se a ordem das notificações não for preservada.
- Reversão da transação: não envie uma notificação de evento se a transação for revertida.
- Service-level tratamento de transações: se a transação abranger serviços que exigem atualizações do armazenamento de dados, use o [padrão de orquestração da saga](#) para preservar a integridade dos dados nos armazenamentos de dados.

Implementação

High-level arquitetura

O diagrama de sequência a seguir mostra a ordem dos eventos que acontecem durante as operações de gravação dupla.



1. O serviço de voo grava no banco de dados e envia uma notificação de evento para o serviço de pagamento.

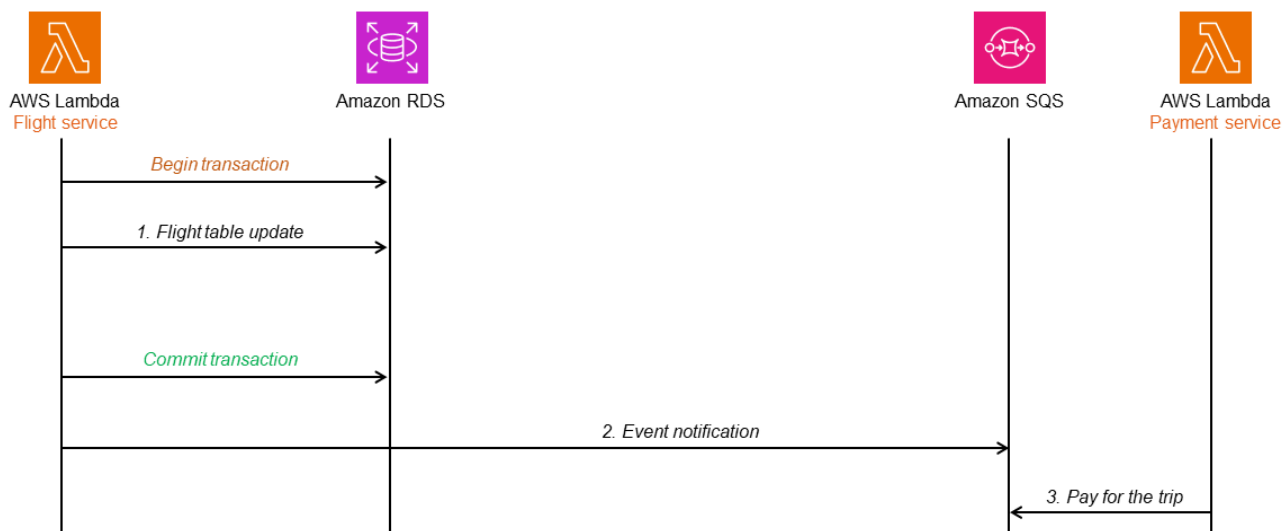
2. O agente de mensagens transporta as mensagens e os eventos para o serviço de pagamento. Qualquer falha no agente de mensagens impede que o serviço de pagamento receba as atualizações.

Se a atualização do banco de dados de voos falhar, mas a notificação for enviada, o serviço de pagamento processará o pagamento com base na notificação do evento. Isso causará inconsistências de dados posteriores.

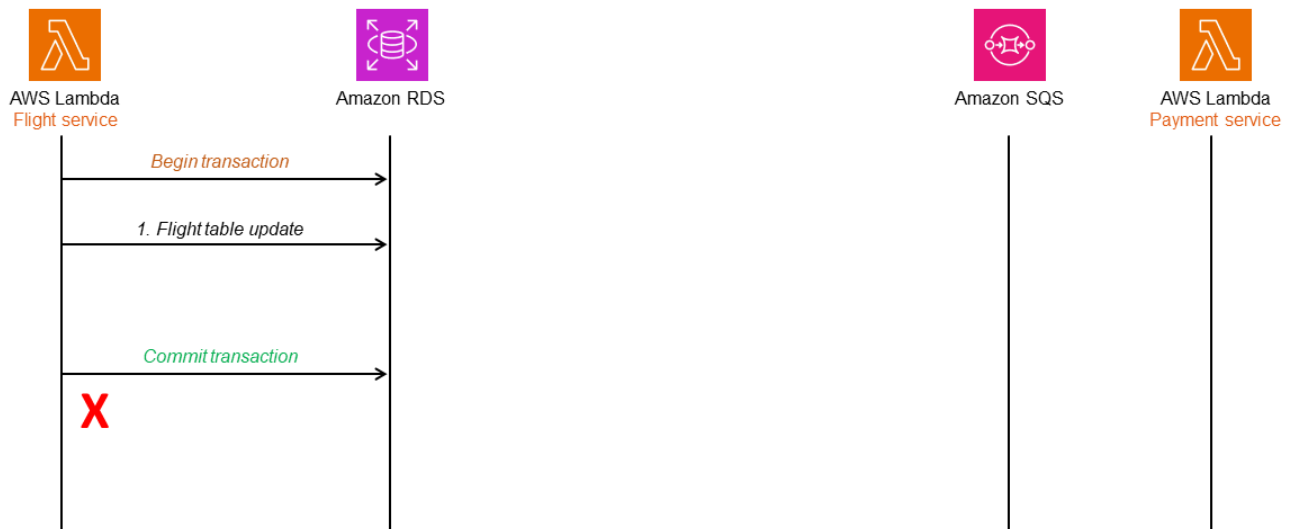
Implementação usando AWS serviços

Para demonstrar o padrão no diagrama de sequência, usaremos os serviços abaixo da AWS , conforme mostrado no diagrama a seguir.

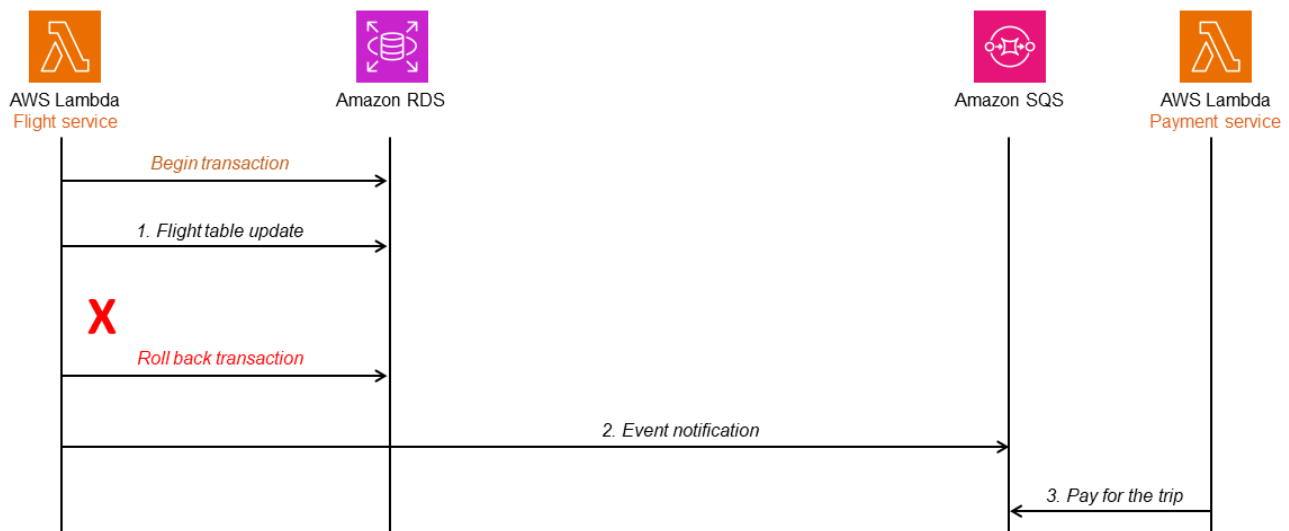
- Os microsserviços são implementados usando [AWS Lambda](#).
- O banco de dados primário é gerenciado pelo [Amazon Relational Database Service \(Amazon RDS\)](#).
- O [Amazon Simple Queue Service \(Amazon SQS\)](#) atua como o agente de mensagens que recebe notificações de eventos.



Se o serviço de voo falhar após a confirmação da transação, isso poderá fazer com que a notificação do evento não seja enviada.



No entanto, a transação pode falhar e ser revertida, mas a notificação do evento ainda pode ser enviada, fazendo com que o serviço de pagamento processe o pagamento.



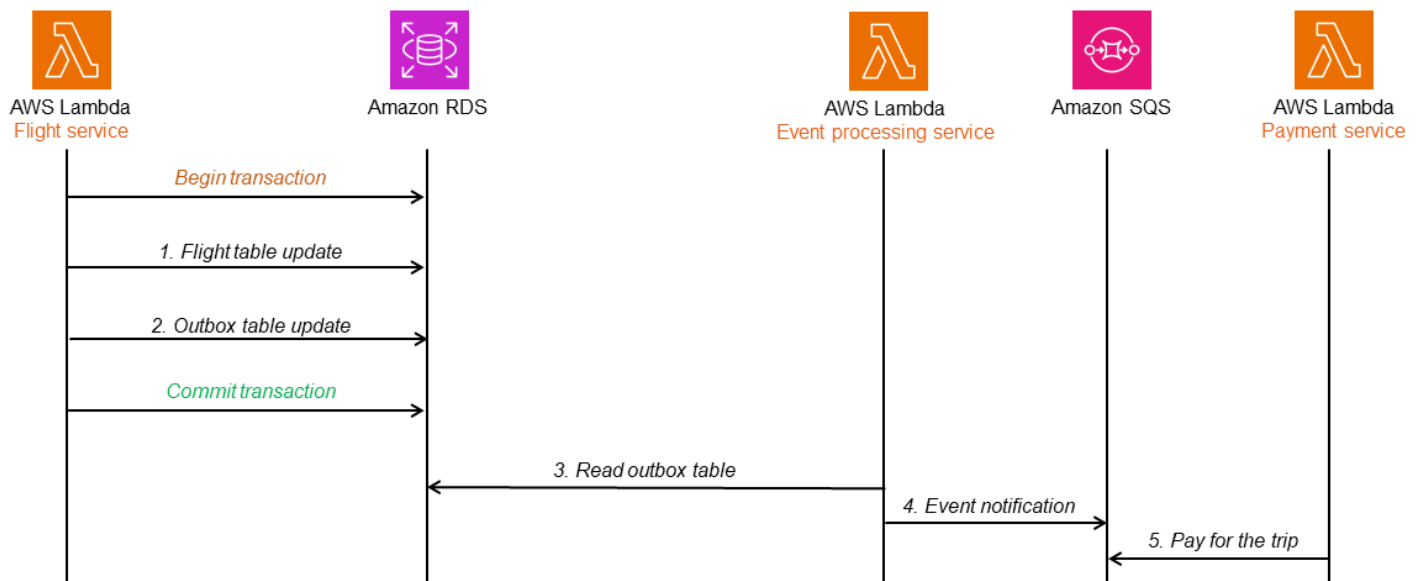
Para resolver esse problema, você pode usar uma tabela de caixa de saída ou captura de dados de alteração (CDC). As seções a seguir discutem essas duas opções e como você pode implementá-las usando os serviços da AWS.

Usar uma tabela de caixa de saída com um banco de dados relacional

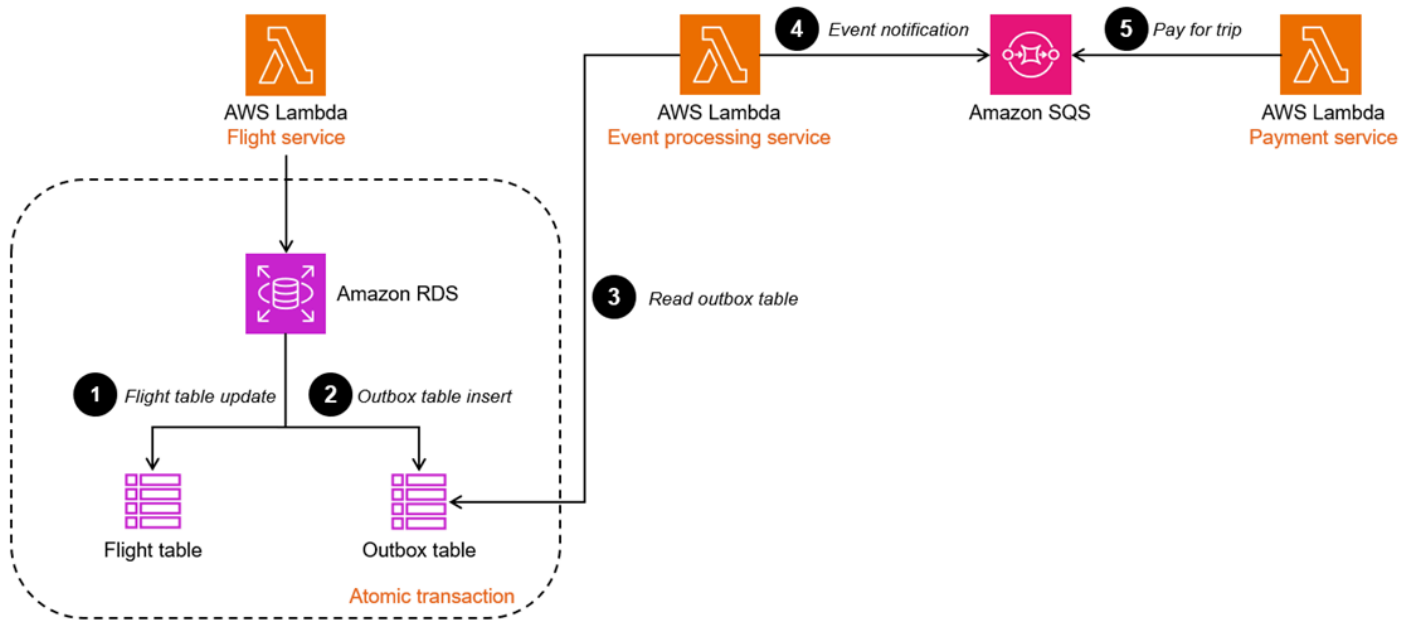
Uma tabela de caixa de saída armazena todos os eventos do serviço de voo com um registro de data e hora e um número de sequência.

Quando a tabela de voos é atualizada, a tabela da caixa de saída também é atualizada na mesma transação. Outro serviço (por ex., o serviço de processamento de eventos) lê a tabela da caixa de saída e envia o evento para o Amazon SQS. O Amazon SQS envia uma mensagem sobre o evento ao serviço de pagamento para processamento adicional. As [filas padrão do Amazon SQS](#) garantem que a mensagem seja entregue pelo menos uma vez e não se perca. No entanto, quando você usa as filas padrão do Amazon SQS, a mesma mensagem ou evento pode ser entregue mais de uma vez, portanto, você deve garantir que o serviço de notificação de eventos seja idempotente (ou seja, processar a mesma mensagem várias vezes não deve ter um efeito adverso). Se você precisar que a mensagem seja processada exatamente uma vez, com a ordenação de mensagens, você pode usar as filas [FIFO \(first in, first out\) do Amazon SQS](#).

Se houver falha na atualização da tabela de voo ou da tabela da caixa de saída, toda a transação será revertida, portanto, não haverá inconsistências de dados posteriores.



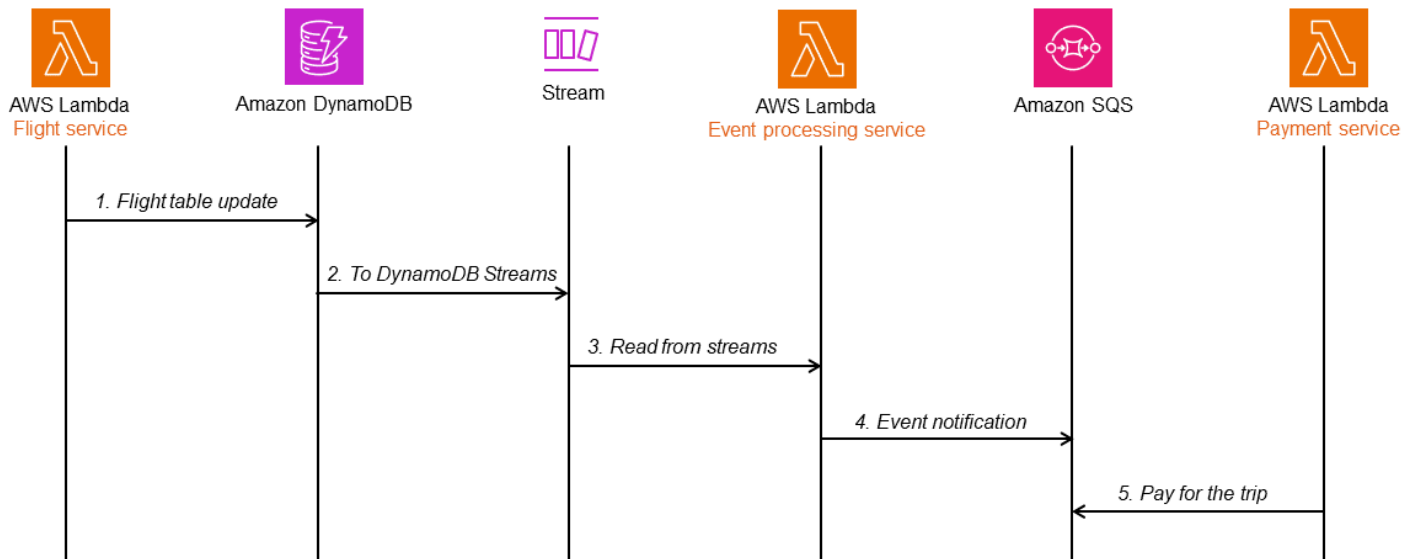
No diagrama a seguir, a arquitetura de caixa de saída transacional é implementada usando um banco de dados do Amazon RDS. Quando o serviço de processamento de eventos lê a tabela da caixa de saída, ele reconhece somente as linhas que fazem parte de uma transação confirmada (bem-sucedida) e, em seguida, coloca a mensagem do evento na fila do SQS, que é lida pelo serviço de pagamento para processamento adicional. Esse design resolve o problema das operações de gravação dupla e preserva a ordem das mensagens e dos eventos usando registros de data e hora e números de sequência.



Uso da captura de dados de alteração (CDC)

Alguns bancos de dados oferecem suporte à publicação de modificações em nível de item para capturar dados alterados. Você pode identificar os itens alterados e enviar uma notificação de evento adequadamente. Isso economiza a sobrecarga de criar outra tabela para rastrear as atualizações. O evento iniciado pelo serviço de voo é armazenado em outro atributo do mesmo item.

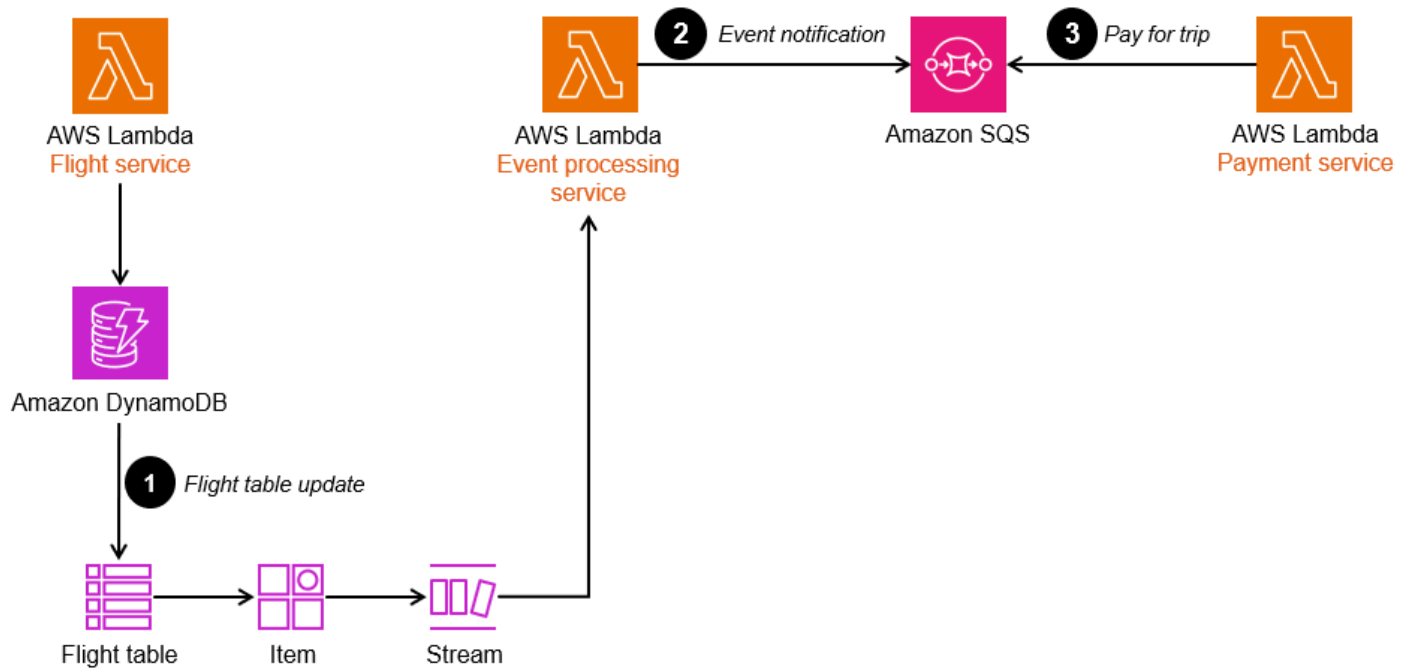
O [Amazon DynamoDB](#) é um banco de dados NoSQL de valor chave que oferece suporte às atualizações do CDC. No diagrama de sequência a seguir, o DynamoDB publica modificações em nível de item no Amazon DynamoDB Streams. O serviço de processamento de eventos lê os fluxos e publica a notificação do evento para o serviço de pagamento para processamento adicional.



O DynamoDB Streams captura o fluxo de informações relacionadas às mudanças no nível do item em uma tabela do DynamoDB usando uma sequência ordenada por tempo.

Você pode implementar um padrão de caixa de saída transacional ativando fluxos na tabela do DynamoDB. A função do Lambda para o serviço de processamento de eventos está associada a esses fluxos.

- Quando a tabela de voos é atualizada, os dados alterados são capturados pelo DynamoDB Streams, e o serviço de processamento de eventos pesquisa o fluxo em busca de novos registros.
- Quando novos registros de fluxo são disponibilizados, a função do Lambda coloca a mensagem do evento de forma síncrona na fila do SQS para processamento adicional. Você pode adicionar um atributo ao item do DynamoDB para capturar o registro de data e hora e o número de sequência conforme necessário para melhorar a robustez da implementação.



Código de exemplo

Uso de uma tabela de outbox

O código de exemplo a seguir mostra como você pode implementar o padrão outbox transacional usando uma tabela de outbox. Para ver o código completo, consulte o [GitHubrepositório](#) deste exemplo.

O trecho de código a seguir salva a entidade `Flight` e o evento `Flight` no banco de dados em suas respectivas tabelas em uma única transação.

```
@PostMapping("/flights")
@Transactional
public Flight createFlight(@Valid @RequestBody Flight flight) {
    Flight savedFlight = flightRepository.save(flight);
    JsonNode flightPayload = objectMapper.convertValue(flight, JsonNode.class);
    FlightOutbox outboxEvent = new FlightOutbox(flight.getId().toString(),
        FlightOutbox.EventType.FLIGHT_BOOKED,
        flightPayload);
    outboxRepository.save(outboxEvent);
    return savedFlight;
}
```

Um serviço separado é responsável por escanear regularmente a tabela de caixa de saída em busca de novos eventos, enviá-los para o Amazon SQS e excluí-los da tabela se o Amazon SQS responder com sucesso. A taxa de pesquisa é configurável no arquivo `application.properties`.

```
@Scheduled(fixedDelayString = "${sqs.polling_ms}")
public void forwardEventsToSQS() {
    List<FlightOutbox> entities =
outboxRepository.findAllByOrderByIdAsc(Pageable.ofSize(batchSize)).toList();
    if (!entities.isEmpty()) {
        GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
            .queueName(sqsQueueName)
            .build();
        String queueUrl = this.sqsClient.getQueueUrl(getQueueRequest).queueUrl();
        List<SendMessageBatchRequestEntry> messageEntries = new ArrayList<>();
        entities.forEach(entity ->
messageEntries.add(SendMessageBatchRequestEntry.builder()
            .id(entity.getId().toString())
            .messageGroupId(entity.getAggregateId())
            .messageDeduplicationId(entity.getId().toString())
            .messageBody(entity.getPayload().toString())
            .build())
        );
        SendMessageBatchRequest sendMessageBatchRequest =
SendMessageBatchRequest.builder()
            .queueUrl(queueUrl)
            .entries(messageEntries)
            .build();
        sqsClient.sendMessageBatch(sendMessageBatchRequest);
        outboxRepository.deleteAllInBatch(entities);
    }
}
```

Uso da captura de dados de alteração (CDC)

O código de exemplo desta seção mostra como você pode implementar o padrão outbox transacional usando os recursos de captura de dados de alteração (CDC) do DynamoDB. Para ver o código completo, consulte o [GitHub repositório](#) deste exemplo.

O trecho de código do AWS Cloud Development Kit (AWS CDK) a seguir cria uma tabela de voos do DynamoDB e um fluxo de dados do Amazon Kinesis (`cdcStream`) e configura a tabela de voos para enviar todas as atualizações para o stream.

```

Const cdcStream = new kinesis.Stream(this, 'flightsCDCStream', {
    streamName: 'flightsCDCStream'
})

const flightTable = new dynamodb.Table(this, 'flight', {
    tableName: 'flight',
    kinesisStream: cdcStream,
    partitionKey: {
        name: 'id',
        type: dynamodb.AttributeType.STRING,
    }
});

```

O trecho de código e a configuração a seguir definem uma função do Spring Cloud Stream que coleta as atualizações no fluxo do Kinesis e encaminha esses eventos para uma fila do SQS para processamento adicional.

```

applications.properties
spring.cloud.stream.bindings.sendToSQS-in-0.destination=${kinesisstreamname}
spring.cloud.stream.bindings.sendToSQS-in-0.content-type=application/ddb

QueueService.java
@Bean
public Consumer<Flight> sendToSQS() {
    return this::forwardEventsToSQS;
}

public void forwardEventsToSQS(Flight flight) {
    GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
        .queueName(sqsQueueName)
        .build();
    String queueUrl = this.sqsClient.getQueueUrl(getQueueRequest).queueUrl();
    try {
        SendMessageRequest send_msg_request = SendMessageRequest.builder()
            .queueUrl(queueUrl)
            .messageBody(objectMapper.writeValueAsString(flight))
            .messageGroupId("1")
            .messageDeduplicationId(flight.getId().toString())
            .build();
        sqsClient.sendMessage(send_msg_request);
    } catch (IOException | AmazonServiceException e) {

```

```
        logger.error("Error sending message to SQS", e);  
    }  
}
```

GitHub repositório

Para uma implementação completa da arquitetura de amostra desse padrão, consulte o GitHub repositório em <https://github.com/aws-samples/transactional-outbox-pattern>.

Recursos

Referências

- [AWS Centro de Arquitetura da](#)
- [AWS Centro do desenvolvedor da](#)
- [A Amazon Builders' Library](#)

Ferramentas

- [AWS Well-Architected Tool](#)
- [AWS App2Container](#)
- [AWS Microservice Extractor for .NET](#)

Metodologias

- [The Twelve-Factor App](#) (ePub de Adam Wiggins)
- Nygard, Michael T. [Release It!: Design and Deploy Production-Ready Software](#). 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.
- [Polyglot Persistence](#) (publicação do blog de Martin Fowler)
- [StranglerFigApplication](#) (publicação do blog de Martin Fowler)

Histórico do documento

A tabela a seguir descreve alterações significativas feitas neste guia. Se desejar receber notificações sobre futuras atualizações, inscreva-se em um [feed RSS](#).

Alteração	Descrição	Data
Novos padrões	Foram adicionados dois novos padrões: arquitetura hexagonal e scatter-gather .	7 de maio de 2024
Novos exemplos de código	Foi adicionado o código de exemplo para o caso de uso de captura de dados de alteração (CDC) ao padrão de caixa de saída transacional.	23 de fevereiro de 2024
Novos exemplos de código	- Atualizou o padrão da caixa de saída transacional com código de amostra. - Foi removida a seção sobre padrões de orquestração e coreografia, que foi substituída pela coreografia saga e pela orquestração de saga.	16 de novembro de 2023
Novos padrões	Foram adicionados três novos padrões: coreografia saga , publicar/assinar e fornecimento de eventos .	14 de novembro de 2023
Atualizar	Foi atualizada a seção de implementação do padrão strangler fig .	2 de outubro de 2023

Publicação inicial

Essa primeira versão inclui oito padrões de design: camada anticorrupção (ACL), roteamento de API, disjuntor, orquestração e coreografia, repetição com recuo, orquestração de saga, strangler fig e caixa de saída transacional.

28 de julho de 2023

As traduções são geradas por tradução automática. Em caso de conflito entre o conteúdo da tradução e da versão original em inglês, a versão em inglês prevalecerá.